MOOCEBFR3 : Ingénierie des Données du Big Data: SGBD Objet relationnel et SQL3 Oracle

Gabriel MOPOLO-MOKE
Professeur chargé d'enseignements
Université Côte d'Azur (UCA)

2022 / 2023

Plan Général

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Plan

- **Module M3.1** : Introduction, les types de base, les types abstraits
- **Module M3.2** : Notion d'identité et les collections dans SQL3 Oracle
- **Module M3.3** : Les tables objets, les liens d'associations
- **Module M3.4** : Lien d'héritage et le Mapping objet des tables relationnelles SQL2 via les vues
- **Module M3.5** : Gestion des objets volumineux
- **Module M3.6** : PLSQL étendu aux objets complexes et Mapping Objet relationnel dans Java/JDBC

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.1 : Introduction, les types de base, les types abstraits

Module M3.1 : Introduction, les types de base, les types abstraits, les pointeurs REFs et les collections dans SQL3

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Plan

- **Module M3.1, section 1** : Introduction, les types de base
- **Module M3.1, section 2** : : Les types abstraits : Création et définition des méthodes
- **Module M3.1, section 3** : : Les types abstraits : Comparaison et compléments sur les ADT
- **Module M3.1, section 4** :: Les types abstraits : Modification, suppression, visualisation, construction des ADT

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.1, section 1 : Introduction, les types de base

Module M3.1, section 1 : Introduction, les types de base

- **Plan**
  - **Introduction**
    - **Pourquoi les Objet dans les SGBD ?**
    - **Approche d'intégration des objets dans les SGBDs**
    - **Approche d'intégration des objets dans Oracle**
  - **Les types de base(Prédéfinis)**

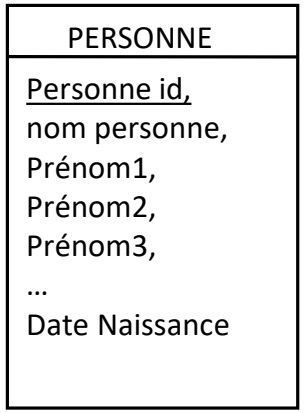
Module M3.1, section 1 : Introduction, les types de base

Introduction

Pourquoi les Objet dans les SGBD?

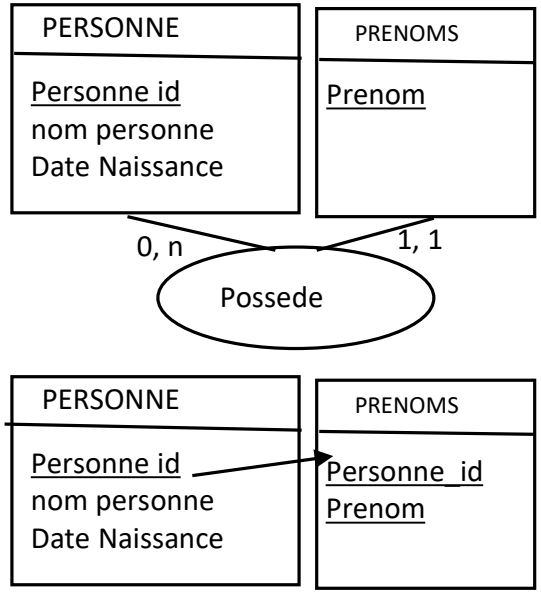
- Dépasser les limites du modèle relationnel SQL2 : contraintes liées à la normalisation (1NF)
- Eviter les jointures (1 personnes plusieurs prénoms)

Solution 1 : SQL2

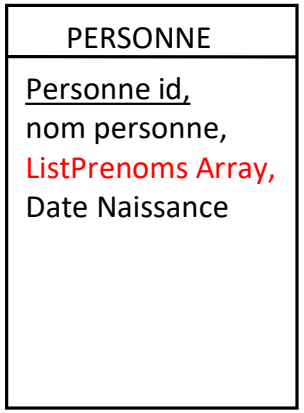


Solutions 1 et 2 possèdent des inconvénients !!!!!

Solution 2 :SQL2



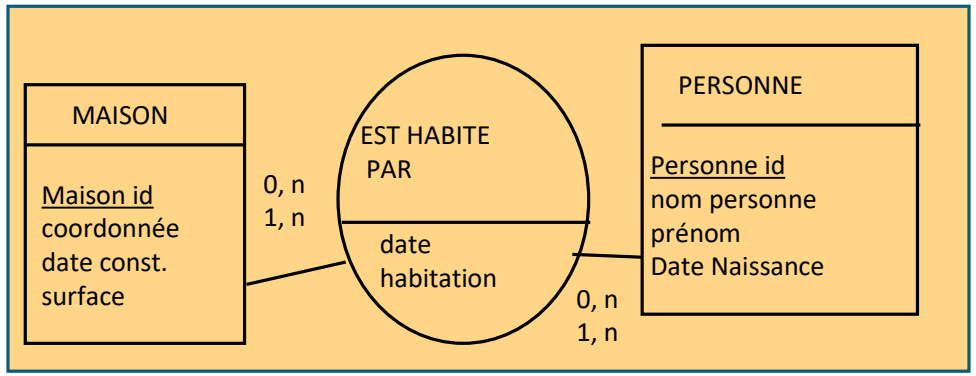
Solution 3 : SQL3



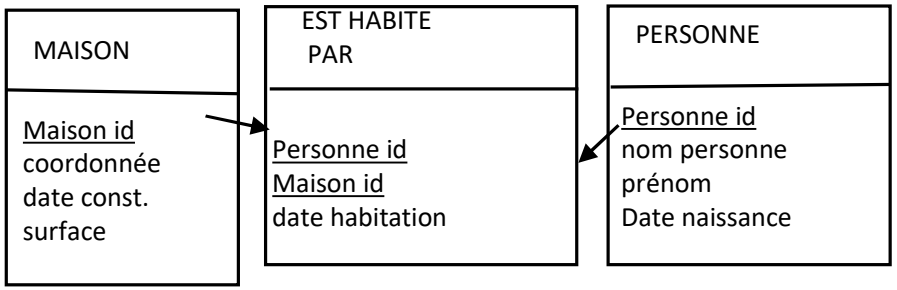
Module M3.1, section 1 : Introduction, les types de base

Introduction

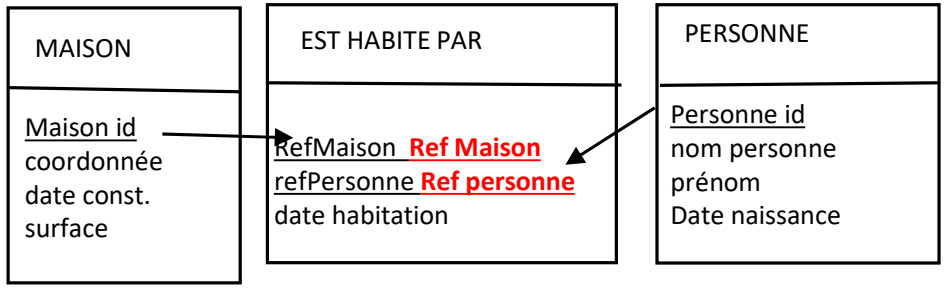
- **Pourquoi les Objet dans les SGBD?**
  - **Dépasser les limites du modèle relationnel et des SGBD relationnels SQL2 : Reduire le coût de Jointure**
  - **Eviter les tables artificielles (Une Maison est habité par 0, 1 ou N Personnes Une Personne habite 0, 1 ou N Maisons)**



Solution 1 : SQL2



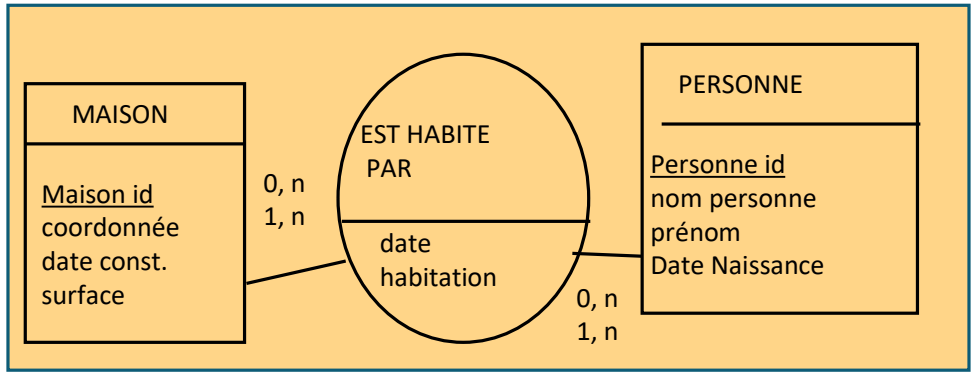
Solution 2: SQL3



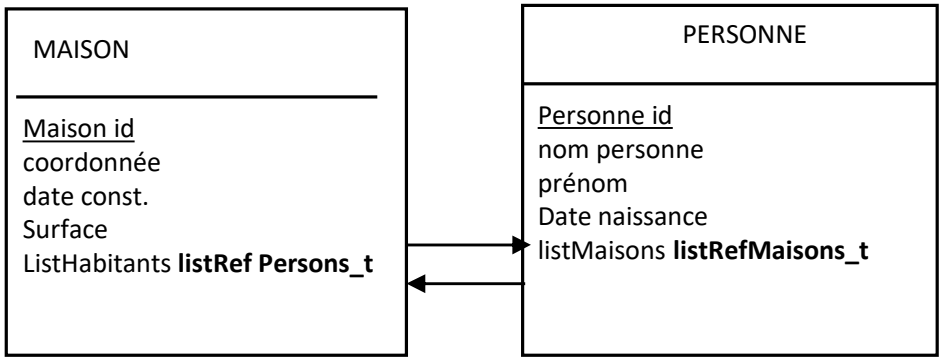
Module M3.1, section 1 : Introduction, les types de base

Introduction

- **Pourquoi les Objet dans les SGBD?**
  - **Dépasser les limites du modèle relationnel et des SGBD relationnels SQL2 : Reduire le coût de Jointure**
  - **Eviter les tables artificielles (Une Maison est habité par 0, 1 ou N Personnes Une Personne habite 0, 1 ou N Maisons)**



- Solution 3 : SQL3



- ListRefPersons_t et ListRefMaisons_t sont des collections de POINTEUR à créer

Introduction

Pourquoi les Objet dans les SGBD?

- **Modélisation des données complexes**
  - ✓ Encapsulation des données
  - ✓ Type abstraits
  - ✓ Collections (varray, nested table)
  - ✓ Liens complexes(composition, association, héritage)

- **Manipulation EFFICACE des objets volumineux non structutées LOBs (Large Objects).**
  Va au delà des blobs traditionnels (LONG, LONG ROW)

Module M3.1, section 1 : Introduction, les types de base

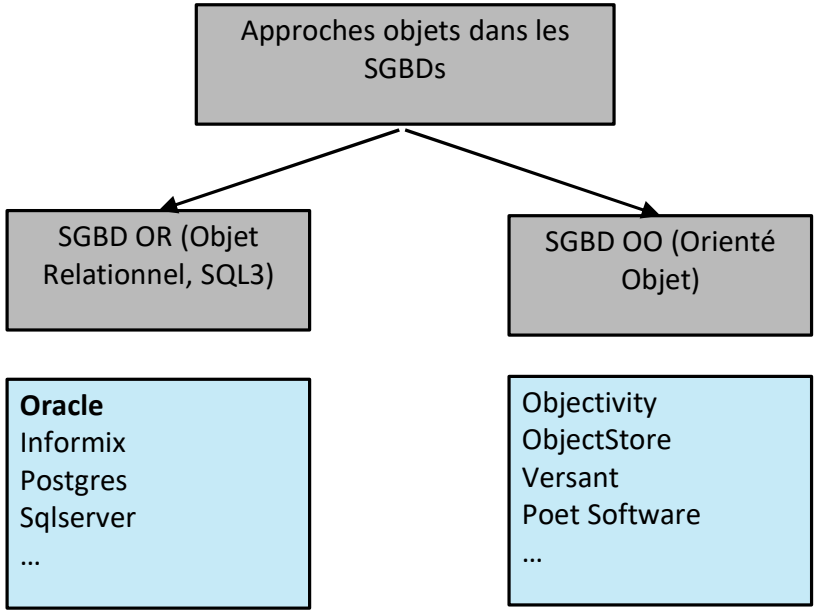
Introduction

- Pourquoi les Objet dans les SGBD?
  - **Evolutivité**
    - Exploiter de nouveaux gisements de données dans les entreprises
    - Données une vision objet aux applications relationnelles existantes
  - **Applications internet et intranet**
    - SGBD ouvert supportant Java en natif au même titre que PL/SQL avec une intégration forte dans l'environnement ORB
    - Données sémistructurées XML/JSON directement gérées par le SGBD

Module M3.1, section 1 : Introduction, les types de base

Introduction

- Approche d'intégration des objets dans les SGBDs



Module M3.1, section 1 : Introduction, les types de base

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Introduction

- **Approche d'intégration des objets dans Oracle**
  - Oracle suit **l'approche SQL3**
  - **Oracle étend son noyau relationnel vers l'objet <u>via l'extension de la notion de domaine (type):</u>**

Relation R (D1 X D2 X … X Dn)

D_i pouvant être un type scalaire ou un type complexe

Note : Les colonnes des tables peuvent maintenant être définis sur des types (domaines) simple ou complexes

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

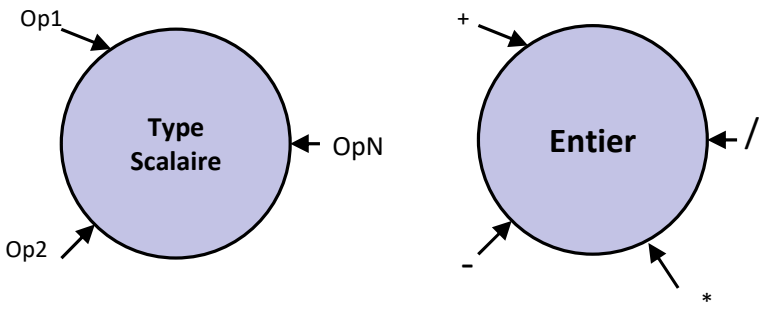
Module M3.1, section 1 : Introduction, les types de base

Introduction

- Approche d'intégration des objets dans Oracle

Justification :

Type abstrait=
Structure **opaque**
+Opération



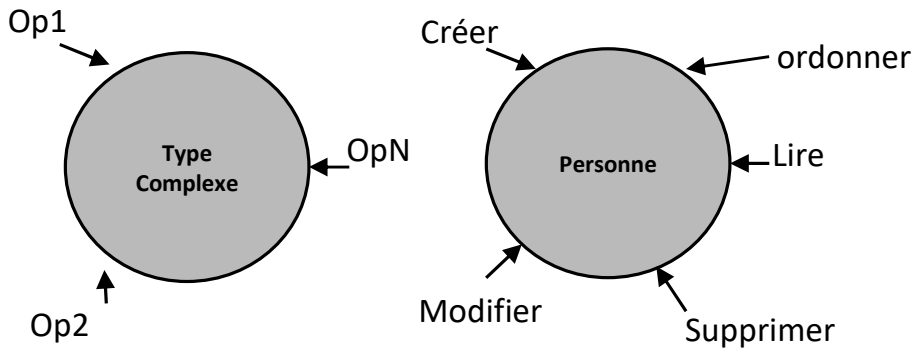
Exemple 1 : Les Types scalaires ou atomiques

Introduction

Approche d'intégration des objets dans Oracle

Justification :

Type abstrait=
Structure **complexe opaque**
+Opération

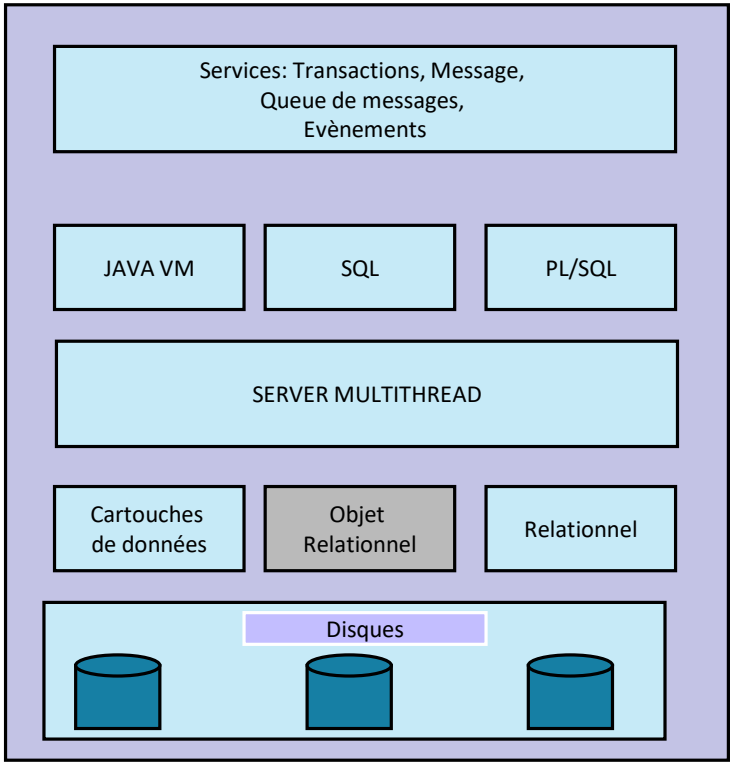


Exemple 2 : Types complexes ou atomiques relatifs (d'après C. DATE)

Module M3.1, section 1 : Introduction, les types de base

Introduction

- Approche d'intégration des objets dans Oracle



Module M3.1, section 1 : Introduction, les types de base

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Les Types de Base(PREDEFINIS)

- **Les types de base ou scalaires** (CHAR, VARCHAR2, DATE, NUMBER, ….) de la version 7 sont conservés dans les versions objets

- **Les nouveaux types**
  - Les associations référentielles "Père/Fils" (**REF**).
  - Les collections : **VARRAY (tableaux)** et **Nested Tables (Tables imbriquées).**
  - **Les LOBs** : BLOB, CLOB, BFILE

  - NOTE : L 'utilisateur pourra ajouter ses propres types: les types données abstraits (ADT).

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.1, section 1 : Quiz

- **Question 1** : Quelles sont les bénéfices de l'objet dans les BD Objet relationnel ?
  - A: Interdire l'utilisation des clés étrangères
  - B: Modéliser les objets complexes
  - C: Réduire le coût des jointures
  - D: Gérer efficacement les objets volumineux

- **Question 2** : Quels sont les nouveaux types prédéfinis en SQL3 Oracle
  - A: Personne_t
  - B: REF
  - C: Varchar2
  - D: VARRAY

- **Question 3** : Comment l'objet a-t-il été introduit en SQL3
  - A: en étendant la définition des colonnes : des colonnes simples au colonnes complexes
  - B: en étendant la notion de domaine ou type : des domaines ou types simples au domaines ou types complexes
  - C: Grâce l'atomicité relative de Chris DATE
  - D: En Introduisant la notion de classe

- **Question 4** : A quoi sert le type prédéfini : TYPE NESTED TABLE
  - A: A gérer des collections pour lesquelles nombre maximum d'éléments est connu à l'avance
  - B: A gérer des collections pour lesquelles nombre maximum d'éléments n'est pas connu à l'avance
  - C: A modéliser les liens d'associations de cardinalité N
  - D: A modéliser les liens d'associations de cardinalité 1

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

ESTIA INSTITUTE OF TECHNOLOGY

UNIVERSITÉ CÔTE D'AZUR

Plan

- Les Types de données Abstraits(ADT)
- Exemple de création d'un ADT sous Oracle
- Définition des Méthodes d'un Type Abstrait

anr

INVESTIR L'AVENIR

CC BY NC ND

DATUM ACADEMY

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

➢ Les Types de donnees Abstraits(ADT)

- Un Type abstrait de données peut être vu **comme une boite noire accessible uniquement par des opérateurs**
- Les spécifications des types sont **indépendantes** de leurs implémentations
- Permet à l'utilisateur de définir lui-même **ses propres types**

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

- **Les Types de donnees Abstraits(ADT)**

SPECIFICATION DU TYPE

DECLARATION DES ATTRIBUTS

DECLARATION DES METHODES

CORPS DU TYPE

IMPLEMENTATION DES METHODES

Les Types de donnees Abstraits(ADT)

- Syntaxe de création **d'un type Forward** ou TYPE INCOMPLET
  - permet de résoudre le problème de références croisées entre type

```
CREATE [OR REPLACE] TYPE nomType;

CREATE OR REPLACE TYPE pilote_t;
```

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

➢ Les Types de donnees Abstraits(ADT)

- Syntaxe de **création d'un type objet**

```
CREATE  [OR REPLACE]        -- récréer le type s'il existe. Echec si dépendance et pas de clause FORCE
TYPE [schema.]nomType       -- schema nom propriétaire du type
[ FORCE]     -- Force de recréer ce type même si d'autres types l'utilize. Echec si utilisé dans la definition d'une table
[ OID ]      -- 'un_object_iditientfier'. permet de créer des types identiques (BD distribuées)
[AUTHID {CURRENT USER | DEFINER}] -- Exécuter avec les droits du créateur ou ceux de celui qui exécute
[{AS | IS} OBJECT(
  nomAttribut1      datatype
  [, nomAttributN       datatype,] …
  [constructor_spec][,constructor_spec]…
  [map_order_function_spec][,map_order_function_spec]…
  [procedure_spec][, procedure_spec] …
  [function_spec][,function_spec] …
  [pragma_restrict_ref_spec][, pragma_restrict_ref_spec]…
  )]
/
```

[] : Facultatif, {} au choix

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

➢ Les Types de donnees Abstraits(ADT)

- Syntaxe de **création d'un type objet**

constructor_spec::=[FINAL][INSTANTIABLE] CONSTRUCTOR FUNCTION nomType

[([SELF IN OUT nomType,] parameter_spec [,parameter_spec]…)] RETURN SELF AS RESULT

map_order_function_spec::= {MAP | ORDER} MEMBER FUNCTION nomMethode

(parameter_spec [,parameter_spec]…) return datatype

procedure_spec::= {STATIC | MEMBER} PROCEDURE nomMethode

(parameter_spec [,parameter_spec]…)

function_spec::= {STATIC | MEMBER} FUNCTION nomMethode

(parameter_spec [,parameter_spec]…) return datatype

Parameter_spec::=parameter datatype

pragma_restrict_ref_spec::= PRAGMA RESTRICT_REFERENCES(

{nomMethode|DEFAULT}, pragma_name[,pragma_name] …)

pragma_name::={WNDS, WNPS, RNDS, RNPS, TRUST}

dataType ::= {char | varchar2 | number | user_type | …}

Notes : Les mots seront discutés au fil du cours

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

➢ Exemple de création d'un ADT sous Oracle

- Type abstrait sans champs complexes

```
CREATE OR REPLACE TYPE adresse_t as object(
        Numero          NUMBER(4),
        Rue             VARCHAR2(20),
        Code_Postal     NUMBER(5),
        Ville           VARCHAR2(20));
```

- Type abstrait avec champs complexes

```
CREATE OR REPLACE TYPE Pilote_t AS OBJECT(
    PL#         NUMBER(4),
    nom         VARCHAR2(12),
    adresse     adresse_t,-- type abstrait défini préalablement
    Tel         VARCHAR2(12),
    dnaiss      DATE,
    salaire NUMBER(7,2));
```

- ADT avec un champ de type référence

```
CREATE OR REPLACE TYPE REF_Pilote_t AS OBJECT(
    PL#         NUMBER(4),
    nom         VARCHAR2(12),
    adresse     REF adresse_t, -- référence sur une adresse
    Tel         VARCHAR2(12),
    dnaiss      DATE,
    salaire     number(7,2) );
```

Définition des Méthodes d'un Type Abstrait

- L'utilisateur peut définir **autant de méthodes** qu'ils le souhaite dans un ADT
- En plus du **CONSTRUCTEUR par défaut**: constructeur ayant tous les champs du type comme paramètre, l'utilisateur peut définir ses propres constructeurs (à partir d'Oracle V10)
- La **déclaration des méthodes se fait dans le TYPE SPECIFICATION** et l'implémentation dans le TYPE BODY
- Le mot **clé MEMBER** précède chaque méthode d'instance
- Le mot **clé STATIC** précède chaque méthode de classe

Définition des Méthodes d'un Type Abstrait

- **Une méthode se caractérise par :**
  - Un **nom**
  - **Un ensemble de paramètres** : les données en entrée(IN), les résultats (OUT), les données/résultats (IN/OUT), les appels par référence (CALL-BY-REFERENCE).
  - Un ensemble de résultats retournés par la méthode
  - Un ensemble **d'exceptions**
  - Un **ensemble d'indicateurs** précisant la nature de la méthode (MAP/ORDER MEMBER, MEMBER, STATIC).
  - Des **niveaux de protection** :PRAGMA RESTRICT_REFERENCE :WNDS, WNPS, RNDS, RNPS, TRUST)
  - Un nom spécifique (un alias).
  - Une clause de langage (**PL/SQL, C, Java**,etc…)
  - Une **implémentation** : le body du type.

Définition des Méthodes d'un Type Abstrait

- **Notes :**
  - **MAP MEMBER function** : permet de s'appuyer sur la règle de comparaison d'un champ ou d'une combinaison de champs d'un type
  - **ORDER MEMBER Function**: permet de définir un algorithme de comparaison complexe
  - **STATIC MEMBER Function** : Une méthode statique n'a pas l'objet courant comme paramètre implicite. Elle n'a pas besoin d'un objet pour être invoquée
  - Paramètres de protection des méthodes avec la directive : **PRAGMA RESTRICT_REFERENCE**
    - ✓ **WNDS** : interdit la modification de l'état de la base
    - ✓ **WNPS** : interdit la modification de l'état du package(type)
    - ✓ **RNDS** : interdit la lecture de l'état de la base
    - ✓ **RNPS** : interdit la lecture de l'état du package(type)
    - ✓ **TRUST**: désactive provisoirement les interdits positionnés

Définition des Méthodes d'un ADT

- **Exemple de définition de méthodes**

Spécification du type PILOTE

```
CREATE OR REPLACE TYPE REF_Pilote_t AS OBJECT(
    PL#         NUMBER(4),
    nom         VARCHAR2(12),
    adresse         adresse_t, -- type abstrait défini préalablement
    Tel             VARCHAR2(12),
    dnaiss          DATE,
    salaire         NUMBER(7,2),
MEMBER FUNCTION getPiloteSalaire RETURN NUMBER,
MEMBER FUNCTION setPiloteSalaire(SELF IN OUT ref_pilote_t, plNum IN NUMBER, newSalaire IN NUMBER) RETURN BOOLEAN,
MEMBER PROCEDURE testSetSalaire,
PRAGMA RESTRICT_REFERENCES(getPiloteSalaire, WNDS),
PRAGMA RESTRICT_REFERENCES(setPiloteSalaire,RNPS)
);
/
```

Définition des Méthodes d'un ADT

- **Exemple d'implémentation de méthodes**

```
-- Implémentation du corps des méthodes getPiloteSalaire et setPiloteSalaire

CREATE OR REPLACE TYPE BODY ref_pilote_t  IS
MEMBER FUNCTION getPiloteSalaire RETURN NUMBER IS
        BEGIN
                RETURN salaire;
        END;
MEMBER FUNCTION setPiloteSalaire (SELF IN OUT ref_pilote_t,
plNum IN NUMBER,
        newSalaire   IN NUMBER) RETURN BOOLEAN IS
```

```
        BEGIN
                IF pl# = plNum THEN
                        salaire := newSalaire;
                        return true;
                ELSE
                        return false;
                END IF;
        END;
MEMBER PROCEDURE testSetSalaire IS retour          BOOLEAN;
        BEGIN
                retour := Self.setPiloteSalaire(10, 10000);
        END;
END;
/
```

Module M3.1, section 2 : : Les types abstraits : Création et définition des méthodes

- Définition des Méthodes d'un ADT
  - **Autre exemple**

```
CREATE TYPE Complex AS OBJECT (
  rpart REAL,
  ipart REAL,
  MEMBER FUNCTION plus (x Complex) RETURN Complex,
  MEMBER FUNCTION less (x Complex) RETURN Complex,
  MEMBER FUNCTION times (x Complex) RETURN Complex,
  MEMBER FUNCTION divby (x Complex) RETURN Complex
);
/
```

```
CREATE TYPE BODY Complex AS
  MEMBER FUNCTION plus (x Complex) RETURN Complex IS
  BEGIN
      RETURN Complex(rpart + x.rpart, ipart + x.ipart);
  END plus;
  MEMBER FUNCTION less (x Complex) RETURN Complex IS
  BEGIN
    RETURN Complex(rpart - x.rpart, ipart - x.ipart);
  END less;

      -- A SUIVRE
```

Définition des Méthodes d'un ADT

Autre exemple

```
-- Suite
MEMBER FUNCTION times (x Complex) RETURN Complex IS
  BEGIN
    RETURN Complex(rpart * x.rpart - ipart * x.ipart,
                   rpart * x.ipart + ipart * x.rpart);
  END times;
  MEMBER FUNCTION divby (x Complex) RETURN Complex IS
    z REAL := x.rpart**2 + x.ipart**2;
  BEGIN
    RETURN Complex((rpart * x.rpart + ipart * x.ipart) / z,
                   (ipart * x.rpart - rpart * x.ipart) / z);
  END divby;
END;
/
```

Module M3.1, section 2 :: Quiz

- **Question 1**: Cochez ce qui caractérise le mieux un type de données abstrait
  - A: Boîte noire accessible uniquement à travers des opérations
  - B: Possède une partie spécification et une partie corps (BODY)
  - C: Le nombre de champs ne peut dépasser le millier
  - D: Il est impossible d'en créer sans méthodes (fonctions, procédures)

- **Question 2**: cochez comment invoquer une méthode précédée du mot clé MEMBER
  - A: Pour l'invoquer il faut faire nomType.nomMéthode;
  - B: Declare var1 nomType; begin var1.nomMéthode; end;

- **Question 3**: cochez comment invoquer une méthode précédée du mot clé STATIC
  - A: Pour l'invoquer il faut faire nomType.nomMéthode;
  - B: Declare var1 nomType; begin var1.nomMéthode; end;

- **Question 4**: Le corps d'un type contient
  - A: Des variables globales
  - B: Le corps de chaque méthode
  - C: Les champs du type et le corps de chaque méthode

- **Question 5**: J'ai créé un type avec 5 méthodes avant de commencer à les utiliser, je dois
  - A: Les coder toutes et après le codage, commencer à les tester
  - B: Les coder une à une en testant au fur et à mesure
  - C: Les coder toutes avec un corps NULL. Puis les coder une à une en testant au fur et à mesure

Module M3.1, section 3 :: Les types abstraits : Comparaison et compléments sur les ADT

Module M3.1, section 3 : : Les types abstraits : Comparaison et compléments sur les ADT

Plan

- Comparaison sur les ADT
- Complément sur les ADT

Module M3.1, section 3 : : Les types abstraits : Comparaison et compléments sur les ADT

➢Comparaison d'ADT

- **Soit deux instances C1 et C2 d'un type Type T1. La comparaison de C1 et C2 peut être faite par défaut ou en réécrivant les méthodes MAP ou ORDER**

- **Comparaison <u>par défaut</u> (sans MAP ou ORDER)**
  - La comparaison par défaut n'est possible que pour **l'égalité ou la différence** entre objet. **Exemple :** C1 = C2, C1 != C2

  - <u>Par défaut</u>, pour que <u>deux objets soient égaux</u>, la valeur de chacun de leurs attributs doit être égale.

- **Comparaison avec MAP ou ORDER**
  - Permet de **généraliser** la comparaison (=, !=, >, >=, <, <=) en codant une de ces méthodes. Les **comparaisons suivantes** deviennent possibles : C1=C2, C1!=C2, C1>C2, C1>=C2, …)

Module M3.1, section 3 : : Les types abstraits : Comparaison et compléments sur les ADT

➢Comparaison d'ADT

- **MAP MEMBER "fonction" :**
  - Permet de définir une **fonction d'ordre** qui renvoie un de ses **champs C** ou **un calcul (R) impliquant un ou plusieurs de ses champs.**
  - **L'ordre des instances du type sera l'ordre définit par le type de C ou de R renvoyé**
  - Les **champs concernés** peuvent être de types scalaires (date, varchar2, number, …) ou complexes munis d'une fonction d'ordre
  - Ne possède **pas de paramètres <u>explicite</u>** en déhors du paramètre **<u>implicite</u> SELF**
  - **On peut lui attribuer le nom qu'on souhaite**
  - Est adaptée pour **le TRI d'un grand nombre d'objets** et les **JOINTURES PAR HACHAGE**

Les fonctions d'ordre des ADT sont généralement de type MAP pour des raisons de performance.

Module M3.1, section 3 : : Les types abstraits : Comparaison et compléments sur les ADT

➢Comparaison d'ADT

➢**ORDER MEMBER "fonction" . N'est à définir que si MAP n'est pas utilisable**

- Permet de comparer **deux Objets selon un ordre sémantique** qui ne permet pas de s'appuyer sur les champs du type tel qu'avec MAP (comparaison complexe).
  - **Exemple :** Nous souhaitons classer les employés d'une entreprise selon leur **POUVOIR**.
    - ✓ L'ordre doit être : 1. PDG, 2. DG, 3. Directeur, 4. Ingénieur, 5. Sécrétaire, 6. Planton, …
    - ✓ Le champ Job qui contient ces valeurs est de type Varchar2.
    - ✓ Malheureusement renvoyer job dans une function MAP n'est pas **possible**. En effet, ici PDG viens avant tout. Alors qu'avec MAP ce serait DG
- Cette méthode doit avoir **UN et UN SEUL paramètre explicite <u>instance du Type</u> en plus du paramètre implicite SELF** (l'objet courant).
- Dans le code la méthode il faut comparer l'objet courant avec l'instance Explicite passé en paramètre
- La **méthode ORDER renvoie** toujours des valeurs de type **NUMBER**. Ces valeurs sont **1, 0 ou -1**. Elle renvoie 1 si l'objet courant est plus grand. -1 si l'objet courant est plus petit. 0 en cas d'égalité.

Comparaison d'ADT

- **Notes :**
  - Il est **impossible de définir** à la fois une méthode ORDER MEMBER et une méthode MAP MEMBER dans un même type
  - Pas plus d'une méthode ORDER ou MAP par type
  - MAP et ORDER **sont appelées automatiquement** lors de la comparaison d'objets instances d'un type

Module M3.1, section 3 : : Les types abstraits : Comparaison et compléments sur les ADT

➢ Comparaison d'ADT

▪ Exemple SANS les méthodes MAP et ORDER

```
CREATE OR REPLACE TYPE adresse2_t as object(
        Numero          NUMBER(4),
        Rue             VARCHAR2(20),
        Code_Postal     NUMBER(5),
        Ville           VARCHAR2(20),
        quartier        VARCHAR2(20));
CREATE TABLE adresse2_table(adr adresse2_t );

INSERT INTO adresse2_table VALUES (adresse2_t(1, 'rue du congres', 6000, 'Nice', 'Centre'));
INSERT INTO adresse2_table VALUES (adresse2_t(3, 'rue du congres', 6000, 'Nice', 'Centre'));
INSERT INTO adresse2_table VALUES(adresse2_t(2, 'rue des garages', 6100, 'Nice', 'Nord'));
INSERT INTO adresse2_table VALUES(adresse2_t(2, 'rue du port', 6300, 'Nice', 'Ouest'));

SQL> SELECT * FROM adresse2_table  ORDER BY adr;SELECT * FROM adresse2_table  order by adr
ERROR at line 1:
ORA-22950: cannot ORDER objects without MAP or ORDER method
```

Comparaison d'ADT

- **Exemple avec la méthode MAP MEMBER**

```
CREATE OR REPLACE TYPE adresse2_t as object(
        Numero          NUMBER(4),
        Rue             VARCHAR2(20),
        Code_Postal     NUMBER(5),
        Ville           VARCHAR2(20),
        quartier        VARCHAR2(20),
        MAP MEMBER FUNCTION compAdresse
                RETURN VARCHAR2,
        PRAGMA RESTRICT_REFERENCES
(compAdresse, WNDS, WNPS, RNPS, RNDS)
);
/
```

```
CREATE OR REPLACE TYPE BODY adresse2_t IS
    MAP MEMBER FUNCTION compAdresse
        RETURN VARCHAR2 IS
    BEGIN
      RETURN ville||rue||numero;
    END;
END;
/
```

NOTE : La valeur de Comparaison des instance de de adresse2_t sera : **ville||rue||numero. Ce qui réduit la comparaison à celle d 'un type scalaire Oracle**

Comparaison d'ADT

Exemple avec la méthode MAP MEMBER

```
CREATE TABLE adresse2_table(adr adresse2_t );

INSERT INTO adresse2_table VALUES (adresse2_t(1, 'rue du congres', 6000, 'Nice', 'Centre'));
INSERT INTO adresse2_table VALUES (adresse2_t(3, 'rue du congres', 6000, 'Nice', 'Centre'));
INSERT INTO adresse2_table VALUES(adresse2_t(2, 'rue des garages', 6100, 'Nice', 'Nord'));
INSERT  INTO adresse2_table VALUES(adresse2_t(2, 'rue du port', 6300, 'Nice', 'Ouest'));
```

```
SQL> SELECT * FROM adresse2_table;

ADR(NUMERO, RUE, CODE_POSTAL, VILLE,QUARTIER)
ADRESSE2_T(1, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(3, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(2, 'rue des garages', 6100, 'Nice', 'Nord')
ADRESSE2_T(2, 'rue du port', 6300, 'Nice', 'Ouest')

SQL> SELECT * FROM adresse2_table  order by adr;

ADR(NUMERO, RUE, CODE_POSTAL, VILLE,QUARTIER)
ADRESSE2_T(2, 'rue des garages', 6100, 'Nice', 'Nord')
ADRESSE2_T(1, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(3, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(2, 'rue du port', 6300, 'Nice', 'Ouest')
```

Comparaison d'ADT

Exemple avec la méthode ORDER MEMBER

- Nous souhaitons **classer les adresses selon leur position géographique**. Nord, Centre, Sud, Est et enfin Ouest. MAP MEMBER ne suffit plus vraiment.

```
CREATE OR REPLACE TYPE adresse2_t as object(
        Numero              NUMBER(4),
        Rue                 VARCHAR2(20),
        Code_Postal         NUMBER(5),
        Ville               VARCHAR2(20),
        quartier            VARCHAR2(20),
        ORDER MEMBER FUNCTION compAdresse (adr IN  adresse2_t) RETURN NUMBER,
        PRAGMA RESTRICT_REFERENCES (compAdresse, WNDS, WNPS, RNPS, RNDS)
);
```

Comparaison d'ADT

- **Exemple avec la méthode ORDER MEMBER**

```
CREATE OR REPLACE TYPE BODY adresse2_t AS
ORDER MEMBER FUNCTION compAdresse (adr IN  adresse2_t )
                RETURN NUMBER IS
        position1 NUMBER :=0;
        position2 NUMBER :=0;
        concAdr1 VARCHAR2(60) := SELF.ville||SELF.rue||SELF.numero;
        concAdr2 VARCHAR2(60) := adr.ville|| adr.rue|| adr.numero;
BEGIN
        CASE SELF.quartier
                WHEN 'Nord' THEN position1 := 1;
                WHEN 'Centre' THEN position1 := 2;
                WHEN 'Sud' THEN position1 := 3;
                WHEN 'Est' THEN position1 := 4;
                WHEN 'Ouest' THEN position1 := 5;
        END CASE;
        CASE adr.quartier
                WHEN 'Nord' THEN position2 := 1;
                WHEN 'Centre' THEN position2 := 2;
                WHEN 'Sud' THEN position2 := 3;
                WHEN 'Est' THEN position2 := 4;
                WHEN  'Ouest' THEN position2 := 5;
        END CASE;
        concAdr1 :=  position1||concAdr1;
        concAdr2 := position2|| concAdr2;
        IF concAdr1 = ConcAdr2        THEN return 0;
                ELSIF concAdr1 > ConcAdr2    THEN return 1;
                ELSIF concAdr1 < ConcAdr2    THEN return -1;
        END IF;
END;
/
```

Comparaison d'ADT

Exemple avec la méthode ORDER MEMBER

```
CREATE TABLE adresse2_table(adr adresse2_t );

INSERT INTO adresse2_table VALUES (adresse2_t(1, 'rue du congres', 6000, 'Nice', 'Centre'));
INSERT INTO adresse2_table VALUES (adresse2_t(3, 'rue du congres', 6000, 'Nice', 'Centre'));
INSERT INTO adresse2_table VALUES(adresse2_t(2, 'rue des garages', 6100, 'Nice', 'Nord'));
INSERT  INTO adresse2_table VALUES(adresse2_t(2, 'rue du port', 6300, 'Nice', 'Ouest'));
```

```
SQL> SELECT * FROM adresse2_table;

ADR(NUMERO, RUE, CODE_POSTAL, VILLE,QUARTIER)
ADRESSE2_T(1, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(3, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(2, 'rue des garages', 6100, 'Nice', 'Nord')
ADRESSE2_T(2, 'rue du port', 6300, 'Nice', 'Ouest')

SQL> SELECT * FROM adresse2_table  order by adr;

ADR(NUMERO, RUE, CODE_POSTAL, VILLE,QUARTIER)
ADRESSE2_T(2, 'rue des garages', 6100, 'Nice', 'Nord')
ADRESSE2_T(1, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(3, 'rue du congres', 6000, 'Nice', 'Centre')
ADRESSE2_T(2, 'rue du port', 6300, 'Nice', 'Ouest')
```

Module M3.1, section 3 : : Les types abstraits : Comparaison et compléments sur les ADT

➢Complément sur les Types Abstraits Oracle

➢ Les **privilèges** sur les types sont **:**

- CREATE TYPE (permet de créer, supprimer, modifier, hériter d'un type, exécuter les méthodes d'un type)
- CREATE ANY TYPE.
- ALTER ANY TYPE.
- DROP ANY TYPE.
- UNDER ANY TYPE (droit d'hériter dans tout schéma)
- UNDER ANY VIEW
- EXECUTE ANY TYPE
- EXECUTE (privilège objet)
- UNDER (privilège objet)

- La fonction **IS NULL** permet de tester si la référence vers un ADT est null ou pas

- La fontion **IS DANGLING** permet de vérifier si une référence pointe vers un objet détruit.

- L'**indexation** d 'attributs d 'ADT est possibles s'ils sont accessibles et ne sont ni des VARRAYs, ni des LOBs, ni des NESTED TABLES.

Module M3.1, section 3 : QUIZ

➢ **Question 1**: Soit le type suivant créé :

```
create or replace type etudiant_t as object(
    idEtu       number(4),
    nomEtu      varchar2(20));
/
```

Cochez ce qui est juste

- A: Declare paul etudiant_t:=etudiant_t(1, 'Paul'); pierre etudiant_t:=etudiant_t(2, 'Pierre'); BEGIN if paul > pierre then dbms_output.put_line('Paul est plus grand que Pierre'); end if; END;
- B: Declare paul etudiant_t:=etudiant_t(1, 'Paul'); pierre etudiant_t:=etudiant_t(2, 'Pierre'); BEGIN if paul = pierre then dbms_output.put_line('Paul est égal à Pierre'); end if; END;
- C: Aucune des réponses précédentes n'est vrai

➢ **Question 2**: Soit le type suivant créé avec le prototype de la méthode d'ordre:

```
create or replace type etudiant_t as object(
    idEtu       number(4),
    nomEtu      varchar2(20));
    map member function match return varchar2
/
```

Cochez ce qui est juste

- A: Declare paul etudiant_t:=etudiant_t(1, 'Paul'); pierre etudiant_t:=etudiant_t(2, 'Pierre'); BEGIN if paul > pierre then dbms_output.put_line('Paul est plus grand que Pierre'); end if; END;
- B: Declare paul etudiant_t:=etudiant_t(1, 'Paul'); pierre etudiant_t:=etudiant_t(2, 'Pierre'); BEGIN if paul = pierre then dbms_output.put_line('Paul est égal à Pierre'); end if; END;
- C: Aucune des réponses précédentes n'est vrai

Module M3.1, section 3 : QUIZ

➢ **Question 3**: Soit le type suivant créé :

```
create or replace type etudiant_t as object(
      idEtu          number(4),
      nomEtu         varchar2(20));
      map member function match return varchar2
/
create or replace type BODY etudiant_t  as
map member function match return varchar2 is
Begin
            return nomEtu || idEtu;
End;
end;
/
```

Cochez ce qui est juste

- A: Declare paul etudiant_t:=etudiant_t(1, 'Paul'); pierre etudiant_t:=etudiant_t(2, 'Pierre'); BEGIN if paul > pierre then dbms_output.put_line('Paul est plus grand que Pierre'); end if; END;
- B: Declare paul etudiant_t:=etudiant_t(1, 'Paul'); pierre etudiant_t:=etudiant_t(2, 'Pierre'); BEGIN if paul = pierre then dbms_output.put_line('Paul est égal à Pierre'); end if; END;
- C: Aucune des réponses précédentes n'est vrai

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, construction des ADT

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

Plan

- Modification des ADT
- Suppression des ADT
- Visualisation des informations sur les ADT
- Instanciation des ADT

Modification d'un ADT

- **La modification d'un type permet** : d'ajouter/modifier/ supprimer des méthodes et des champs

- **Syntaxe**

ALTER TYPE nomType
| alter_methode_definition
| alter_attribute_definition

OPTION	DESCRIPTION
alter_methode_definition	Option d'ajout et modification de méthodes
alter_attribute_definition	Option d'ajout et modification d'attributs

BY NC ND

Modification d'un ADT

L'option ajout / suppression de méthodes

- **Syntaxe**

  alter_methode_definition::=
  {ADD | DROP} {map_order_function_spec
  | subprogram_clauses}

- **Exemple 1 : ajout**

  Alter type adresse3_t
  add **Member function setNumero(self IN OUT adresse3_t, numero1 IN number) return boolean;**

- **Exemple 2 : suppression**

  Alter type adresse3_t
  drop member **function setNumero (self IN OUT adresse3_t, numero1 IN number) return boolean;**

BY NC ND

Modification d'un ADT

L'option ajout / modification / suppression d'attributs

- **Syntaxe**
  alter_attribute_definition::=
  {ADD | MODIFY} ATTRIBUTE attribute datatype
  [(attribute datatype, ...)]
  DROP ATTRIBUTE attribute [(attribute, ...)]

- **Exemple 1 : ajout**
  Alter type adresse3_t **add** attribute state varchar2(20);
  Alter type adresse3_t **add** attribute(state2 varchar2(20));

- **Exemple 2 : Modification**
  Alter type adresse3_t **modify** attribute state2 varchar2(30);

- **Exemple 3 : Suppression**
  Alter type adresse3_t **drop** attribute state2;

Modification d'un ADT

L'option ajout / modification / suppression d'attributs

- **<u>Les options INVALIDATE et CASCADE</u>. S'il y a des dépendances, la modification d'un type n'est possibles qu'en utilisant les options INVALIDATE et CASCADE.**
  - ✓ L'option **CASCADE** permet de propager immédiatement la modification sur les objets dépendants.
  - ✓ L'option **INVALIDATE** permet d'invalider provisoirement tous les objets dépendants. Au prochain accès les objets sont à nouveau validé. ATTENTION !!! Il faut être sûr des conséquences

Exemple

```
Create table adresse3 of adresse3_t;
Insert into adresse3 values(1, 'rue de la gare', 6560, 'Valbonne', 'AM');
Alter type adresse3_t modify attribute state varchar2(30)
    *
ERREUR à la ligne 1 :
ORA-22312: vous devez indiquer l'option CASCADE ou INVALIDATE
```

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

➢Modification d'un ADT

- **L'option ajout / modification / suppression d'attributs**

➢**<u>Les options INVALIDATE et CASCADE</u>. S'il y a des dépendances, la modification d'un type n'est possibles qu'en utilisant les options INVALIDATE et CASCADE.**

✓ **Exemple 1 : Modification avec INVALIDATE**

```
Alter type adresse3_t modify attribute state varchar2(30)
 invalidate;
Insert into adresse3 values(2, 'rue de la gare', 6560, 'Nice', 'AM');
```

✓ **Exemple 2 : Modification avec CASCADE**

```
Alter type adresse3_t add attribute(state2 varchar2(20)) cascade;
Insert into adresse3 values(2, 'rue de la gare', 6560, 'Nice', 'AM','AM');
```

L'option CASCADE peut être suivi de INCLUDING DATA ou NOT INCLUDING DATA : avec ou sans conversion de données des tables. INCLUDING DATA est l'option par défaut

Modification d'un ADT

L'option ajout / modification / suppression d'attributs

- **Limites**

✓ **Il n'est pas possible de supprimer :**

Un attribut d'un super type dans un sous-type

Un attribut dépendant sans l'option cascade ou **« invalidate »**

Un attribut utilisé comme clé primaire ou clé de partitionnement

✓ **Il n'est pas possible de modifier :**

La taille d'un attribut participant à un Index basé sur une fonction, un domaine index ou une clé cluster

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

➢Suppression d'un ADT

- **Syntaxe de suppression du corps d'un type**

DROP TYPE BODY nomType;

- La suppression du corps **d'un type permet d'implémenter de meilleurs alogorithmes sans remettre en cause la spécification du type**.

- **Syntaxe pour suppression d'un type (Spécification et Corps)**

DROP TYPE nomType [FORCE | VALIDATE]

- L'option **FORCE** permet de supprimer un type en ignorant les dépendances. Les colonnes ou les attributs définis sur ce type deviennent invalide
- L'option **VALIDATE** permet de vérifier que des instances des super types n'ont pas étés construits avec le sous-type. **Héritage uniquement.**

Suppression d'un ADT

- **Exemple**

```
DROP TYPE adresse_t;
DROP TYPE adresse_t FORCE;
DROP TYPE Employee_t VALIDATE;
```

- **Syntaxe pour la suppression du corps d'un type**

```
DROP TYPE BODY nomType ;
```

✓ **Exemple**

```
DROP TYPE BODY adresse_t;
```

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

ESTIA INSTITUTE OF TECHNOLOGY

UNIVERSITÉ CÔTE D'AZUR

➢Visualisation des informations sur les ADT

NOM DE LA VUE	DESCRIPTION
CDB_TYPES	Infos sur les types
DBA_TYPES	Infos sur les types
ALL_TYPES	Infos sur les types
USER_TYPES	Infos sur vos types
CDB_COLL_TYPES	Infos sur les types collections
DBA_COLL_TYPES	Infos sur les types collections
ALL_COLL_TYPES	Infos sur les types collections
USER_COLL_TYPES	Infos sur les types collections

NOM DE LA VUE	DESCRIPTION
CDB_TYPE_ATTRS	Infos sur les attributs d'un type
DBA_TYPE_ATTRS	Infos sur les attributs d'un type
ALL_TYPE_ATTRS	Infos sur les attributs d'un type
USER_TYPE_ATTRS	Infos sur les attributs d'un type
CDB_TYPE_METHODS	Infos sur les méthodes d'un type
DBA_TYPE_METHODS	Infos sur les méthodes d'un type
ALL_TYPE_METHODS	Infos sur les méthodes d'un type
USER_TYPE_METHODS	Infos sur les méthodes d'un type

Notes :

CDB_* : Infos sur les objets du container courant et ses fils

DBA_* : Infos sur les objets du container courant

ALL_* : Infos sur les objets du container courant sur lesquels vous avez des droits d'accès

USER_* : Infos sur vos objets dans le container courant

anr

INVESTIR L'AVENIR

CC BY NC ND

DATUM ACADEMY

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

➢Visualisation des informations sur les ADT

NOM DE LA VUE	DESCRIPTION
CDB_INDEXTYPES	Infos sur les types index définis dans le container courant et ses fils
DBA_INDEXTYPES	Infos sur les types index définis dans le container courant
ALL_INDEXTYPES	Infos sur les types index définis dans le container courant pour lesquels vous avez des droits
USER_INDEXTYPES	Infos sur vos types index définis dans le container courant
CDB_INDEXTYPE_COMMENTS	Commentaires sur les types index définis dans le container courant et ses fils
DBA_INDEXTYPE_COMMENTS	Commentaires sur les types index définis dans le container courant
ALL_INDEXTYPE_COMMENTS	Commentaires sur les types index définis dans le container courant pour lesquels vous avez des droits
USER_INDEXTYPE_COMMENTS	Commentaires sur vos types index définis dans le container courant
CDB_INDEXTYPE_OPERATORS	Infos sur les opérateurs des types index définis dans le container courant et ses fils
DBA_INDEXTYPE_OPERATORS	Infos sur les opérateurs des types index définis dans le container courant
ALL_INDEXTYPE_OPERATORS	Infos sur les opérateurs des types index définis dans le container courant pour lesquels vous avez des droits
USER_INDEXTYPE_OPERATORS	Infos sur opérateurs de vos types index définis dans le container courant

Visualisation des informations sur les ADT

```
SQL> desc user_types
    TYPE_NAME
    TYPE_OID
    TYPECODE
    ATTRIBUTES
    METHODS
    PREDEFINED
    INCOMPLETE
    FINAL
    INSTANTIABLE
    SUPERTYPE_OWNER
    SUPERTYPE_NAME
    LOCAL_ATTRIBUTES
    LOCAL_METHODS
    TYPEID
```

Visualisation des informations sur les ADT

- **Liste des types créés par l'utilisateur courant**

```
sql>select type_name, type_oid, attributes, methods, local_attributes
from user_types;
```

TYPE_NAME	TYPE_OID	ATTRIBUTES	METHODS
ADRESSE2_T	A73CC5B2852F418E89136F467DA27C77	4	1
ADRESSE3_T	17E3DBF26849423EB0F6F44CFF2C5EE7	6	2
ADRESSE_T	E1F25E99482849E49A9B6AF3A10C2A03	4	0
PILOTE_T	0B930966561E4FF8B8140663F78A9960	6	0
REF_PILOTE_T	AB2360EC8BEC4F1EBFE81D0E5F270559	6	3

…

Visualisation des informations sur les ADT

- **Liste des attributs des types créés par l'utilisateur courant**

```
SQL> SELECT type_name, attr_name, attr_type_name
FROM user_type_attrs;
```

TYPE_NAME	ATTR_NAME	ATTR_TYPE_NAME
ADRESSE2_T	NUMERO	NUMBER
ADRESSE2_T	RUE	VARCHAR2
ADRESSE2_T	CODE_POSTAL	NUMBER
ADRESSE2_T	VILLE	VARCHAR2
ADRESSE3_T	NUMERO	NUMBER

…

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

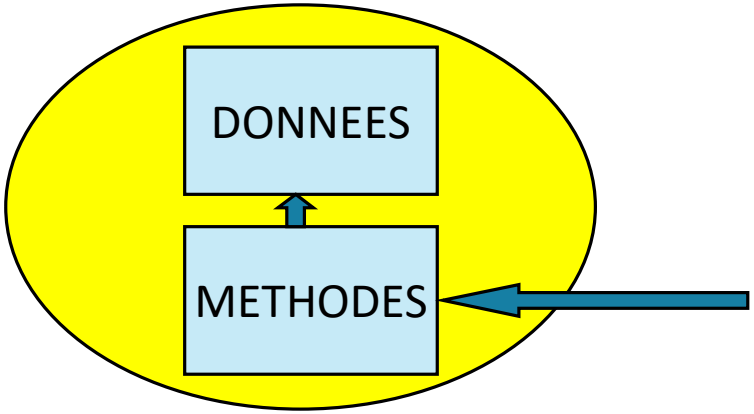
Encapsulation

- L'encapsulation est le principe qui **consiste à considérer un objet comme une BOITE NOIRE ACCESSIBLE UNIQUEMENT A TRAVERS LES OPERATIONS DEFINIES** sur celle -ci

- L'Encapsulation en C++ par exemple est gérée à travers trois mots clés : PRIVATE, PROTECTED et PUBLIC

- **Le moteur Oracle ne supporte pas l'encapsulation telle que définie.** C'est au programmeur de se limiter à l'appel de méthodes

Encapsulation



Constructeur et instanciation

- Un Constructeur est une méthode **spéciale ayant le même nom que le type**. Le constructeur du type ADRESSE_T s'appelle ADRESSE_T.
- un constructeur sert à **la création et à la mise à jour** des objets (dans les ordres SQL : INSERT, UPDATE, dans des fonctions PL/SQL).
- Un Constructeur en Oracle **est appelable partout ou une méthode peut être appelée**
- Un constructeur **par défaut est disponible**. Ses paramètres sont **l'ensemble des champs** du type
- L'utilisateur **peut DEFINIR** ses propres constructeurs

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

➢Constructeur et instanciation

- **Exemple 1 : constructeur par défaut**
  - **Soit la table objet Pilote de pilote_t:**

```
CREATE TABLE o_pilote OF ref_pilote_t;

-- Insertion d 'un élement de la table objet.
-- Pour la colonne, le Constructeur adresse_t
-- sera appelé
INSERT INTO o_pilote VALUES
( ref_pilote_t(1, 'Jean DUPOND',
    adresse_t(1,  'rue  Arago',6000,  'NICE'),
'0493825084', '12-JUN-47', 12000.50)  );
```

NOTE:l'appel de ref_PILOTE_T(…) est facultatif

- **Exemple 2 : Mise à jour d'un champ via un constructeur**

```
UPDATE o_pilote op
 SET op.adr = ref_adresse_t(5, 'rue de la
  traverse',6000, 'Nice')
WHERE op.pl#=1;
```

Constructeur et instanciation

- **Exemple : constructeur autre que celui par défaut**

```
CREATE OR REPLACE  TYPE  adresse4_t AS
OBJECT(
Numero          NUMBER(4),
Rue             VARCHAR2(20),
Code_Postal     NUMBER(5),
Ville           VARCHAR2(20),
CONSTRUCTOR FUNCTION adresse4_t(
SELF IN OUT adresse4_t, numero Number, rue VARCHAR2 ) RETURN SELF AS RESULT ,
ORDER MEMBER FUNCTION compAdresse (adr IN  adresse4_t) RETURN NUMBER,
PRAGMA RESTRICT_REFERENCES (compAdresse, WNDS, WNPS, RNPS, RNDS)
);
/
```

Module M3.1, section 4 :: Les types abstraits : Modification, suppression, visualisation, instanciation des ADT

➢ **Constructeur et instanciation**

- **Exemple : constructeur autre que celui par défaut**

```
create or replace type body adresse4_t is
CONSTRUCTOR FUNCTION adresse4_t(
  SELF IN OUT NOCOPY adresse4_t,
  numero Number ,
  rue VARCHAR2
) RETURN SELF AS RESULT IS
Begin
Self.numero:=numero;
Self.rue:=rue;
RETURN ;
END;
```

Exemple : constructeur autre que celui par défaut (suite)

```
ORDER MEMBER FUNCTION
 compAdresse (adr IN  adresse4_t) RETURN NUMBER is
Begin
  if self.numero<adr.numero then return -1;
  end if;
  if self.numero>adr.numero then return 1 ;
  end if;
  return 0;
End;
/
```

NOTE : NOCOPY permet le passage par référence et non par valeur)

SELF IN OUT : Paramètre facultatif

Constructeur et instanciation

- **Exemple : Appel du constructeur autre que celui par défaut**

```
declare
adr1 adresse4_t:=adresse4_t(1, 'Alufa', 5000, 'Ifa');
adr2 adresse4_t:=adresse4_t(1, 'Josoe');
begin
adr2.ville:=  'Jerusalem ';
Adr2.code_postal :=10000;
end;
/
```

Constructeur et instanciation

- **Il n'est pas possible d'instancier un type abstrait *sinon que dans une table ou dans un programme PL/SQL***

```
CREATE TABLE r_pilote(pil   PILOTE_T);
Declare
                pil_var        PILOTE_T;
Begin
End;
/
```

BY NC ND

Constructeur et instanciation

- **Instanciation dans un programme PLSQL**

```
Declare
  pil_var PILOTE_T:= pilote_t(1, 'ZEMBLA', adresse_t(1, 'rue des Miracle',6000, 'NICE'), ' 0622732123', ' 01/01/1982 ', 30000.00);
Begin … End;
```

- **Instanciation lors de l'insertion d'une ligne dans une table ayant une colonne complexe**

```
CREATE TABLE r_pilote(pil   PILOTE_T);
Insert into r_pilote  values (pilote_t(1, ' ZEMBLA', adresse_t(1, 'rue des Miracle',6000, 'NICE'), '
0622732123', ' 01/01/1982 ', 30000.00));
```

Module M3.1, section 4 : QUIZ

➢ Soit le type suivant créé :

```
create or replace type etudiant_t as object(
    idEtu       number(4),
    nomEtu      varchar2(20));
/
```

Question 1 : Nous souhaitons ajouter une méthode statique getEtudiant qui renvoie une instance d'étudiant. Cochez ce qui est juste:

ALTER TYPE etudiant_t ADD static function getEtudiant (etuNumero IN number) RETURN etudiant_t;

- A: Cette commande s'exécutera sans erreur dès qu'elle est lancée
- B: Pour invoquer cette méthode il faut la précéder du nom du type
- C: Pour invoquer cette méthode il faut une instance de type etudiant_t
- D: Pour invoquer cette méthode avec succès il faut que sont corps est été codé

Question 2 : Nous souhaitons ajouter un champ dateNaissanceEtu, cochez ce qui est juste :

- A: **ALTER TYPE etudiant_t ADD attribute (**dateNaissanceEtu date**);**
- B: **ALTER TYPE etudiant_t ADD attribute (**dateNaissanceEtu date**)** cascade**;**
- C: **ALTER TYPE etudiant_t ADD attribute (**dateNaissanceEtu date**)** invalidate **;**
- D: Aucune des solutions n'est juste

Module M3.1, section 4 : QUIZ

- Supposons que la table relationnelle suivante est créée :

```
Create table r_etudiant(etu etudiant_t,
Constraint pk_r_etudiant_idetu primary key(etu.idEtu));
```

Question 3 : : Considérons la table r_etudiant. Cochez ce qui fonctionne pour construire un objet de type etudiant_t

- A: insert into r_etudiant values (1, ‘King’);
- B: insert into r_etudiant values (etudiant_t(2, ‘Leroi’));
- C: insert into r_etudiant values (etudiant_t(3, null));
- D: insert into r_etudiant values (etudiant_t(null, ‘Lechef’));

Question 4 : Nous souhaitons ajouter un champ dateNaissanceEtu cochez ce qui est juste :

- A: **ALTER TYPE etudiant_t ADD attribute (**dateNaissanceEtu date**);**
- B: **ALTER TYPE etudiant_t ADD attribute (**dateNaissanceEtu date**)** cascade**;**
- C: **ALTER TYPE etudiant_t ADD attribute (**dateNaissanceEtu date**)** invalidate **;**
- D: Aucune des solutions n’est juste

Module M3.1 : Bilan et exercices

- Bilan

- Exercices

Module M3.2 : Notion d'identité et les collections dans SQL3 Oracle

Module M3.2 : Notion d'identité et les collections dans SQL3 Oracle

ESTIA INSTITUTE OF TECHNOLOGY

UNIVERSITÉ CÔTE D'AZUR

Plan

- **Module M3.2, section 1 :** NOTION D'IDENTITE
- **Module M3.2, section 2 ::** NOTION DE COLLECTION : Les Varrays
- **Module M3.2, section 3, partie 1 ::** NOTION DE COLLECTION : Les Nested tables
- **Module M3.2, section 3, partie 2 ::** NOTION DE COLLECTION : Les Nested tables

anr

INVESTIR L'AVENIR

CC BY NC ND

DATUM ACADEMY

Module M3.2, section 1 : NOTION D'IDENTITE

Module M3.2, section 1 : NOTION D'IDENTITE

➢ **Plan**

- **Identité sur les tables**
- **Identité sur les vues**
- **Le package UTL_REF**
- **Différences entre un OID et ROWID**

Identité sur les tables

- Lors de l'insertion d'un objet dans la base, **un OID ou Object Identifier lui est affecté**. C'est identifiant **ne peut changer** pendant la durée de vie de l'objet

- Seuls **les objets des tables ou des vues objets sont identifiés** par des OID. Une table Objet est créée avec la commande

« CREATE TABLE nomTable OF nomType … »

- Le type **référence « REF » permet** de stocker des références vers des objets (OID).

- Une référence permet de définir les liens entre objets

- **Attention! :** les OID sont très différents des ROWID!

Identité sur les tables

- Une référence **REF** comprend trois champs :
  - L'**OID** vers la table (16 bytes)
  - L'**OID** vers une ligne dans la table (16 bytes)
  - Et le **ROWID** vers une ligne dans la table (10 bytes)

Identité sur les tables

Exemple : Manipulation des références

```
-- Création d'une table objet
CREATE TABLE O_REF_PILOTE OF REF_PILOTE_T;

-- Création d'une table relation avec une référence
CREATE TABLE ref_Vol (
        Vol#        NUMBER(4) PRIMARY KEY,
        VD          VARCHAR2(20),
        VA          VARCHAR2(20),
        Pilote      REF ref_pilote_t); --type abstrait défini préalablement
```

Identité sur les tables

Exemple : Manipulation des références (suite)

-- Insertion en exploitant les astuces du langage SQL

```
INSERT INTO o_ref_pilote VALUES ( ref_pilote_t(2,  'Milou TINTIN',
 adresse_t(12, 'rue nord', 75000, 'Paris'), '0493825084', '11-AUG-63', 10000.5));

INSERT INTO ref_vol SELECT 300, 'Paris', 'Nice', REF(px)
      from o_ref_pilote px  where px.nom = 'Milou TINTIN';
```

Identité sur les tables

Exemple : Manipulation des références et la clause RETURNING REF(…) INTO ...

La clause **RETURNING... REF(…) INTO …** permet, lors de l'insertion d'une ligne dans une table Objet de capturer à la volée la référence vers cet Objet. L'intérêt est la mise à jour des liens. Elle permet aussi de capturer à la volée les valeurs de d'autres colonnes. Elle n'est utilisable que dans un bloc PL/SQL.

Identité sur les tables

- **Exemple : Manipulation des références et la clause RETURNING REF(…) INTO …**

```
// Insertion d 'un pilote et d 'un vol dans un programme PL/SQL
declare
        pil_ref REF ref_pilote_t :=null;
Begin

INSERT INTO o_ref_pilote op VALUES ( ref_pilote_t(1, 'Jean Dupond',
adresse_t(3, 'rue Arago', 6000, 'NICE'), '0493825084','11-AUG-62', 10000.5)) RETURNING ref(op) INTO pil_ref;

INSERT INTO ref_vol values(400, 'Paris', 'Nice', pil_ref);
end;
/
```

Identité sur les vues

- A l'image des tables, **deux types de vues** sont introduites avec **la version objet relationnelle d'Oracle** :
  - Les vues r**elationnelles** et les vues **dites objets**

- Les **vues objets** possèdent des **identifiants** physiques appelés **Primary key OID** (**pseudo OID**) avec leur pendant à la manipulation **Primary key REF** (**pseudo REF**)

- les **vues objets** servent entre autre à **créer des OID sur les tables relationnelles** SQL2 **!!!**

Identité sur les vues

- **Syntaxe de création d'une vue object**

```
CREATE OR REPLACE [FORCE] VIEW nomVue
  OF nomTypeObjet
  WITH OBJECT IDENTIFIER
  {DEFAULT
  | (attribut1[, attribut2, …])
  }
  AS sousRequête;
```

- **Description des paramètres**

OF nomTypeObjet :

Création d'une vue objet sur le type NomTypeObjet

WITH OBJECT IDENTIFIER :

Permet de spécifier la ou les colonnes utilisées comme clé pour identifier de façon unique les lignes de la vue objet.

Identité sur les vues

- **Description des paramètres**

Attribut1, attribut2, ...

Attributs permettant d'identifier de façon unique une vue. Ces attributs mappent la clé primaire de la table relationnelle sous-jacente

DEFAULT

indique que l'identification d'objets de la vue s'appuie sur les OID d'une table objet ou d'une vue objet sous-jacente

Identité sur les vues

- **Exemple : attribution des OID à une table relationnelle**
  - **Etape 1 : Création de la table relationnelle POLOTE1**

```
CREATE table Pilote1 (
  PL#       NUMBER(4) CONSTRAINT pk_pilote1 PRIMARY KEY,
  nom       VARCHAR2(12),
  adresse   adresse_t,-- type abstrait défini préalablement
  Tel       VARCHAR2(12),
  dnaiss    DATE,
  salaire   NUMBER(7,2)
);
```

Note : on suppose que le type adresse_t existe.

Identité sur les vues

- **Exemple : attribution des OID à une table relationnelle**
  - **Etape 2 : Insertion des lignes dans la table relationnelle POLOTE1**

Sql> INSERT INTO **pilote1** VALUES (5, 'TINTIN',
adresse_t(15, 'Av de Mars', 1000, 'BRUXELLES'), '0493825084', '11/08/64', 15000.5);

Sql> INSERT INTO **pilote1** VALUES (6, 'RACKHAM',
adresse_t(16, 'Av des épaves', 97137, 'POINTE-A-PITRE'), '+59064938284', '12/08/68', 20000.5);

Identité sur les vues

- **Exemple : attribution des OID à une table relationnelle**
  - **Etape 3 : création du type Pilote1_t ayant la structure de la table Pilote1**

```
CREATE OR REPLACE TYPE  Pilote1_t as object(
  PL#    NUMBER(4),
  nom  VARCHAR2(12),
  adresse      adresse_t,-- type abstrait défini préalablement
  Tel     VARCHAR2(12),
  dnaiss  DATE,
  salaire NUMBER(7,2));
```

Identité sur les vues

- **Exemple : attribution des OID à une table relationnelle**
  - **Etape 4 : Création de la vue objet V_PILOTE1 sur la table relationnelle SQL2** Pilote1

```
CREATE OR REPLACE VIEW V_PILOTE1 OF Pilote1_t
  WITH OBJECT IDENTIFIER(pl#) --PL# mappe la clé primaire de la table PILOTE1
  AS SELECT * FROM Pilote1  p;
```

Identité sur les vues

- **Exemple : attribution des OID à une table relationnelle**
  - **Etape 5 : Consultation à travers la vue objet V_PILOTE1 et vérification de la présence des OID**

SQL> select **REF(V)** FROM **v_Pilote1 v** ;

```
REF(V)
--------------------------------------------------------------------------------
00004A038A0046C46C248CD42944D9A3C4E3CDB108AC4800000014260100010001002900000000000
090604002A00078401FE0000000A02C1060000000000000000000000000000000000000000

00004A038A0046C46C248CD42944D9A3C4E3CDB108AC4800000014260100010001002900000000000
090604002A00078401FE0000000A02C1070000000000000000000000000000000000000000
```

Le package UTL_REF

- Ce package possède un certain nombre de méthodes permettant **de manipuler un objet connaissant sa référence**

- **Liste des méthodes du package UTL_REF**
  - PROCEDURE DELETE_OBJECT (REF IN REF OF STANDARD);
  - PROCEDURE LOCK_OBJECT (REF IN REF OF STANDARD);
  - PROCEDURE LOCK_OBJECT (REF IN REF OF STANDARD, OBJECT OUT STANDARD);
  - PROCEDURE SELECT_OBJECT (REF IN REF OF STANDARD, OBJECT OUT STANDARD);
  - PROCEDURE SELECT_OBJECT_WITH_CR (REF IN REF OF STANDARD, OBJECT OUT STANDARD);
  - PROCEDURE UPDATE_OBJECT (REF IN REF OF STANDARD, OBJECT IN STANDARD);

Module M3.2, section 1 : NOTION D'IDENTITE

➢ Le package UTL_REF

- **Exemple d'utilisation du package UTL_REF**

```
declare
   refPilote1 REF  pilote_t;
   unPilote        pilote_t;
begin
      SELECT ref(orp) INTO refPilote1 FROM o_pilote  orp WHERE orp.pl#=1;

      dbms_output.put_line('Nom pilote=' || refPilote1.nom);
      -- impossible :PLS-00536: La navigation entre variables
      -- REF n'est pas prise en charge dansPL/SQL.

      -- Utiliser utl_ref.SELECT_OBJECT
      utl_ref.SELECT_OBJECT(refPilote1, unPilote);

      dbms_output.put_line(Nom Pilote=' || unPilote.nom);
end;
/
```

Nota : Le **package utl_ref** permet de **résoudre le problème de navigation entre REF** en PL/SQL

Module M3.2, section 1 : NOTION D'IDENTITE

- **Différence entre un OID et un ROWID**

Critère	ROWID	OID
Taille	10	16
identifiant immuable	NON	OUI
Sert à indexer	OUI	NON
partage d'objets	NON	OUI

SQL> SELECT **REF(op)**, **ROWID** FROM o_pilote op;

REF(OP)	ROWID
000028020985B3AD5BB8C04D39BF81C8DB5573EFE53AFC0C486CA3447283858183F28672A6020001B20000	**AAAIFzAAIAAAAGyAAA**
0000280209A36782FB22EE44409D6560943CFD7A433AFC0C486CA3447283858183F28672A6020001B20001	**AAAIFzAAIAAAAGyAAB**

Module M3.2, section 1 : QUIZ

➢ Soit le type suivant créé :

```
create or replace type etudiant_t as object(
    idEtu        number(4),
    nomEtu       varchar2(20))
/
```

➢ Supposons les tables suivantes créées :

```
Create table r_etudiant(etu etudiant_t,
Constraint pk_r_etudiant_idetu primary key(etu.idEtu));

Create table o_etudiant of etudiant_t(
Constraint pk_o_etudiant_idetu primary key(idEtu));
```

Question 1 : : Considérons la table r_etudiant. Cochez ce qui fonctionne

- A: insert into **o_etudiant** values (1, 'King');
- B: insert into r_etudiant values (etudiant_t(2, 'Leroi'));
- C: insert into **o_etudiant** values (etudiant_t(3, null));
- D: insert into r_etudiant values (etudiant_t(null, 'Lechef'));

Question 2 : Considérons qu'il y a un étudiant Nr 1 dans la table o_etudiant et un étudiant nr 2 dans la table table r_etudiant. Cochez ce qui renvoie une référence:

- A: select ref(oe) from r_etudiant oe where oe.etu.idEtu=2;
- B: select ref(oe) from o_etudiant oe where oe.idEtu=1;
- C: Aucune des deux requête ne renvoie de référence.

Module M3.2, section 1 : QUIZ

Question 3 : : Cochez ce qui caractérise une référence

- A: **Peux** changer pendant la durée de vie d'un objet (ligne dans une table objet)
- B: **Ne peux** changer pendant la durée de vie d'un objet (ligne dans une table objet)
- C: Est une structure avec trois champs (OID vers la table, l'OID vers la ligne et le ROWID vers la,ligne)
- D: Toutes ces affirmations précédentes sont justes

Question 4 : Le type REF

- A: Permet de modéliser le lien d'association de cardinalité 1
- B: Permet d'éviter les clés étrangères
- C: Permet d'identifier un objet de façon unique pendant sa durée de vie
- D: Permet de modéliser le lien d'association de cardinalité N
- E: Toutes ces affirmations ne sont pas justes

Module M3.2, section 2 :: NOTION DE COLLECTION : Les Varrays

Module M3.2, section 2 :: NOTION DE COLLECTION : Les Varrays

- **Plan**
  - **Généralités sur les collections Varrays et Nested table**
  - **Définition d’un VARRAY**
  - **Syntaxe de création d'un Varray**
  - **Exemple de création de Varray**
  - **Organisation du stockage d’un VARRAY**

Module M3.2, section 2 :: NOTION DE COLLECTION : Les Varrays

➢Généralités sur les collections Varrays et Nested table

- **Deux types de collections sont supportés dans la version SQL3 Oracle :**
  - **Les collections de type VARRAY** : Les tableaux de longueur variable (VARRAY) ANSI SQL. Le nombre maximum **d'éléments EST CONNU**
  - **Les collection de type NESTED TABLE** : Ensemble Multiple ou Bag (Nested Table) ANSI standard. Le nombre maximum **d'éléments N'EST PAS CONNU**
- **Une collection peut contenir :**
  - Des types de données élémentaires.
  - Des types de données abstraits.
  - Des références vers des types de données abstraits
  - Des collections (VARRAY ou NESTED TABLE)

Module M3.2, section 2 :: NOTION DE COLLECTION : Les Varrays

➢ Définition d'un VARRAY

- **Un « varying ARRAY »** est un ensemble **d'éléments ordonnés** de même type. Chaque élément a une valeur et occupe une position unique.
- Le **NOMBRE maximum d'éléments** d'un VARRAY **est connu à l'avance**

➢ Syntaxe de création d'un VARRAY

CREATE [OR REPLACE] TYPE nomType
AS {VARRAY | VARYING ARRAY}
(nombreMaxD'éléments**) OF** typeDeDonnées

➢Exemple de Création d'un type VARRAY

- **Création du type AVION_T des éléments à mettre dans la collection**

```
CREATE TYPE avion_t AS OBJECT(
        Av#         NUMBER(4),
        AvType      VARCHAR2(45),
        Loc         VARCHAR2(20),
        Cap         NUMBER(4)
);
/
```

➢Exemple de Création d'un type VARRAY

- **Création du type VARRAY TAB_AVIONS_T dont les éléments sont des AVION_T**

```
CREATE OR REPLACE TYPE TAB_AVIONS_T  AS VARRAY (5) OF avion_t
/
```

- **Création de la table HANGAR avec une colonne avions de type TAB_AVIONS_T**

```
CREATE TABLE HANGAR (
hangar#    NUMBER(2),
avions     TAB_AVIONS_T
);
```

➢ Exemple de Création d'un type VARRAY

- **Insertion dans une table ayant un VARRAY**

```
INSERT INTO HANGAR VALUES (1, TAB_AVIONS_T (avion_t ( 1, 'AIRBUS', 'NICE',300) )) ;
INSERT INTO HANGAR VALUES (2, TAB_AVIONS_T (avion_t ( 2, 'AIRBUS',  'PARIS',320))) ;
INSERT INTO HANGAR VALUES (3, TAB_AVIONS_T (avion_t ( 3, 'AIRBUS',  'PARIS',320),  avion_t ( 5,
  'AIRBUS',  'PARIS',320))) ;
```

- **Consultation d'une table ayant un VARRAY**

```
SQL> SELECT * FROM hangar;
```

➢Exemple de Création d'un type VARRAY

- **Consultation d'une table ayant un VARRAY**

SQL> **SELECT * FROM HANGAR**;

HANGAR#	AVIONS
1	TAB_AVIONS_T(AVION_T(1, 'AIRBUS', 'NICE', 300))
2	TAB_AVIONS_T(AVION_T(2, 'AIRBUS', 'NICE', 300))
3	TAB_AVIONS_T(AVION_T(3, 'AIRBUS', 'NICE', 300), AVION_T (5, 'AIRBUS', 'PARIS',320))

NOTE : Le nom du type VARRAY est aussi le nom du constructeur par défaut(ici:TAB_AVIONS_T)

➢ Exemple de Création d'un type VARRAY

- **Consultation d'une table avec un VARRAY en utilisant l'opérateur TABLE**

SQL>SELECT h.hangar#, TAV.* FROM HANGAR h, TABLE(h.avions) TAV;

HANGAR#	AV#	AVTYPE	LOC	CAP
1	1	AIRBUS	NICE	300
2	2	AIRBUS	PARIS	320
3	3	AIRBUS	PARIS	320
3	5	AIRBUS	PARIS	320

-- NOTES : L'opérateur TABLE permet de convertir le VARRAY avions en une table

➢ Exemple de Création d'un type VARRAY

- **Consultation des éléments d'un VARRAY pour un hangar donnée**

```
SQL> SELECT TAV.*
        FROM TABLE (SELECT h.avions FROM HANGAR h WHERE h.hangar#=3)  TAV;
```

HANGAR#	AV#	AVTYPE	LOC	CAP
3	3	AIRBUS	PARIS	320
3	5	AIRBUS	PARIS	320

➢ Exemple de Création d'un type VARRAY

- **Mise à jour d'une colonne de type VARRAY**

SQL> UPDATE hangar SET avions =TAB_AVIONS_T(avion_t (4, 'CARAVELLE','PARIS',320)) WHERE hangar#=2 ;

- **NOTE: Il n'est pas possible de mettre à jour un champs d'un VARRAY via l'ordre SQL UPDATE**

SQL> UPDATE TABLE (SELECT **h.avions** FROM HANGAR h WHERE h.hangar#=3) TAV
SET tav.avtype='CARAVELLE' WHERE tav.av#=5;

```
set tav.avtype='CARAVELLE'
   *
ERREUR à la ligne 3 :
ORA-25015: impossible d'effectuer un DML sur cette colonne de vue de table
Imbriquée
```

-- Avec les Ordres SQL (UPDATE, INSERT, DELETE), un VARRAY est manipulable comme un <u>TOUT</u>
-- La mise à jour d'un élément particulier est possible par programmation PL/SQL.

Module M3.2, section 2 :: NOTION DE COLLECTION : Les Varrays

➢ Organisation du stockage d'un VARRAY

- Le **stockage d'un VARRAY se fait à l'intérieur de chaque ligne** de la table ou il a été déclaré.
- **Toutefois si un VARRAY est trop grand** (plus 4000 octets), il peut est stocké en dehors du segment de la table. Mais dans le même TABLESPACE que la table
- **Syntaxe de stockage** d'un VARRAY dans un segment séparé

```
CREATE TABLE nomTableAvecVarray….(…)
VARRAY nomColonneVarray
STORE AS LOB {lob_segmentName
              | [lob_segmentName]
(lob_parameters)
```

Paramètre	Description
STORE AS LOB	close permettant de spécifier le segment de stockage du VARRAY
nomColonneVarray	Nom de la colonne de type varray
Lob_segmentName	Nom du segment de stockage des valeurs de la colonne VARRAY
Lob_parameters	Paramètres de stockages du VARRAY (voir la section sur les LOBs)

Organisation du stockage d'un VARRAY

- **Exemple**

```
CREATE TABLE hangar2 (
hangar#    NUMBER(2),
avions     TAB_AVIONS_T
)
VARRAY avions STORE AS LOB
storeLobArrayAvions(STORAGE(
INITIAL 100K
MINEXTENTS 2
));
```

Module M3.2, section 2 :: QUIZ

➢ **Question 1** : Cochez ce qui caractérise le mieux un Varray

- A : Un Varray est une collection pour laquelle les éléments sont ordonnés
- B : Un Varray est une collection pour laquelle les éléments ne sont pas ordonnés
- C : Un Varray est une collection pour laquelle le nombre maximum d'éléments est illimité
- D : Un Varray est une collection pour laquelle le nombre maximum d'éléments n'est pas illimité
- E : Une collection de type Varray ne peut contenir des éléments de type Varray

➢ **Question 2** : considérons la table hangar créée dans : **Module M3.2, section 2 .** Cochez ce qui marche

- A : insert into hangar h values (4, TAB_AVIONS_T(avion_t(10, 'AIRBUS', 'NICE',500))); -- Ajoute une collection avec 1 avion
- B : insert into hangar h values (5, TAB_AVIONS_T(avion_t(11, 'AIRBUS', 'NICE',500), avion_t(12, 'AIRBUS', 'NICE',500), avion_t(13, 'AIRBUS', 'NICE',500), avion_t(14, 'AIRBUS', 'NICE',500), avion_t(15, 'AIRBUS', 'NICE',500), avion_t(16, 'AIRBUS', 'NICE',500), avion_t(17, 'AIRBUS', 'NICE',500))); -- Ajoute une collection avec 6 avions
- C : insert into hangar h values (6, TAB_AVIONS_T()); -- Ajoute une collection vide
- D: insert into hangar h values (7, null); -- Initialise la collection à NULL

Module M3.2, section 2 :: QUIZ

➢**Question 3** : considérons l'insertion suivante dans la table hangar :

insert into hangar h values (8, TAB_AVIONS_T(avion_t(21, 'AIRBUS', 'NICE',500)));

L'insertion s' est effectuée. Nous nous sommes trompés de type d'avion. Ce doit être B707 à la place de AIRBIUS

Cochez ce qui marche

- A : UPDATE TABLE(select h.avions from hangar h where h.hangar#=8) lav SET lav.avtype='B707' WHERE lav.av#=21;
- B : UPDATE hangar h SET h.avions= TAB_AVIONS_T(avion_t(21, 'B707', 'NICE',500)) WHERE h.hangar#=8;
- C : Un programme PLSQL ferait aussi l'affaire
- D: Jamais on ne peut modifier une collection VARRAY avec un programme PLSQL

➢**Question 4** : cochez les requêtes qui fonctionnent et qui renvoient des instances de TAB_AVIONS_T (collection d'avions)

- A : SELECT h.hangar#, h.avions FROM hangar h ;
- B : SELECT h.hangar#, lav.* FROM hangar h , TABLE(h.avions) lav ;
- C: select * from TABLE(select h.avions from hangar h where h.hangar#=8) lav ;
- D: select h.avions from hangar h where h.hangar#=8;

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

Plan

- Définition d'une Nested table (bag ou set) et caractéristiques
- Syntaxe de création d'un type « TYPE TABLE »
- Exemple de collection Nested sans partage des éléments

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢ Définition d'une Nested table (bag ou set) et caractéristiques

- **Définition d'une nested table (bag ou set)**
  - Les nested tables sont des tables imbriquées efficaces pour la gestion **des relations maître-détail** (**N:M, 1:N**)
- **Caractéristiques**
  - Oracle étend le relationnel **au niveau domaine** (EF Codd). **Un TYPE TABLE est nécessaire** pour définir une NESTED TABLE pour définir **des listes comme colonnes** de table
  - Ces **NESTED TABLE** sont accessibles INDIRECTEMENT via **des ordres LMD** : SELECT, UPDATE, INSERT, DELETE en convertissant la collection en une table grâce à l'opérateur **« TABLE »**
  - Ces **NESTED TABLE** sont manipulables DIRECTEMENT via **des ordre LDD** (Création d'index, pose de contraintes, …).

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢ Définition d'une Nested table (bag ou set) et caractéristiques

▪ Caractéristiques

- Les **enregistrements d'une Nested Table sont stockés dans un SEGMENT SEPARE.** Ce segment est une table relationnelle SQL2**.**
- Un **constructeur par défaut** est fourni pour construire les objets d'une NESTED TABLE, il doit être du même type que le TYPE TABLE de la colonne ou la variable concernée
- **Le TYPE TABLE** peut être utilisé comme type de données dans les déclarations suivantes : **colonne** d'une table, **attribut** d'un ADT, variable, **paramètre** ou **résultat** dans un programme PL/SQL.
- Le nombre **maximum d'élément** d'un TYPE TABLE **n'est pas connu à l'avance**

Syntaxe de création d'un type « TYPE TABLE »

- Syntaxe de création d'un type « TYPE TABLE », collection illimité

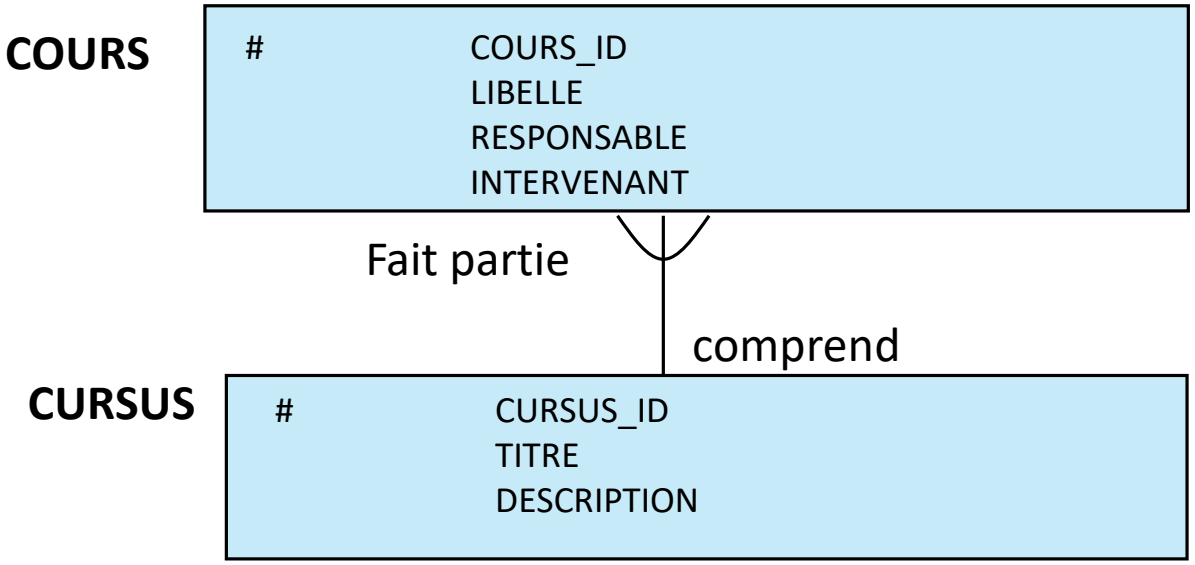
```
CREATE OR REPLACE TYPE nomType
  AS TABLE OF  typeDesElements
```

- **TypeDesElements** : type des éléments de la collection

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢ Exemple de collection Nested SANS PARTAGE des éléments

- Considérons le schéma conceptuel Entité/Relation (E/R) suivant



- **#** : indique un champ identifiant
- **Lien :** Un COURS « fait partie » d'un CURSUS. Un CURSUS comprend 1 ou N COURS

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢ Exemple de collection Nested SANS PARTAGE des éléments

```
DROP TABLE O_CURSUS;
DROP TYPE COURS_t;
DROP TYPE CURSUS_T;
DROP TYPE LISTE_DE_COURS_T;



-- Création du type COURS_T
-- Ce type permettra de construire les éléments de la collection
CREATE OR REPLACE  TYPE COURS_T AS OBJECT (
                COURS_ID            NUMBER(4),
                LIBELLE
        VARCHAR2(100),
                RESPONSABLE    VARCHAR2(50),
                INTERVENANT    VARCHAR2(50))
/
```

```
-- Création du type Nested table LISTE_DE_COURS_T
-- contenant les cours d'un cursus
CREATE OR REPLACE TYPE LISTE_DE_COURS_T AS TABLE OF COURS_T;

-- Création du type CURSUS_T
CREATE OR REPLACE  TYPE CURSUS_T AS OBJECT (
        CURSUS_ID          NUMBER(4),
        TITRE                  VARCHAR2(50),
        DESCRIPTION            VARCHAR2(100),
        LISTECOURS             LISTE_DE_COURS_T )
/
```

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢Exemple de collection Nested SANS PARTAGE des éléments

--Création de la table O_CURSUS avec sa table imbriqué LISTECOURS

--Définition du segment *storeListeCours* (STORE TABLE) pour stocker les objets de la table imbriquée.

--L'accès à la table *storeListeCours* n'est possible qu'à partir de la table O_CURSUS

```
CREATE TABLE O_CURSUS OF CURSUS_T(
        constraint pk_cursus primary key(cursus_id)
)
NESTED TABLE LISTECOURS STORE AS storeListeCours;
```

-- Insertion des lignes

```
INSERT INTO O_CURSUS
VALUES(1,
'M2 MBDS',
'Multimédia Bases de Données et Intégration de Systèmes',
LISTE_DE_COURS_T( COURS_T(100, 'SGBDR', 'Codd', 'Date'),
COURS_T(102, 'XML', 'Jacobson', 'Smirnoff')));

INSERT INTO O_CURSUS
VALUES(2,
'M1 MIAGE',
'Méthodes Informatiques Appliquées à la Gestion',
LISTE_DE_COURS_T( COURS_T(101, 'Java', 'Mc Nelly', 'Buffa'));
```

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢ **Exemple de collection Nested SANS PARTAGE des éléments**

```
-- Insertion des lignes
INSERT INTO O_CURSUS
VALUES(3,
'M1 INFO',
'Master 1 Informatique',
LISTE_DE_COURS_T( COURS_T(103, 'UML', 'Booch', 'Jackson'))));

INSERT INTO O_CURSUS
VALUES(4,
'M2 ISI',
'M2 Ingéniérie des Systèmes Informatques',
LISTE_DE_COURS_T());
```

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢ **Exemple de collection Nested SANS PARTAGEdes éléments**

SEGMENT DE LA TABLE MAITRE : O_CURSUS

Cursus_id	1	2	3	4
Titre	M2 MBDS	M1 MIAGE	M1INFO	M2 ISI
Description	Master BD	MIAGE	Informatique	ISI
ListeCours	@1	@2	@3	@4

Segment de la tableDétail (imbriquée), Store table:storeListeCours

Nested_table_id	@1	@2	@1	@3
Cours_id	100	101	102	103
Libelle	SGBDR	Java	XML	UML
Responsable	Codd	Mc Nelly	Jacobson	Booch
Intervenant	Date	Buffa	Smirnoff	Jackson

NESTED_TABLE_ID: Colonne implicite permettant d'identifier les lignes Du segment détail attachées à une même ligne de la table maître.

Module M3.2, section 3, partie 1 :: NOTION DE COLLECTION : Les Nested tables

➢ **Exemple de collection Nested SANS PARTAGE des éléments**

```
-- consultation des cours du CURSUS MBDS
SELECT t1.*
 FROM TABLE(SELECT listeCours
FROM o_cursus
WHERE cursus_id=1) t1;
```

COURS_ID	LIBELLE	RESPONSABLE	INTERVENANT
100	SGBDR	Codd	Date
102	XML	Jacobson	Smirnoff

Module M3.2, section 3, partie 1 :: QUIZ

- **Question 1** : Cochez ce qui caractérise le mieux un type Nested table
  - A : Un type Nested table est une collection pour laquelle les éléments sont ordonnés
  - B : Un type Nested table est une collection pour laquelle les éléments ne sont pas ordonnés
  - C : Un type Nested table est une collection pour laquelle le nombre maximum d'éléments est illimité
  - D : Un type Nested table est une collection pour laquelle le nombre maximum d'éléments n'est pas illimité
  - E : Une collection de type Nested table ne peut contenir des éléments de type Nested table ou Varray

- **Question 2** : considérons la table O_CURSUS créée dans : **Module M3.2, section 3, partie 1 .** Cochez ce qui marche
  - **A :** insert into o_cursus oc values (4,'L3MIAGE', 'L3MIAGE', LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date'))); -- Ajoute une collection avec 1 cours
  - **B :** insert into o_cursus oc values (5, 'M2MIAGE MBDS', 'M2MIAGE MBDS',LISTE_DE_COURS_T(COURS_T(101, 'XML', 'Jacobson', 'Smirnoff'), COURS_T(102, 'Java', 'Mc Nelly', 'Buffa'), COURS_T(103, 'SGBDOR', 'Stonebraker', 'Stonebraker'), COURS_T(104, 'SGBDNOSQL', 'Gabriel', 'Gabriel'), COURS_T(105, 'Data lake', 'Ullman', 'Ullman'), COURS_T(106, 'Oracle NoSQL', 'Larry', 'Larry'), COURS_T(107, 'MongoDB', 'Horowitz', 'Horowitz'))); -- Ajoute une collection avec 6 courss
  - **C :** insert into o_cursus oc values (6, 'M1MIAGE', 'M1MIAGE',LISTE_DE_COURS_T()); -- Ajoute une collection vide
  - **D:** A ne fonctionnement pas du tout

Module M3.2, section 3, partie 1 :: QUIZ

➢**Question 3** : considérons l'insertion suivante dans la table hangar :

insert into o_cursus oc values (4,'L3MIAGE', 'L3MIAGE', LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date')));

L'insertion s' est effectuée. Nous nous sommes trompés de professeur intervenant. Ce doit être 'Delobel' à la place de 'Date'

Cochez ce qui marche

- A : UPDATE TABLE(select oc.LISTECOURS from o_cursus oc where oc.cursus_id=4) lc SET lc.INTERVENANT='Stonebraker' WHERE lc.cours_id=100;
- B : UPDATE o_cursus oc SET oc.LISTECOURS= LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date')) WHERE oc.cursus_id=4;
- C : Un programme PLSQL ferait aussi l'affaire
- D: Jamais on ne peut modifier une collection de type NESTED TABLE avec un programme PLSQL

➢**Question 4** : cochez les requêtes qui fonctionnent et qui renvoient des instances de LISTE_DE_COURS_T (collection de cours)

- A : SELECT oc.cursus_id, oc.LISTECOURS FROM o_cursus oc ;
- B : SELECT oc.cursus_id, lc.* FROM o_cursus oc , TABLE(oc.LISTECOURS) lc ;
- C: select * from TABLE(select oc.LISTECOURS from o_cursus oc where oc.cursus_id=4) lc ;
- D: select oc.LISTECOURS from o_cursus oc where oc.cursus_id=4;

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

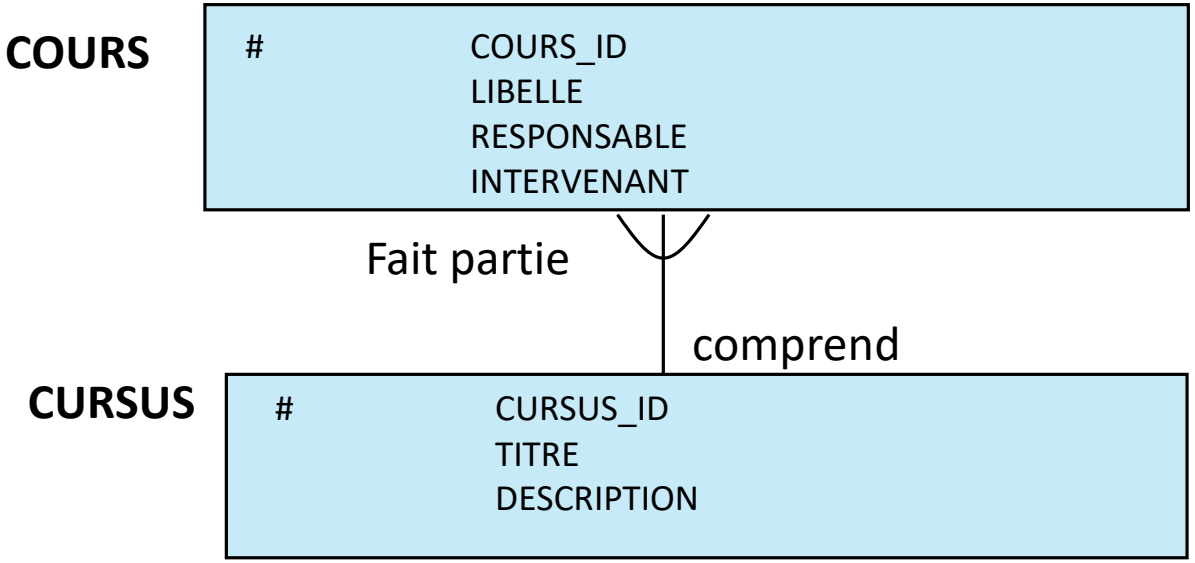
Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

Plan

- Exemple de collection Nested **AVEC PARTAGE** des éléments
- Organisation du stockage d'une Nested Table
- Indexation dans une Nested table
- Méthodes sur les collections Varray et Nested table
- Comparaison entre VARRAYS et NESTED TABLE

Exemple de collection Nested AVEC PARTAGE des éléments

- Considérons le schéma conceptuel Entité/Relation (E/R) suivant



: indique un champ identifiant

Lien entre CURSUS et COURS : Un cours « fait partie d'un cursus et un cursus comprend 1 ou N cours

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

➢Exemple de collection Nested AVEC PARTAGE des éléments

```
Drop table o_cursus cascade constraints;
Drop table o_cours cascade constraints;
Drop type cursus_t force;
Drop type cours_t force;
drop type LISTE_REF_DE_COURS_T force;

-- Création du type incomplet cursus_t
CREATE OR REPLACE TYPE cursus_t;
/
-- Création du type COURS_T
CREATE OR REPLACE  TYPE COURS_T AS OBJECT (
        COURS_ID        NUMBER(4),
        LIBELLE         VARCHAR2(100),
        RESPONSABLE     VARCHAR2(50),
        INTERVENANT     VARCHAR2(50),
        refCursus       REF cursus_t)
/
```

```
-- Création du type collection de REF vers les cours
CREATE OR REPLACE TYPE  LISTE_REF_DE_COURS_T
AS TABLE OF REF COURS_T
/
-- Création du type cursus_t
CREATE OR REPLACE  TYPE CURSUS_T AS OBJECT (
        CURSUS_ID       NUMBER(4),
        TITRE           VARCHAR2(50),
        DESCRIPTION     VARCHAR2(100),
        ListRefCours    LISTE_REF_DE_COURS_T )
/
```

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

➢ **Exemple de collection Nested AVEC PARTAGE des éléments**

```
-- Création de la table O_CURSUS
CREATE TABLE O_CURSUS OF CURSUS_T(
        constraint pk_cursus_cursus_id primary key(cursus_id)
)
NESTED TABLE listRefCours STORE AS storeListRefCours;

-- Création de la table O_COURS
CREATE TABLE O_COURS OF COURS_T(
constraint pk_cours_cours_id primary key(cours_id));
```

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

- **Exemple de collection Nested AVEC PARTAGE des éléments**

SEGMENT DE LA TABLE MAITRE : O_CURSUS

RefCursus	@RefCu1	@RefCu2	@RefCu3	@RefCu4
Cursus_id	1	2	3	4
Titre	M2 MBDS	M1 MIAGE	M1INFO	M2 ISI
Description	Master BD	MIAGE	Informatique	ISI
ListRefCours	@1	@2	@3	@4

Segment de la table Détail (imbriquée) : storeListRefCours

Nested_table_id	@1	@2	@1	@3
Column_*value*	@RefC100	@RefC101	@RefC102	@RefC103

SEGMENT DE LA TABLE : O_COURS

RefCours	@RefC100	@RefC101	@RefC102	@RefC103
Cours_id	100	101	102	103
Libelle	SGBDR	Java	XML	UML
Responsable	Codd	Mc Nelly	Jacobson	Booch
Intervenant	Date	Buffa	Smirnoff	Jackson
refCursus	**@RefCu1**	**@RefCu2**	**@RefCu1**	**@RefCu3**

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Exemple de collection Nested AVEC PARTAGE des éléments

```
-- Insertion des lignes dans la table O_CURSUS
-- Les listes sont au départ vides

INSERT INTO O_CURSUS
VALUES(1, 'M2 MBDS',
'Multimédia Bases de Données et Intégration de Systèmes',
LISTE_REF_DE_COURS_T());

INSERT INTO O_CURSUS
VALUES(2, 'M1 MIAGE',
'Méthodes Informatiques Appliquées à la Gestion',
LISTE_REF_DE_COURS_T());
```

```
-- Insertion des lignes dans la table O_CURSUS
-- Les listes sont au départ vides

INSERT INTO O_CURSUS
VALUES(3, 'M1 INFO',
'Master 1 Informatique', LISTE_REF_DE_COURS_T());

INSERT INTO O_CURSUS
VALUES(4, 'M2 ISI',
'M2 Ingéniérie des Systèmes Informatques',
LISTE_REF_DE_COURS_T());
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

➢ **Exemple de collection Nested AVEC PARTAGE des éléments**

```
-- Insertion des lignes dans la table O_COURS

-- Cours du curus 1
INSERT INTO O_COURS
Select 100, 'SGBDR', 'Codd', 'Date',
ref(oc) from o_cursus oc where oc.cursus_id=1 ;

INSERT INTO O_COURS
Select 102, 'XML', 'Jacobson', 'Smirnoff',
ref(oc) from o_cursus oc where oc.cursus_id=1 ;
```

```
-- Insertion des lignes dans la table O_COURS

-- Cours du cursus 2
INSERT INTO O_COURS
Select 101, 'Java', 'Mc Nelly', 'Buffa',
ref(oc) from o_cursus oc where oc.cursus_id=2 ;

-- Cours du cursus 3
INSERT INTO O_COURS
Select 103, 'UML', 'Booch', 'Jackson',
ref(oc) from o_cursus oc where oc.cursus_id=3;
```

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Exemple de collection Nested AVEC PARTAGE des éléments

```
-- Mise à jour des listes

-- Mise à jour de l liste des REF de cours du cursus 1
INSERT INTO
TABLE(SELECT oc.listRefCours From o_cursus oc
WHERE oc.cursus_id=1)
SELECT REF(oco) FROM O_COURS oco
WHERE cours_id in (100, 102) ;

-- Mise à jour de l liste des REF de cours du cursus 2
INSERT INTO
TABLE(SELECT oc.listRefCours From o_cursus oc
WHERE oc.cursus_id=2)
SELECT REF(oco) FROM O_COURS oco
WHERE cours_id=101 ;
```

```
-- Mise à jour de l liste des REF de cours du cursus 3
INSERT INTO
TABLE(SELECT oc.listRefCours From o_cursus oc WHERE
oc.cursus_id=3)
SELECT REF(oco)  FROM O_COURS oco
WHERE cours_id=103 ;
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

➢Exemple de collection Nested AVEC PARTAGE des éléments

-- La colonne COLUMN_VALUE
-- Rappel de la commande ayant permis de définir la collection
-- TYPE LISTE_DE_COURS_T.
--CREATE OR REPLACE TYPE LISTE_REF_DE_COURS_T
--AS TABLE OF **REF COURS_T**
--/
Lors de la définition d'une collection, le HANDLE permettant d'accéder implicitement à chaque élément de la Collection s' appelle COLUMN_VALUE.
Dans le cas de la collection LISTE_REF_DE_COURS_T. COLUMN_VALUE contient des pointeurs ou références vers des lignes de table O_COURS
COLUMN_VALUE EST UTILISE PAR ORACLE CHAQUE FOIS QUE LES ELEMENTS D'UNE COLLECTIONS NE SONT PAS UNE STRUCTURE.

-- Consultation des références de cours du CURSUS MBDS

```
SELECT t1.column_value
 FROM table(SELECT ListRefCours
FROM o_cursus
WHERE cursus_id=1) t1;
```

```
COLUMN_VALUE
--------------------------------------------------------------------------------
000022020884103A869F964291A8BC64E2C90C471A3A6E4942FB8947D5BB528DAD78A0C438
0000220208E240226981BB45709801EFC512DF43823A6E4942FB8947D5BB528DAD78A0C438
```

BY NC ND

Module M3.2, section 3, partie 2 :: NOTION DE COLLECTION : Les Nested tables

➢ Exemple de collection Nested AVEC PARTAGE des éléments

```
-- La colonne COLUMN_VALUE

-- Consultation des références de cours du CURSUS MBDS
Col LIBELLE format a20
Col TITRE format A20
SELECT t1.column_value.cours_id  cours_id,  t1.column_value.libelle libelle, t1.column_value.RefCursus.titre titre
 FROM TABLE(SELECT listRefCours
FROM o_cursus
WHERE cursus_id=1) t1;
```

COURS_ID	LIBELLE	TITRE
100	SGBDR	M2 MBDS
102	XML	M2 MBDS

Organisation du stockage d'une Nested Table

- Une Nested Table **est stockée dans un segment de données séparé** de celui de la table qui le contient
- Il est maintenant possible (depuis la version 10g) de localiser une **Nested Table** dans un **TABLESPACE autre que celui de la table qui la contient**. Il est aussi possible de lui assigner ses propres paramètres de stockage
- **Syntaxe de stockage d'un NESTED TABLE dans un segment séparé**

```
CREATE TABLE nomTableAvecUneNestedTable….(…)
NESTED TABLE nomColonneNestedTable
STORE AS {{nestedTableSegmentName| Column_value}
                    [ ({(object_properties)
                              | physical_properties
                              | column_properties
                        })
                    ]
  [RETURN AS {LOCATOR | VALUE}]
```

Organisation du stockage d'une Nested Table

physical_properties::={segment_attributes_clause
| ORGANIZATION organization_parameters
| CLUSTER cluster_parameters
}

Segment_attributes_clause::={[{**pctfree** value| **pctused** value
| **initrans** value | storage_clause}]…
| **TABLESPACE** nomTablespace
| logging_clause
}

storage_clause::=STORAGE(
INITIAL value | NEXT value | PCTINCREASE value
MINEXTENTS value | MAXEXTENTS value)

BY NC ND

Organisation du stockage d'une Nested Table

- Syntaxe de stockage d'un NESTED TABLE dans un segment séparé

Paramètre	Description
Object_properties	Permet de définir par exemple des valeurs par défaut, les contraintes d'intégrités etc sur les colonnes ou attributs d'un élément d'un collection
physical_properties	Permet de définir l'organisation physique du segment de la collection
Column_properties	Permet de définir les propriétés d'organisation physique des champs des éléments d'une collection. Surtout si l'élément contient des collections (nested ou varray), des LOB : large objet, des type XML.
STORE AS	close permettant de spécifier le nom du segment de stockage du VARRAY. Ce nom sera **column_value** s'il s'agit de localiser le segment d'un élément d'une collection qui est lui-même une collection
nomColonneNestedTable	Nom de la colonne de type NESTED TABLE
nestedTableSegmentName	Nom du segment de stockage permet de choisir un mode d'organisation de la table
ORGANIZATION CLUSTER	permet de mettre la table en cluster

Organisation du stockage d'une NESTED TABLE

- **Exemple**

```
CREATE TABLE O_CURSUS OF CURSUS_T(
  constraint pk_cursus_cursus_id primary key(cursus_id)
)
NESTED TABLE ListRefCours STORE AS storeListRefCours(
  PCTFREE 20
  TABLESPACE users
  STORAGE(INITIAL 100K MINEXTENTS 2)
);
```

Indexation dans une nested table

- **Les éléments associés à un objet** par exemple <la liste des cours d'un cursus >lorsqu'ils sont modélisés à travers des Nested Tables, sont stockés de façon **non ordonnée et non contigüe** dans la STORE TABLE de la Nested Table
- Il est donc utile de poser **des index sur les colonnes de la Nested Table** afin d'en accélérer l'accès
- Une colonne implicite **NESTED_TABLE_ID** permet d'identifier les éléments de la Nested Table ayant un même parent
- Un **identifiant UNIQUE** pour la Nested Table peut être obtenu en créant un INDEX CONCATENE qui inclut la colonne **Nested_table_Id et d'autres colonnes**

Indexation dans une nested table

- Indexation de la colonne **nested_table_id jusque 10g uniquement**
  - Création d'un index sur la COLONNE implicite NESTED_TABLE_ID de la NESTED TABLE « listeCours"de la table CURSUS.

    CREATE INDEX idx_listeCours_nested_table_id ON storeListeCours (NESTED_TABLE_ID);

  - Création d'un index unique sur une Nested Table

    CREATE UNIQUE INDEX IDX__libelle_nested_table_id ON storeListRefCours (**nested_table_id, column_value**);

- Indexation de la colonne **nested_table_id à partir de la 11g**
  - **L'index est créé automatiquement**.

Méthodes sur les collections

Nom	Description
EXISTS (n)	Rend vrai si n-ième élément existe
COUNT	Nombre d'éléments courants
LIMIT	Nombre maximum d'éléments (pour les VARRAY)
FIRST and LAST	Rend l'index du 1er ou dernier élément
PRIOR and NEXT	Rend l'index de l'élément précédent ou suivant
EXTEND(n[,i_1]… [,i_n])	Ajout de **n** éléments NULL à la fin si pas de i_i ou **n** éléments recopiés à partir des positions i_i
TRIM(n)	Supprime **n** élément à partir de la fin d'une collection
DELETE([m[, n]])	Sans paramètre, supprime tous les éléments. Avec un paramètre m. Supprime l'élément m. Avec 2 paramètre m et n, supprime de l'élément m jusqu'à l'élément **n**

Méthodes sur les collections

Exemples d'utilsation des méthodes avec PL/SQL

```
unCours    cours_t:=cours_t(100, 'J2EE STRUTS',
'Thomas', 'Sai');

IF listCours.EXISTS(i) THEN
  listCours(i) := unCours; …
End if;

IF (arrayAvions.COUNT + 20) < arrayAvions.LIMIT
THEN -- ajouter 20 éléments
End if;

FOR i IN  listCours.FIRST.. listCours.LAST
LOOP …
End loop;
```

```
unCours := listeCours.PRIOR(listeCours.NEXT(i));
                              -- affectation de l'élément i

 listCours.EXTEND(5,1);--ajout de 5 élém. recopiés à partir de 1

listCours.DELETE(2);     --suppression de l'élément 2
listCours.DELETE(3,6);  --suppression de l'élément 3 à l'élément 6
arrayAvions.DELETE;     --suppression de tous les éléments
```

Méthodes sur les collections

Exemples d'utilsation des méthodes avec PL/SQL

```
-- Accès  et mise à jour des élements d 'un VARRAY
DECLARE
        lav      TAB_AVIONS_T;
        i        number :=1;
BEGIN
SELECT avions INTO lav FROM hangar where hangar#=2;
dbms_output.put_line('COUNT='|| lav.count || 'LIMIT =‘|| lav.limit);
LOOP
          dbms_output.put_line (lav(i).av#  || '   '  || lav(i).avtype);
          lav(i).av#=20+i;
          IF  i = lav.count THEN      exit;
            i := i + 1;
          END IF;
END LOOP;
```

```
-- Accès  et mise à jour des élements d 'un VARRAY
        update hangar
        set avions=lav
        where hangar#=2;

        END;
END;
/
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Comparaison entre VARRAYS et NESTED TABLE

VARRAYs	***Nested Table***
Collection ordonnée	Se comporte comme un ensemble
Stocké dans le même segment que la la table ou dans un segment séparé	Stockée dans des segments physiques différents
N'autorise pas les indexes	Autorise les indexes sur ces éléments
Adapté pour un nombre limité d'éléments	Adaptée pour un nombre illimité d'éléments

Module M3.2, section 3, partie 2 :: QUIZ

- Considérons les type **LISTE_REF_DE_COURS_T** contenant des pointeurs vers les cours d'un cursus. Cours_t contenant un champ **refCursus** (pointeur vers le cursus d'un cours) et le **type cursus_t** contenant le champ **ListRefCours** définit sur le type **LISTE_REF_DE_COURS_T** créée dans : **Module M3.2, section 3, partie 2 .**

considérons les tables objets **O_CURSUS** et **O_COURS** créée dans : **Module M3.2, section 3, partie 2 .**

Considérons la table storeListRefCours

- **desc o_cursus**

Nom	NULL ?	Type
CURSUS_ID	NOT NULL	NUMBER(4)
TITRE		VARCHAR2(50)
DESCRIPTION		VARCHAR2(100)
LISTREFCOURS		LISTE_REF_DE_COURS_T

- desc **storeListRefCours**

Nom	NULL ?	Type
COLUMN_VALUE		REF OF COURS_T

- **Question 1** : cochez ce qui correspond au rôle de la table **storeListRefCours**
  - A: Cette table contient les instances de cours associées à chaque cursus
  - B: Cette table contient les pointeurs vers les cours associés à chaque cursus
  - C: Cette table est supprimée automatiquement si la table O_CURSUS est supprimée
  - D: Cette table peut être supprimée avec la commande : DROP TABLE **storeListRefCours**

Module M3.2, section 3, partie 2 :: QUIZ

- **Question 2** : Considérons la table **storeListRefCours** nous souhaitons ajouter le pointeur vers un nouveau cours (refCours1) dans la collections des pointeurs du cursus numéro 1. Cochez ce qui est juste
  - A: insert into **storeListRefCours values (refCours1) where nested_table_id=refCursus1;**
  - B: insert into TABLE(SELCT listRefCours FROM o_cursus oc WHERE oc.cursus_id=1) values (refCours1);
  - C: L'insert dans la table **storeListRefCours ne peut se faire qu'indirectement**
  - D: L'affirmation faite en C est juste

- **Question 3** : Considérons la table **storeListRefCours** cette table contient une colonne par défaut appelée COLUMN_VALUE et une colonne implicité appelée NESTED_TABLE_ID. Cochez ce qui est juste
  - A: La colonne COLUMN_VALUE permet de mémoriser les références vers les cours de chaque cursus
  - B: La colonne COLUMN_VALUE permet de mémoriser les instances des cours de chaque cursus
  - C: La colonne Nested_table_id permet de lier un cursus avec les références vers ses cours
  - D: Tous les références vers les cours d'un même cursus ont un même valeur pour la colonne nested_table_id
  - **E: storeListRefCours est une table imbriquée dans la table O_CURSUS**

Module M3.2, section 3, partie 2 :: QUIZ

- **Question 4** : Considérons la table **storeListRefCours** cette table contient une colonne par défaut appelée COLUMN_VALUE et une colonne implicité appelée NESTED_TABLE_ID. Nous souhaitons consulter les données contenues dans **storeListRefCours.** Cochez ce qui est permet d'afficher les réféences vers les cours
  - A: SELCT column_value FROM **storeListRefCours;**
  - **B: SELECT lrc.column_value FROM** TABLE(SELCT listRefCours FROM o_cursus oc WHERE oc.cursus_id=1) lrc;
  - **C: SELECT lrc.column_value.* FROM** TABLE(SELCT listRefCours FROM o_cursus oc WHERE oc.cursus_id=1) lrc;
  - **D: SELECT lrc.column_value.**libelle**, lrc.column_value.i**ntervenant **FROM** TABLE(SELCT listRefCours FROM o_cursus oc WHERE oc.cursus_id=1) lrc;

- **Question 5** : Considérons la table **storeListRefCours** cette table contient une colonne par défaut appelée COLUMN_VALUE et une colonne implicité appelée NESTED_TABLE_ID. Nous souhaitons consulter les données contenues dans **storeListRefCours.** Cochez ce qui est permet d'afficher les informations sur les cours
  - A: SELCT column_value FROM **storeListRefCours;**
  - **B: SELECT lrc.column_value FROM** TABLE(SELCT listRefCours FROM o_cursus oc WHERE oc.cursus_id=1) lrc;
  - **C: SELECT lrc.column_value.* FROM** TABLE(SELCT listRefCours FROM o_cursus oc WHERE oc.cursus_id=1) lrc;
  - **D: SELECT lrc.column_value.**libelle**, lrc.column_value.i**ntervenant **FROM** TABLE(SELCT listRefCours FROM o_cursus oc WHERE oc.cursus_id=1) lrc;

Module M3.2, section 3, partie 2 :: QUIZ

- **Question 6** :cochez ce qui est vrai pour les collections. Soit une collect COL1
  - A: COL1.LIMIT donne le nombre maximum d'éléments s'il s'agit d'un VARRAY
  - B: COL1.LIMIT donne le nombre maximum d'éléments s'il s'agit d'un type NESTED TABLE
  - C: COL1.FIRST et COL1.LAST correspondent aux indices du premier et du dernier élément d'une collection
  - D: COL1.COUNT donne le nombre d'éléments actuellement dans la collection s'il s'agit d'un VARRAY
  - E: COL1.COUNT donne le nombre d'éléments actuellement dans la collection s'il s'agit d'un type NESTED TABLE
  - Pour parcourir une collection en PLSQL on peut faire:
    - BEGIN For i in COL1.FIRST .. COL1.LAST LOOP … END LOOP; END;

Module M3.2 : Bilan et exercices

- Bilan
- Exercices

Module M3.3 : Les tables objets, les liens d’associations

Module M3.3 : Les tables objets, les liens d'associations

Plan

- **Module M3.3, section 1** : Les tables objets
- **Module M3.3, section 2, partie 1** : Les liens d'association
- **Module M3.3, section 2, partie 2** : Les liens d'association
- **Module M3.3, section 2, partie 3** : Les liens d'association

Module M3.3, section 1 : Les tables objets

Module M3.3, section 1 : Les tables objets

Plan

- Généralités
- Création et manipulation des tables objets
- Insertion dans une table objet
- Consultation d'une table objet
- Mise à jour dans une table objet
- Contraintes d'intégrité dans les tables objets
- Les nouveaux Opérateurs DML
- Les nouvelles Extensions DML
- Les colonnes virtuelle OBJECT_ID et OBJECT_VALUE
- Informations sur les tables objets et les Nested Tables

Généralités

- Oracle propose à partir d'Oracle V8 **deux types de tables** :
  - **Les tables Objets**
  - Les tables relationnelles traditionnelles

- Les **lignes des tables objets** sont identifiées **par les OID et les ROWID**, les **lignes des tables Relationnelles** sont identifiées **uniquement par les ROWID**

- **De nouveaux opérateurs et mots clés**(TABLE, REF, DEREF, , RETURNING, VALUE, IS OF TYPE, SYS_TYPEID, CURSORS, CAST, MULTISET, …) sont introduits pour faciliter la manipulation des objets complexes

Création et manipulation des tables objets

Définition

- Une table objet est une table dont les objets (lignes) ont chacun un identifiant (OID) unique et immuable.

Création d'une table Objet

- La création d 'une table objet se fait à l 'aide de la commande

CREATE TABLE **nomTableObjet** OF **TYPE nomTypeComplex**e[(…)];

- **Exemple** :

```
CREATE OR REPLACE TYPE Pilote4_t AS OBJECT (
        PL#         NUMBER(4),
        nom         VARCHAR2(12),
        adresse             ADRESSE_T,          -- adresse_t type abstrait défini préalablement
        Tel         VARCHAR2(12),
        dnaiss      DATE,
        salaire     NUMBER(7,2));

CREATE TABLE o_pilote4 OF pilote4_t ;
```

BY NC ND

Création et manipulation des tables objets

Création d'une table Objet

- **Contre exemple (table relationnelle SQL2 avec une colonne complexe)**

```
CREATE TABLE r_pilote4 (
    p            pilote4_t
) ;
```

-- la colonne **p** est un attribut **objet** de type pilote4_t. Mais **r_pilote 4** n 'est pas une table objet.

-- Les lignes de la table r_pilote4 ne sont pas identifiées par des OID. Mais uniquement par des ROWID.

-- Un ROWID peut changer au cours de la durée de vie d'une ligne.

Insertion d'une table objet

- L'insertion se fait avec la commande **INSERT INTO …. VALUES** avec ensuite application du constructeur

- **Exemple** avec application explicite du constructeur

```
INSERT INTO o_pilote4
VALUES ( pilote4_t(1, 'Jean DUPOND',
adresse_t(1, 'rue Arago',6000, 'NICE'), '0493825084', '12-06-1947', 12000.50)  );
```

- **Exemple** avec application implicite du constructeur

```
INSERT INTO o_pilote4
VALUES (2, 'Jean MAKOUN',
adresse_t(2, 'Boulevard Henri II',6000, 'NICE'), '0493825084', '12-06-1948', 12000.50);
```

- **Contre exemple**

```
INSERT INTO r_pilote4
VALUES ( pilote4_t(1, 'Alain DURAND',
adresse_t(1, 'Traverse des Brucs',6560, 'VALBONNE'), '0493825096', '12-06-1957', 16000.50)  );
```

Consultation d'une table objet

- La **consultation** se fait avec la commande SELECT * FROM …. WHERE …

- **Exemple**

```
-- Consultation de la table objet
SELECT nom, adresse FROM o_pilote4
WHERE nom = 'Jean DUPOND' ;
```

NOM	ADRESSE(NUMERO, RUE, CODE_POSTAL,VILLE)
Jean DUPOND	ADRESSE_T(1, 'rue Arago', 6000, 'NICE')

```
-- consultation de la table relationnelle avec type complexe
Select r.p.nom,r.p.adresse FROM r_pilote4 r
WHERE r.p.nom = 'Alain DURAND' ;
```

P.NOM	P.ADRESSE(NUMERO, RUE, CODE_POSTAL,VILLE)
Alain DURAND	ADRESSE_T(1, 'Traverse des Brucs', 6560, 'VALBONNE')

Consultation d'une table objet

- Une table relationnelle **ne contient pas d'OID** à la différence d'une table Objet

- **Exemple**

```
SELECT nom, rowid, ref(o) FROM o_pilote4 o
WHERE o.nom = 'Jean DUPOND' ;
```

NOM	ROWID	REF(O)
Jean DUPOND	AAAAw/AAFAAAAJWAAA	00002802096ABAF6D9B23511D2A64E00A0245FEFFD6ABAF6D8B23511D2A64E00A0245FEFFD014002560000

Consultation d'une table objet

- Une table relationnelle **ne contient pas d'OID** à la différence d'une table Objet
- **Contre exemple : l'opérateur REF ne s'applique pas ici**

```
SELECT r.p.nom,  rowid, ref(r )
FROM r_pilote4 r
WHERE r.p.nom = 'Alain DURAND' ;
  ERROR at line 1:
  ORA-00904: invalid column name r

SELECT r.p.nom,  rowid FROM r_pilote4 r        -- ref(r) doit être enlevé de la requête
WHERE r.p.nom = 'Alain DURAND' ;
```

P.NOM	ROWID
Alain DURAND	AAAAxBAAFAAAAJgAAA

BY NC ND

Mise à jour d'une table objet

- La **commande** utilisée **pour les mises à jour** est **UPDATE … SET … [WHERE …].** Les Colonnes peuvent être mises à jour *directement*, à travers des *constructeurs* ou à travers des *méthodes*

- **Exemple de maj sans utilisation de constructeurs**

```
UPDATE o_pilote4
SET Tel = '0493825095'
WHERE nom= 'Jean DUPOND' ;

UPDATE o_pilote4 o
SET o.adresse.numero = 22
WHERE o.nom= 'Jean DUPOND' ;
```

Mise à jour d'une table objet

- **Exemple de MAJ via un CONSTRUCTEUR**

```
UPDATE o_pilote4 o
SET o.adresse=adresse_t( 22,'Mariner', '06555', 'Mt Doc')
WHERE o.nom= 'Jean DUPOND' ;

SELECT o.nom, o.adresse.numero, o.adresse.rue
FROM o_pilote4 o
WHERE o.nom= 'Jean DUPOND';
```

NOM	ADRESSE.NUMERO	ADRESSE.RUE
Jean DUPOND	22	Mariner

Note : l'utilisation d'un constructeur oblige à renseigner tous les champs

Contraintes d'intégrité dans les tables objets

- **Les tables Objets supportent les constraintes d'intégrités :**
  - **D'entité** : clé primaire
  - **De référence** : clé étrangère
  - **De domaine** : NOT NULL, NULL, CHECK
- **Ces contraintes peuvent être exprimées pendant la création de la table ou pendant sa mise à jour**

CREATE TABLE nomTableObjet **OF nomType** (
{{colonne | attribut} [DEFAULT expr]
[**contrainteDeNiveauColonne**], ...
| **contrainteDeNiveauTable**;})

ALTER TABLE nomTableObjet **ADD** ContrainteDeNiveauTable;

BY NC ND

Contraintes d'intégrité dans les tables objets

- **Exemple de définition de contraintes lors de la création d'une table**

```
CREATE TABLE o_pilote5 OF pilote4_t(
  CONSTRAINT nl_nom    nom   NOT NULL,
  CONSTRAINT pk_o_pilote5 PRIMARY KEY(pl#),
  CONSTRAINT chk_dnaiss CHECK(dnaiss<to_date('12-OCT-1998', 'DD-MON-YYYY')),
  CONSTRAINT chk_salaire
        CHECK (salaire >2000 AND Salaire<70000),
  CONSTRAINT nl_adresse adresse NOT NULL,
  CONSTRAINT nl_adresse_rue
         CHECK(adresse.rue IS NOT NULL)
);
```

NOTE : Il n'est pas possible de poser des <u>contraintes au niveau TYPE</u>

Module M3.3, section 1 : Les tables objets

➢ Les Nouveaux Opérateurs DML

- **Opérateur sur les références** : REF, DEREF, REFTOHEX, IS DANGLING
- **Opérateurs sur les instances d'objets** : VALUE, RETURNING, **.** (point)
- **Opérateurs sur les collections** : TABLE, CAST, MULTISET, SUBMULTISET, CARDINALITY, CURSORS, MEMBER OF, IS [NOT] EMPTY, IS A SET, COLLECT, MULTISET {UNION | INTERSECT | EXCEPT}, POWERMULITSET, SET, POWERMULTISET_BY_CARDINALITY. Ils seront détaillés dans les sections concernant les liens d'associations
- **Opérateurs sur les vues** : MAKE_REF. Voir les sections concernant les vues
- **Opérateurs sur la hiérarchie de types (héritage)** : TREAT, IS OF TYPE , SYS_TYPEID. Ils seront détaillés dans les sections concernant l'héritage
- **NOTE** : DML (Data Manipulation Language) ou LMD (Langage de Manipulation de Données)

Module M3.3, section 1 : Les tables objets

➢ Les Nouveaux Opérateurs DML

- **L'opérateur VALUE**
  - Cet opérateur prend en paramètre l'alias d'une table ou une vue objet et renvoie l'INSTANCE d'un objet.
  - **Exemple d'utilisation de VALUE**

-- **consultation de la table OBJET o_pilote4**

SELECT **VALUE** (p) uneInstance FROM o_pilote4 p WHERE p.nom LIKE '%DUPOND%';

```
VALUE(P)(PL#, NOM, ADRESSE(NUMERO, RUE, CODE_POSTAL, VILLE), TEL, DNAISS, SALAIR
--------------------------------------------------------------------------------
PILOTE4_T(1, 'Jean DUPOND', ADRESSE_T(1, 'rue Arago', 6000, 'NICE'), '0493825084
', '12/06/47', 12000,5)
```

BY NC ND

Module M3.3, section 1 : Les tables objets

➢Les Nouveaux Opérateurs DML

- **L'opérateur VALUE**
  - Cet opérateur prend en paramètre l'alias d'une table ou une vue objet et renvoie l'INSTANCE d'un objet.
  - **Exemple d'utilisation de VALUE**

```
INSERT INTO r_pilote4    -- table  NON objet
SELECT VALUE (p)
FROM o_pilote4  p        -- table  objet
WHERE p.nom LIKE '%DUPOND%';
```

-- **Consultation de la table non objet r_pilote4**

SELECT rp.p FROM **r_pilote4** rp WHER rp.**p**.nom like '%DUPOND%';

```
P(PL#, NOM, ADRESSE(NUMERO, RUE, CODE_POSTAL, VILLE), TEL, DNAISS, SALAIRE)
--------------------------------------------------------------------------
PILOTE4_T(1, 'Jean DUPOND', ADRESSE_T(1, 'rue Arago', 6000, 'NICE'), '0493825084
', '12/06/47', 12000,5)
```

Les Nouveaux Opérateurs DML

L'opérateur REF

- **L'opérateur REF** permet de connaître référence vers une ligne donnée.
- Une référence est composée de : **l'OID vers la table objet** de l'objet (16 octets), **l'OID vers la ligne** (16 octets) et **du ROWID vers la ligne** (10 octets)
- **Exemple**

```
SELECT REF (px) FROM o_pilote4 px  -- table objet
WHERE px.nom = 'Jean DUPOND';
```

PX
0000280209E4715187332C11D1A51300A0245FEFFDE4715186332C11D1A51300A0245FEFFD0140010C0000

Les Nouveaux Opérateurs DML

L ’opérateur DEREF

- Retourne la valeur ou l’INSTANCE de l’objet identifié par la référence.
- **Exemple** :

```
-- Création de la table vol4
CREATE TABLE Vol4 (
        Vol#        NUMBER(4) PRIMARY KEY,
        VD          VARCHAR2(20),
        VA          VARCHAR2(20),
        Pilote      REF pilote4_t);

INSERT INTO vol4
SELECT 200, 'PARIS', 'TOULOUSE', REF(p)
    FROM o_pilote4 p WHERE p.nom ='Jean DUPOND';
```

Les Nouveaux Opérateurs DML

L ’opérateur DEREF

- **Exemple (suite)** :

-- Déréférencement d'une référence
SELECT **DEREF** (Pilote)
FROM Vol4 WHERE VD = 'PARIS';

DEREF(PILOTE)(PL#, NOM, ADRESSE(NUMERO, RUE, CODE_POSTAL, VILLE), TEL, DNAISS, SALAIRE)

PILOTE4_T(1, 'Jean DUPOND', ADRESSE_T(22, 'Mariner', 6555, 'Mt Doc'), '0493825095', '12-JUN-47', 12001)

SELECT Pilote.nom, pilote.adresse.ville
FROM Vol4 WHERE VD = 'PARIS';

Les Nouveaux Opérateurs DML

L'opérateur REFTOHEX

- Permet de convertir explicitement une référence en une chaîne Hexadécimale. Cette conversion peut être obtenue implicitement.

- **Exemple**

SELECT pilote FROM vol4;

PILOTE
00002202085793DF953AEE494F96D4165C98E440A32DC775DDC8F54618A57420A774CDCACC

SELECT REFTOHEX(pilote) FROM vol4;

REFTOHEX(PILOTE)
00002202085793DF953AEE494F96D4165C98E440A32DC775DDC8F54618A57420A774CDCACC

Les Nouveaux Opérateurs DML

- **L ’opérateur IS DANGLING**
  - Retourne TRUE si l’objet référencée n’existe plus.
  - **Exemple** :

```
-- Avant suppression du pilote
SELECT * FROM vol4
WHERE vol#=550 AND Pilote  IS DANGLING;
aucune ligne sélectionnée

DELETE FROM o_PILOTE4 p
WHERE p.nom ='Jean DUPOND’;

-- Après suppression du pilote
SELECT vol# FROM vol4
WHERE vol#=550 AND pilote  IS DANGLING;
      VOL#
      550
```

Nouvelles Extensions DML

L'extension RETURNING REF(…) INTO ...

- **Récupère la référence d'un nouvel enregistrement lors d'une insertion dans une table objet**. Ne peut être utilisé que dans une instruction de MISE A JOUR et ne doit renvoyer qu'une seule valeur
- **NOTE :** Cette clause permet de renvoyer la valeur de n'importe quelle colonne d'une table.
- **Exemple**

```
-- Insertion d 'un pilote et d 'un vol dans un programme PL/SQL
declare
   pil_ref          REF pilote4_t;
Begin
   INSERT INTO o_pilote4 op VALUES ( pilote4_t(3, 'Jean ADAMS', adresse_t(3, 'rue du Port',6000, 'NICE'),
   '0493825033', '12-AUG-64', 10000.5))
          RETURNING REF(op) INTO pil_ref;
   INSERT INTO vol4 VALUES (400, 'Paris', 'Nice', pil_ref) ;
end;
/
```

Les colonnes virtuelle OBJECT_ID et OBJECT_VALUE

- Les colonnes OBJECT_ID et OBJECT_VALUE renvoient respectivement l'OID d'un objet et la valeur d'un objet dans une table objet
  - **Exemple** :

SQL> **SELECT oc.object_id FROM o_cursus oc;**

OBJECT_ID

C693CD6629354EA8B38968409055EBAB

D949D78513D145079B9A0ED685B8839F

065C4099A5A542A193DBC885AE5845D3

17AF08461A964CEAB6EEF3DD7BBB5110

Les colonnes virtuelle OBJECT_ID et OBJECT_VALUE

- Les colonnes OBJECT_ID et OBJECT_VALUE renvoient respectivement l'OID d'un objet et la valeur d'un objet dans une table objet
  - **Exemple** :

SQL> **select oc.object_value from o_cursus oc;**

OBJECT_VALUE(CURSUS_ID, TITRE, DESCRIPTION, LISTECOURS(COURS_ID, LIBELLE, RESPONSABLE, INTERVENANT))

CURSUS_T(1, 'M2 MBDS', 'Multimédia Bases de Données et Intégration de Systèmes',

LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date'), COURS_T(102, 'XML', 'Jacobson', 'Smirnoff')))

CURSUS_T(2, 'M1 MIAGE', 'Méthodes Informatiques Appliquées à la Gestion',

LISTE_DE_COURS_T(COURS_T(101, 'Java', 'Mc Nelly', 'Buffa')))

CURSUS_T(3, 'M1 INFO', 'Master 1 Informatique', LISTE_DE_COURS_T(COURS_T(103, 'UML', 'Booch', 'Jacks on')))

CURSUS_T(4, 'M2 ISI', 'M2 Ingéniérie des Systèmes Informatques', LISTE_DE_COURS_T())

Informations sur les tables objets et les Nested Tables

- ALL_OBJECT_TABLES
- DBA_OBJECT_TABLES
- CDB_OBJECT_TABLES
- USER_OBJECT_TABLES
- ALL_NESTED_TABLES
- DBA_NESTED_TABLES
- CDB_NESTED_TABLES
- USER_NESTED_TABLES
- USER_NESTED_TABLE_COLS
- ALL_NESTED_TABLE_COLS
- DBA_NESTED_TABLE_COLS
- CDB_NESTED_TABLE_COLS

Informations sur les tables objets et les Nested Tables

- **Informations sur les TABLES OBJETS de l'utilisateur connecté**

SELECT table_name **FROM** user_object_tables **ORDER BY** table_name;

```
TABLE_NAME
--------------------
DEPT_O
EMPLOYE_O
O_AVION
O_COURS
O_CURSUS
O_PILOTE
O_PILOTE4
O_VOL

8 lignes slectionnes.
```

Informations sur les tables objets et les Nested Tables

Informations sur les TABLES IMBRIQUEES (Nested Tables) de l'utilisateur connecté

```
set linesize 200
col TABLE_NAME format A20
col TABLE_TYPE_NAME format A20
col PARENT_TABLE_NAME format A20
col PARENT_TABLE_COLUMN format A20

SELECT  TABLE_NAME, TABLE_TYPE_NAME, PARENT_TABLE_NAME, PARENT_TABLE_COLUMN
FROM user_nested_tables ORDER BY table_name ;
```

TABLE_NAME	TABLE_TYPE_NAME	PARENT_TABLE_NAME	PARENT_TABLE_COLUMN
STORELISTREFCOURS	LISTE_REF_DE_COURS_T	O_CURSUS	LISTREFCOURS
TABLE_ALISTREFVOLS	LISTREFVOLS_T	O_AVION	ALISTREFVOLS
TABLE_LISTREFEMPS	LISTREFEMPLOYES_T	DEPT_O	LISTREFEMPS
TABLE_PLISTREFVOLS	LISTREFVOLS_T	O_PILOTE	PLISTREFVOLS

Informations sur les tables objets et les Nested Tables

- **Informations sur les COLONNES des TABLES IMBRIQUEES (Nested Tables) de l'utilisateur connecté**

```
set linesize 200
col TABLE_NAME format A20
col COLUMN_NAME format A20
col DATA_TYPE format A20
col DATA_LENGTH format A20
SELECT  TABLE_NAME, COLUMN_NAME,DATA_TYPE, DATA_LENGTH
FROM  USER_NESTED_TABLE_COLS ORDER BY table_name ;
```

```
TABLE_NAME           COLUMN_NAME          DATA_TYPE
-------------------- -------------------- ----------
STORELISTREFCOURS    NESTED_TABLE_ID      RAW
STORELISTREFCOURS    COLUMN_VALUE         COURS_T
TABLE_ALISTREFVOLS   NESTED_TABLE_ID      RAW
TABLE_ALISTREFVOLS   COLUMN_VALUE         VOL_T
TABLE_LISTREFEMPS    NESTED_TABLE_ID      RAW
TABLE_LISTREFEMPS    COLUMN_VALUE         EMPLOYE_T
TABLE_PLISTREFVOLS   NESTED_TABLE_ID      RAW
TABLE_PLISTREFVOLS   COLUMN_VALUE         VOL_T

8 lignes slectionnes.
```

Module M3.3, section 1 : QUIZ

➢ **Question 1** : cochez ce qui caractérise une table objet

- A: Une table objet ne peut contenir des colonnes définies sur un type scalaire (char, varchar2, date, etc.)
- B: Une table objet ne peut contenir des colonnes définies sur un type complexe (Type Abstrait de Données : TAD ou TDA)
- C: Les lignes d'une table objet sont identifiées par ROWID (identifiant physique Oracle)
- D: Les lignes d'une table objet sont identifiées par des OID (Object Identifier)

➢ **Question 2** : cochez ce qui caractérise une table relationnelle SQL2

- A: Une table relationnelle SQL2 ne peut contenir des colonnes définies sur un type scalaire (char, varchar2, date, etc.)
- B: Une table relationnelle SQL2 ne peut contenir des colonnes définies sur un type complexe (Type Abstrait de Données : TAD ou TDA)
- C: Les lignes d'une table relationnelle SQL2 sont identifiées par ROWID (identifiant physique Oracle)
- D: Les lignes d'une table relationnelle SQL2 sont identifiées par des OID (Object Identifier)

➢ **Question 3** : Cochez ce qui correspond à la syntaxe de création d'une table Objet

- A: CREATE TABLE nomTable (c1 typeDeC1, C2 typeDeC2, …);
- B: CREATE TABLE nomTable of nomTypeComplexe (…);
- C: CREATE TABLE nomTable as select * from uneTableObjet;
- D: CREATE TABLE nomTable as select * from uneTableRelationnelle;

Module M3.3, section 1 : QUIZ

- **Question 4** : Cochez ce qui correspond à la syntaxe de création d'une table Relationnelle SQL2
  - A: CREATE TABLE nomTable (c1 typeDeC1, C2 typeDeC2, …);
  - B: CREATE TABLE nomTable of nomTypeComplexe (…);
  - C: CREATE TABLE nomTable as select * from uneTableObjet;
  - D: CREATE TABLE nomTable as select * from uneTableRelationnelle;

- **Question 5** : Considérons la table o_pilote4 créée dans le Module M3.3, section 1. Que fait la requête suivante

  SELECT VALUE(op) FROM O_PILOTE4;
  - A: Cette requête renvoie les références vers les pilotes insérés dans la table O_PILOTE4
  - B: Cette requête renvoie les instances (**pilote4_t(…)**) des pilotes insérés dans la table O_PILOTE4
  - C: Cette requête renvoie les valeurs (**pl#, plnom, …**) des pilotes insérés dans la table O_PILOTE4
  - D: Aucune des actions précédentes ne fonctionne

- **Question 6** : Considérons la table o_pilote4 créée dans le Module M3.3, section 1. Que fait la requête suivante

  SELECT REF(op) FROM O_PILOTE4;
  - A: Cette requête renvoie les références vers les pilotes insérés dans la table O_PILOTE4
  - B: Cette requête renvoie les instances (**pilote4_t(…)**) des pilotes insérés dans la table O_PILOTE4
  - C: Cette requête renvoie les valeurs (**pl#, plnom, …**) des pilotes insérés dans la table O_PILOTE4
  - D: Aucune des actions précédentes ne fonctionne

Module M3.3, section 1 : QUIZ

- **Question 7** : Cochez ce qui est valide concernant l'opérateur DEREF
  - A: DEREF appliqué à une colonne de type REF renvoie l'instance de l'objet pointé
  - B: DEREF appliqué à une colonne de type REF renvoie la valeur de l'objet pointé
  - C: Il est possible d'appliqué DEREF sur des colonnes de type CLOB
  - D: Il est possible d'appliqué DEREF sur des colonnes de type VARCHAR2
  - E: La réponse B est vrai

- **Question 8** : Cochez ce qui est valide concernant la clause RETURNING. Considérons la table objet o_pilote4 créée dans le Module M3.3, section 1
  - A: DECLARE nom1 o_pilote4.nom%type; BEGIN select * FROM o_pilote4 op WHERE op.pl#=1 RETURNING nom INTO nom1; END;
  - B: DECLARE nom1 o_pilote4.nom%type; BEGIN INSERT INTO o_pilote4 op (pl#, nom, …) VALUES (10, 'Rex', …) RETURNING nom INTO nom1; END;
  - B: DECLARE refPilote Ref pilote4_t; BEGIN INSERT INTO o_pilote4 op (pl#, nom, …) VALUES (10, 'Rex', …) RETURNING ref (of) INTO refPilote; END;
  - C: A, B et C sont faut

Module M3.3, section 2, partie 1 : Les liens d'association

Module M3.3, section 2, partie 1 : Les liens d'association

Plan

- Généralités
- Etapes de mise en place d'une application objet relationnelle
- Définition du schéma conceptuel avec MERISE, UML, Entité/Relation, …
- Définition du schéma types
- Création des types
- Création des tables et définition des contraintes d'intégrités
- Création d'indexes
- *Insertion des lignes des tables ayant des liens d'association*
- *Mise à jour des lignes des tables ayant des liens d'association*
- *Suppression des lignes des tables ayant des liens d'association*
- *Consultation des tables ayant des liens d'association*
- *Utilisation des opérateurs sur les collections*
- *Autres considérations*

Module M3.3, section 2, partie 1 : Les liens d'association

➢ Généralités

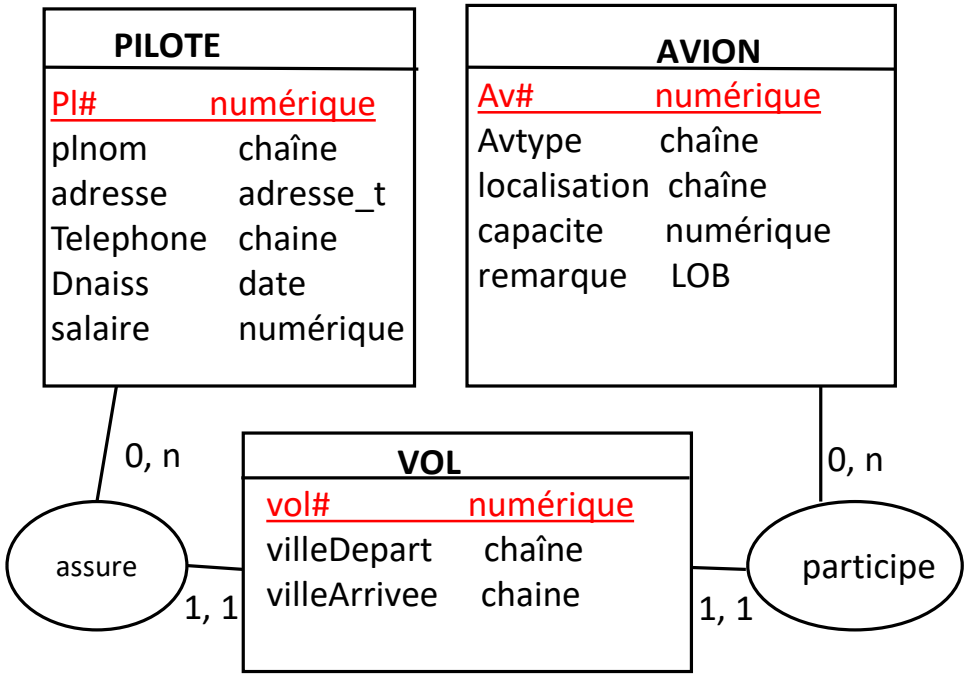
- D'un point de vue conceptuel, un lien d'association entre deux ENTITES **est une relation (de type 1:N ou N:M)** entre les instances de ces deux ENTITES.
- **La mise en œuvre** des liens d'associations se fait à travers la **notion de Référence et de collections de références**
- Peuvent être qualifiés de **jointure** « **cablée** »
- Peuvent être posés au **niveau table** ou au **niveau Type**
- Doivent être entièrement **gérés par le développeur**

Etapes de mise en place d'une application objet relationnelle

- Etape 1 : Définir le schéma conceptuel avec MERISE, UML, Entité/Relation, ...
- Etape 2 : Définir le schéma types
- Etape 3 : Créer les types
- Etape 4 : Créer les tables et définir les contraintes d'intégrités
- Etape 5 : Créer les indexes si nécessaire
- Etape 6 : Insérer des lignes dans les tables.

Module M3.3, section 2, partie 1 : Les liens d'association

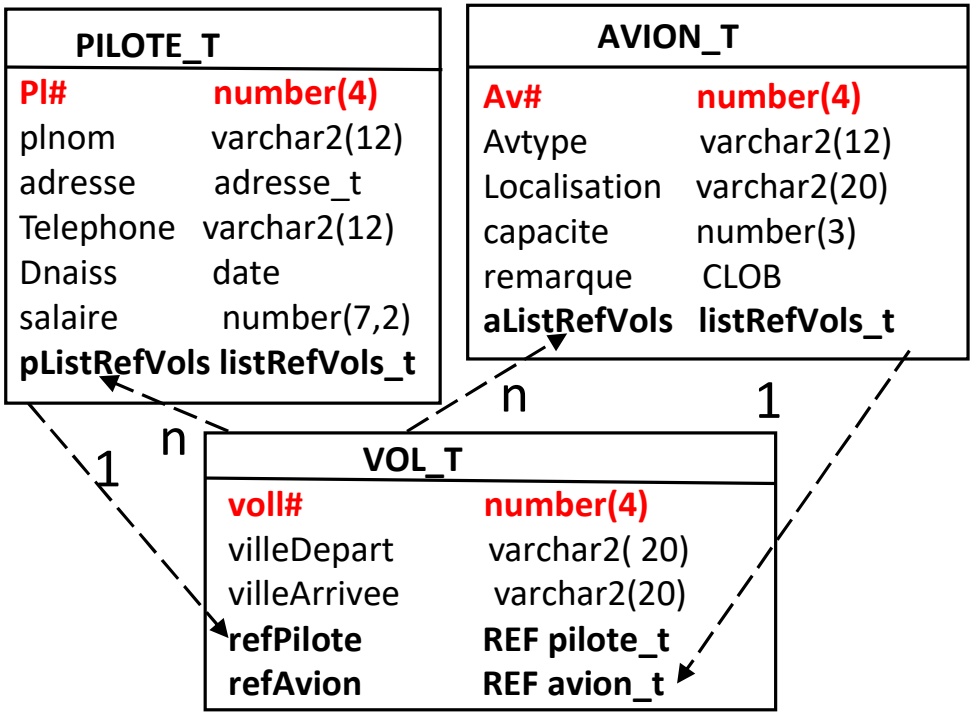
- **Définition du schéma conceptuel avec MERISE, UML, Entité/Relation, …**
  - Nous allons dans cette section mettre en place une application : Du schéma conceptuel en passant par le schéma objet relationnel jusqu'à l'insertion des lignes.
  - Soit le schéma conceptuel MERISE suivant



Module M3.3, section 2, partie 1 : Les liens d'association

➢ Définition du schéma types

- Implémentation du **schéma Merise en Objet Relationnel Oracle** (schéma de type)



listRefVol_t : Type collection de références Vers les VOLs d'un PILOTE ou d'un AVION.

Création des types

```
-- Suppression des tables et des types s'ils
-- existent: phase développement
Drop table o_vol cascade constraints;
Drop table o_pilote cascade constraints;
Drop table o_avion cascade constraints;
Drop type avion_t force;
Drop type pilote_t force;
Drop type vol_t force;
Drop type listRefVols_t ;
Drop type adresse_t;
```

```
-- Déclaration Forward des Types afin d'éviter l'interblocage dû au
référencement mutuel entre types

CREATE OR REPLACE TYPE VOL_T
/
--Création de type table (listRefVols_t) : Les éléments de cette liste
sont des références vers des vols.

-- Solution 1: avec handle personnalisé
Create or replace type refVol_t as object(
 refVol      REF  VOL_T)
/
CREATE OR REPLACE TYPE listRefVols_t  AS TABLE OF refVol_t
/
```

Module M3.3, section 2, partie 1 : Les liens d'association

➢Création des types

```
-- Solution 2: avec handle implicite appelé Column_Value
-- Le handle permet d'accéder aux éléments de la liste.
CREATE OR REPLACE TYPE listRefVols_t
AS TABLE OF REF VOL_T
/
```

NOTE : C'est la solution 2 qui a été retenue

```
--Création du  type PILOTE_T avec un champ liste pListRefVols
CREATE OR REPLACE TYPE adresse_t as object(
        Numero          NUMBER(4),
        Rue             VARCHAR2(20),
        Code_Postal     NUMBER(5),
        Ville           VARCHAR2(20)
)
/
```

```
CREATE OR REPLACE TYPE Pilote_t AS OBJECT (
        PL#             NUMBER(4),
        plnom           VARCHAR2(12),
        adresse         ADRESSE_T,
        Telephone       VARCHAR2(12),
        dnaiss          DATE,
        salaire         NUMBER(7,2),
        pListRefVols    listRefVols_t
)
/
```

BY NC ND

Création des types

--Création du type AVION_T avec un champ multivalué aListRefVols

```
CREATE OR REPLACE TYPE Avion_t  AS OBJECT(
      Av#              NUMBER(3),
      avtype           VARCHAR2(12),
      localisation     VARCHAR2(20),
      capacite         NUMBER(3),
      remarque         CLOB,
      aListRefVols     listRefVols_t
)
/
```

--Création du type VOL_T avec 2 champs de type REF

```
CREATE OR REPLACE  TYPE  Vol_t AS OBJECT(
      Vol#             NUMBER(4),
      VilleDepart      VARCHAR2(20),
      VilleArrivee     VARCHAR2(20),
      refPilote        REF pilote_t,
      refAvion         REF avion_t
)
/
```

Création des tables et définition des contraintes d'intégrités

```
-- Table o_pilote
CREATE TABLE O_PILOTE OF PILOTE_T(
CONSTRAINT pk_o_pilote_pl# PRIMARY KEY(pl#),
CONSTRAINT chk_o_pilote_salaire CHECK (salaire >2000 AND Salaire<70000))
NESTED TABLE pListRefVols STORE AS table_pListRefVols;
-- table_pListRefVols : table de Ref vers les vols des pilotes
```

```
-- Table o_avion
CREATE TABLE O_AVION OF AVION_T(
CONSTRAINT pk_o_avion_av# PRIMARY KEY(AV#))
NESTED TABLE aListRefVols STORE AS table_aListRefVols;
-- table_aListRefVols : table de Ref vers les vols des avions

-- Table o_vol
CREATE TABLE O_VOL OF VOL_T(
CONSTRAINT pk_o_vol_vol# PRIMARY KEY(vol#));
```

Création des tables et définition des contraintes d'intégrités

Limitation de la visibilité des Références (SCOPE FOR) et création de tables

- Le mot **clé SCOPE FOR** permet **de limiter la visibilité des références** à une table particulière. On peut aussi parler d'intégrité de REFERENCE sur les REFs
- La table sur laquelle le ALTER TABLE est exécutée **doit être vide**

ALTER TABLE **table_pListRefVols** ADD (**SCOPE FOR** (column_value) IS O_VOL);

ALTER TABLE **table_aListRefVols** ADD (**SCOPE FOR** (column_value) IS O_VOL);

ALTER TABLE O_VOL ADD (**SCOPE FOR** (refPilote) IS O_PILOTE);

ALTER TABLE O_VOL ADD (**SCOPE FOR** (refAvion) IS O_AVION);

Création d'indexes

- **Création d'indexes sur la colonne virtuelle NESTED_TABLE_ID des NESTED TABLE.**
  A partir d'Oracle 11g il n'est nécessaire de les créer. Ils le sont automatiquement.

  - Cet **index** est **nécessaire** pour **accélérer l'accès aux éléments d'une NESTED TABLE** attaché à une MEME LIGNE dans la table « MAITRE »

    ✓ **Création d'un index sur la Nested table table_pListRefVols :**
    CREATE INDEX idx_tprv_nested_table_id on **table_pListRefVols**(NESTED_TABLE_ID)
    TABLESPACE USERS;

    ✓ **Création d'un index sur la Nested table table_aListRefVols :**
    CREATE INDEX idx_tarv_nested_table_id on **table_aListRefVols**(NESTED_TABLE_ID)
    TABLESPACE USERS;

Création d'indexes

Création d'indexes sur les colonnes de type REF

- Il est maintenant possible (Oracle 10.2) de **créer des indexes sur des colonnes de type référence**
- **Exemple** de création d'indexes sur les colonnes **refPilote** et **refAvion** de la table O_VOL
  - ✓ **Création d'un index sur la colonne refPilote**
    CREATE INDEX idx_o_vol_refPilote on O_VOL(refPilote)
    TABLESPACE USERS;
  - ✓ **Création d'un index sur la colonne refPilote**
    CREATE INDEX idx_o_vol_refAvion on O_VOL(refAvion)
    TABLESPACE USERS;
- **NOTE**
  - Une contrainte SCOPE FOR doit avoir été posée sur la colonne REF

Module M3.3, section 2, partie 1 : QUIZ

- **Question 1 : Cochez ce qui est juste**
  - A: Les méthodes de conception d'un schéma conceptuel (MERISE, UML ou équivalent) sont aussi valables pour les applications objets relationnelles
  - B: Si schéma conceptuel il ya, il doit être traduit en un schéma de type
  - C: Seules les lignes des tables objets sont identifiées par des OID (par extrapolation des REF)
  - D: Les lignes de toute table créée dans Oracle sont identifiées par OID (par extrapolation des REF)

- **Question 2 : Lien d'association de cardinalité 1. cochez la bonne traduction de ce lien du schéma conceptuel vers le schéma de type**
  - A: Il sera matérialisé par un champ collection de REF (Type Nested table)
  - B: Il sera matérialisé par un champ REF
  - C: Il sera matérialisé par un champ clé étrangère
  - D: Il sera matérialisé par un champ collection de clés étrangères

- **Question 3 : Lien d'association de cardinalité N: cochez la bonne traduction de ce lien du schéma conceptuel vers le schéma de type**
  - A: Il sera matérialisé par un champ collection de REF (Type Nested table)
  - B: Il sera matérialisé par un champ REF
  - C: Il sera matérialisé par un champ clé étrangère
  - D: Il sera matérialisé par un champ collection de clés étrangères

Module M3.3, section 2, partie 1 : QUIZ

- **Question 4 : C**ochez ce qui caractérise la clause SCOPE FOR
  - A: La clause SCOPE permet d'indiquer l'origine (de quelle table) des REF a affecter à une colonne REF
  - B: SCOPE FOR joue pour les tables objets, le même rôle que FOREIGN KEY pour les tables relationnelles SQL2
  - C: La taille d'une référence limitée via SCOPE FOR diminunue de 16 Octets. Au lieu de 42 pour un REF normal
  - D: SCOPE FOR ne s'applique que sur les colonnes REF
  - D: SCOPE FOR ne s'applique que sur les colonnes de tout type

- **Question 5 : Nous souhaitons créer un index sur une colonne de type REF. Cochez ce qui est juste**
  - A: Il n'est pas possible de créer un index sur une colonne de type REF
  - B: Il est possible de créer un index sur une colonne de type REF en toute condition
  - C: La création de l'index doit être précédée de l'application de SCOPE FOR à la colonne REF
  - D: La création de l'index doit être suivie de l'application de SCOPE FOR à la colonne REF

Module M3.3, section 2, partie 2 : Les liens d'association

Module M3.3, section 2, partie 2 : Les liens d'association

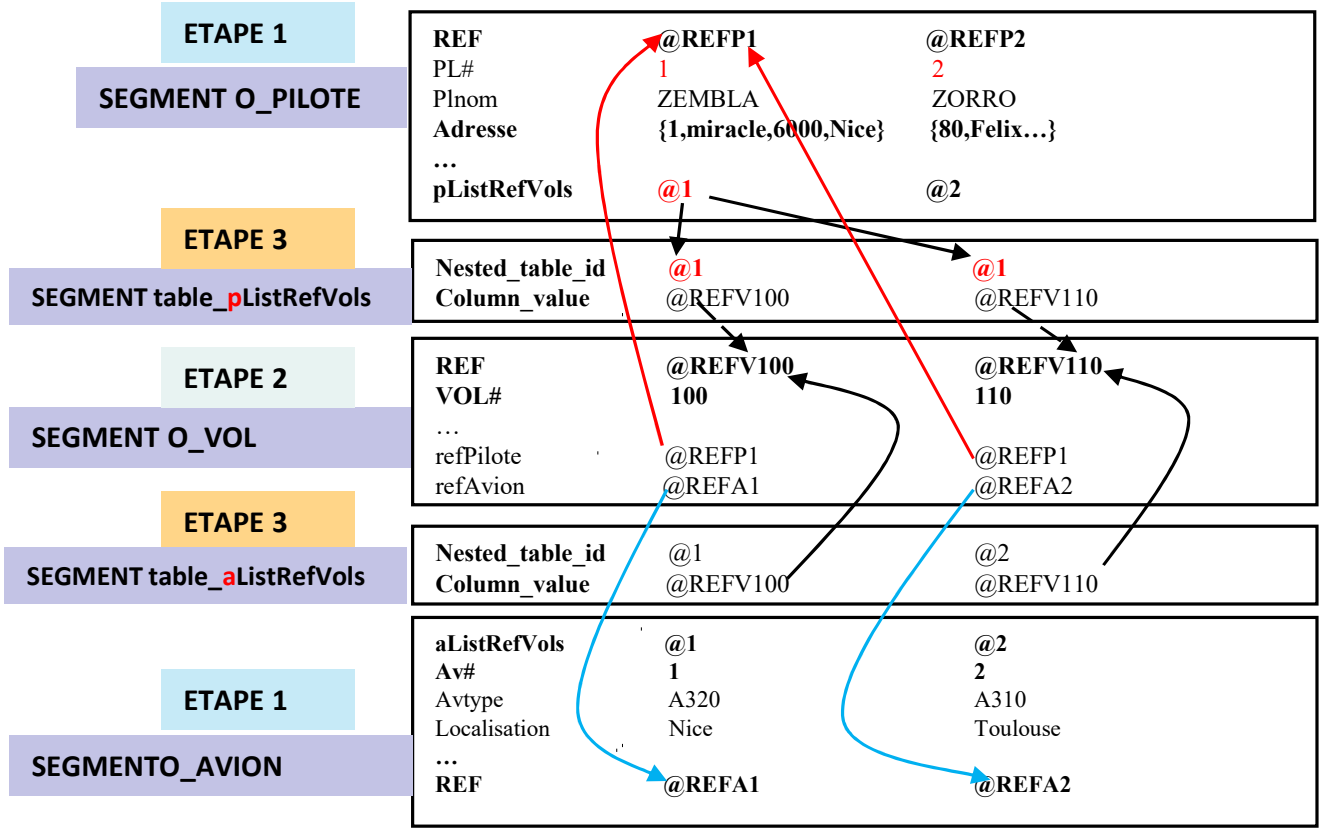
Plan

- *Généralités*
- *Etapes de mise en place d'une application objet relationnelle*
- *Définition du schéma conceptuel avec MERISE, UML, Entité/Relation, …*
- *Définition du schéma types*
- *Création des types*
- *Création des tables et définition des contraintes d'intégrités*
- *Création d'indexes*
- **Insertion des lignes des tables ayant des liens d'association**
- **Mise à jour des lignes des tables ayant des liens d'association**
- **Suppression des lignes des tables ayant des liens d'association**
- *Consultation des tables ayant des liens d'association*
- *Utilisation des opérateurs sur les collections*
- *Autres considérations*

Module M3.3, section 2, partie 2 : Les liens d'association

Insertion des lignes des tables ayant des liens d'association

- **Création des objets par étapes**
  - **Etape 1:** Insertion dans les tables ayant des colonnes ***Nested table***: avec des ***Nested Table*** vides
  - **Etape 2:** Insertion dans les tables ayant des Colonnes REF
  - **Etape 3:** Mise à jour des Nested Tables créées à Vide dans l'étape 1



Insertion des lignes des tables ayant des liens d'association

- **Insertion à partir d'un programme PL/SQL**

```
declare
refavion1       REF avion_t:=NULL;
refavion2       REF avion_t:=NULL;
refpilote1      REF pilote_t:=NULL;
refpilote2      REF pilote_t:=NULL;
refVol1   REF vol_t:=NULL;
refVol2   REF vol_t:=NULL;

BEGIN
```

Insertion des lignes des tables ayant des liens d'association

```
-- Etape 1: Insertion des lignes dans les tables
-- O_PILOTE et O_AVION avec des listes vides

-- insertion dans O_PILOTE
INSERT INTO O_PILOTE OP VALUES (
  pilote_t(1, 'ZEMBLA', adresse_t(1, 'rue des Miracle',6000, 'NICE'),
    '0622732123', ' 01/01/1982 ', 30000.00, listRefVols_t())) returning ref(op) into refPilote1;

INSERT INTO O_PILOTE OP VALUES (
  pilote_t(2, 'ZORRO', adresse_t(80, 'Avenue Felix Faure',6000, 'Bruxelles'),
    '0622711111', ' 25/12/1973', 40000.00, listRefVols_t()) ) returning ref(op) into refPilote2;
```

Insertion des lignes des tables ayant des liens d'association

```
--Etape 1: Insertion des lignes dans les tables
--O_PILOTE et O_AVION avec des listes vides

-- Insertion dans la table objet O_AVION

INSERT INTO O_AVION oa VALUES (
AVION_T(1,'A320', 'Nice',350, 'En Service', listRefVols_t())) returning ref(oa) into refAvion1 ;

INSERT INTO O_AVION  oa VALUES (
AVION_T(2,'A310', 'Toulouse',300, 'En Service', listRefVols_t())) returning ref(oa) into refAvion2 ;
```

Insertion des lignes des tables ayant des liens d'association

```
--Etape 2 : Insertion des lignes dans la table O_VOL
-- avec mise à jour des références

INSERT INTO O_VOL ov
Values(100, 'Nice', 'Lyon', refPilote1, refAvion1) Returning ref(ov) INTO refVol1;

INSERT INTO O_VOL OV
Values(110, 'Paris', 'Toulouse', refPilote1, refAvion2) Returning ref(ov) INTO refVol2;
```

Insertion des lignes des tables ayant des liens d'association

```
-- Etape 3 : Mise à jour indirecte des tables imbriquées
-- table_pListRefVols et table_aListRefVols contenant les

-- Insertion indirecte dans table_pListRefVols
INSERT INTO  TABLE(SELECT p.pListRefVols  FROM o_pilote p
WHERE p.pl# = 1)   values (refVol1);

INSERT INTO  TABLE(SELECT p.pListRefVols  FROM o_pilote p
WHERE p.pl# = 1)   values (refVol2);

END;
/
```

```
-- Insertion indirecte dans table_aListRefVols
INSERT INTO TABLE(SELECT a.aListRefVols  FROM o_avion a WHERE
   a.av#=1)
values (refVol1);

INSERT INTO
TABLE(SELECT a.aListRefVols  FROM o_avion a WHERE a.av#=2)
values (refVol2);

END;
/
```

Mise à jour des lignes des tables ayant des liens d'association

- **Modifier le salaire du pilote qui assure le vol 100. Le mettre à 70000**

```
UPDATE O_VOL OV
SET OV.refPilote.salaire=70000
WHERE ov.VOL#=100;
```

Suppression des lignes des tables ayant des liens d'association

- Soient **deux entités E1 et E2** associées à travers un **lien 1:N** ou, à une occurrence Oe1 de E1 est associée N occurrences Oe2i de E2 (i : 0 à N)

- La suppression à travers un lien consiste à assurer l'intégrité de données**. Lors de la suppression de Oe1 par exemple, tous les Oe2i associés seront supprimés**

- **NOTE :** La gestion des liens doit, pour une plus grande efficacité, être prise en compte à travers des méthodes

Suppression des lignes des tables ayant des liens d'association

- **Supprimer la référence au VOL 100 dans liste des vols d'un pilote donné connaissant son numéro**

```
DECLARE
  refvol1                REF VOL_T;
  piloteNum              Number;
BEGIN
  SELECT v.refPilote.pl# INTO piloteNum FROM o_vol v where v.vol#=100;

  DELETE FROM o_VOL V6 WHERE vol# = 100  RETURNING REF(v6) INTO refvol1;
  DELETE FROM TABLE(
    SELECT p6.pListRefVols FROM o_pilote p6
    WHERE p6.pl#=piloteNum ) listvols   WHERE listvols.column_value = refvol1;
  END;
  /
```

Suppression des lignes des tables ayant des liens d'association

- **Exemple :Suppression du Pilote numéro 2 et ses vols. Variante 1**

```
Declare
  listVols    listRefVols_T;
  i           INTEGER;
BEGIN
  SELECT p6.plistRefvols INTO listVols FROM o_pilote p6 WHERE p6.pl#=2;

  FOR i IN 1.. listVols.COUNT LOOP
    DELETE FROM o_vol v WHERE  REF(v) = listVols(i);
  END LOOP;

  DELETE FROM o_pilote WHERE pl# = 2;
END;
/
```

Suppression des lignes des tables ayant des liens d'association

- **Exemple :Suppression du Pilote numéro 2 et ses vols. Variante 2**

```
-- Supposant l'index sur la colonne refPilote posé
Declare
refPilote1        REF pilote_t;
BEGIN
  DELETE FROM o_pilote  p6  WHERE pl# = 2 RETURNING  ref(p6) INTO refpilote1;
  DELETE FROM o_vol v6   WHERE v6.refPilote=refPilote1;
END;
/
```

Notes : d'autres possibilités de suppression existent

Suppression des lignes des tables ayant des liens d'association

- **Exemple :Suppression du Pilote numéro 2 et ses vols. Variante 3**

```
-- Supposant l'index sur la colonne refPilote posé
DELETE FROM o_vol v6   WHERE v6.refPilote.pl#=2;
DELETE FROM o_pilote  p6  WHERE pl# = 2;
```

Suppression des lignes des tables ayant des liens d'association

- **Suppression dans une Nested Table contenant des REFERENCEs vers des objets déjà supprimés**
- **L'opérateur IS [NOT] DANGLING permet d'identifier les références qui ne pointent plus vers un objet.**
- **Exemple**
  - suppression de toutes les références au VOLs supprimés du PILOTE Nr. 2

```
DELETE FROM TABLE(SELECT p6.pListRefVols
FROM o_pilote p6 WHERE p6.pl#=2) listvols
WHERE listvols.column_value IS DANGLING ;
```

Module M3.3, section 2, partie 2 : QUIZ

Considérons les types PILOTE_T, AVION_T et AVION_T. Considérons les tables O_PILOTE, O_AVION et O_VOL : définies dans la section Module M3.3, section 2, partie 1. Considérons la requête suivante :

INSERT INTO O_PILOTE OP VALUES (pilote_t(1, 'ZEMBLA', adresse_t(1, 'rue des Miracle',6000, 'NICE'), '0622732123', ' 01/01/1982 ', 30000.00, **listRefVols_t()**)) returning **ref(op)** into **refPilote1**;

- **Question 1 : cochez ce qui est juste**
  - A: Cette requête ne peut fonctionner que si elle est encapsulée dans un programme PLSQL
  - B: La clause RETURNING permet d'affecter le pointeur vers le pilote nr 1 à une variable refPilote1
  - C: L'invocation du constructeur listRefVols_t() créé une liste vide
  - D: L'invocation du constructeur listRefVols_t() créé une liste initialisée à NULL
  - E: La clause RETURNING créée une référence et l'affecte à la variable refPilote1

- **Question 2 : cochez ce qui est juste**
  - A: L'insertion du pilote Nr 1 entraine la mise à jour automatique du lien inverse dans la table O_VOL
  - B: L'insertion du pilote Nr 1 implique la mise à jour manuelle du lien inverse dans la table O_VOL
  - C: refPilote1 servira pour la mise à jour manuelle du lien inverse dans la table O_VOL

Module M3.3, section 2, partie 2 : QUIZ

Considérons les types PILOTE_T, AVION_T et AVION_T. Considérons les tables O_PILOTE, O_AVION et O_VOL : définies dans la section Module M3.3, section 2, partie 1. Considérons la requête suivante :

INSERT INTO O_PILOTE OP VALUES (pilote_t(1, 'ZEMBLA', adresse_t(1, 'rue des Miracle',6000, 'NICE'), '0622732123', ' 01/01/1982 ', 30000.00, **listRefVols_t()**)) returning **ref(op)** into **refPilote1**;

INSERT INTO O_VOL ov

Values(100, 'Nice', 'Lyon', **refPilote1, refAvion1) Returning ref(ov) INTO refVol1;**

➢ **Question 3 : cochez ce que fait la requête suivante :**

SELECT ov.vol#, ov.**refPilote**.nom, ov.**refAvion**.avtype **FROM o_vol** ov **WHERE** ov.vol#=100;

- A: Cette requête opère une jointure entre les tables O_PILOTE, O_AVION et O_VOL
- B: Cette requête évite la jointure entre les tables O_PILOTE, O_AVION et O_VOL;
- C: Cette est extrêmement rapide par rapport ce qu'on aurait fait en SQL2
- D: L'équivalent en SQL2 aurait été de remplacer les REF par les clés étrangères

Module M3.3, section 2, partie 2 : QUIZ

Considérons les types PILOTE_T, AVION_T et AVION_T. Considérons les tables O_PILOTE, O_AVION et O_VOL : définies dans la section Module M3.3, section 2, partie 1. Considérons la requête suivante :

R1: INSERT INTO O_PILOTE OP VALUES (pilote_t(1, 'ZEMBLA', adresse_t(1, 'rue des Miracle',6000, 'NICE'), '0622732123', ' 01/01/1982 ', 30000.00, **listRefVols_t()**)) returning **ref(op)** into **refPilote1**;

R2: INSERT INTO O_VOL ov
Values(100, 'Nice', 'Lyon', **refPilote1, refAvion1) Returning ref(ov) INTO refVol1;**

R3: INSERT INTO **TABLE**(SELECT p.pListRefVols FROM o_pilote p
WHERE p.pl# = 1) **values (refVol1);**

➢ **Question 4 : cochez ce qui est juste**

- A: La requête R3 ajoute la référence refVol1 dans la colonne plistRefVols de la table o_pilote pour le pilote nr1
- B: La requête R3 ajoute la référence refVol1 dans la table imbriqué **table_pListRefVols**
- C: Toutes les références vers les vols de chaque pilote sont stockées dans la table **table_pListRefVols**
- **D: Non non la requête R3 est mal écrite. Il faut faire : INSERT INTO table_pListRefVols values (refVol1);**

Module M3.3, section 2, partie 2 : QUIZ

Considérons les types PILOTE_T, AVION_T et AVION_T. Considérons les tables O_PILOTE, O_AVION et O_VOL : définies dans la section Module M3.3, section 2, partie 1. Considérons la requête suivante :

Supposons que le vol dont le pointeur est refVol1 a été supprimé. Il faut enlever ce pointeur de la liste des pointeurs vers les vols du pilote Nr 1.

- **Question 5 : cochez les requêtes justes**

A: DELETE FROM **TABLE**(SELECT p.pListRefVols FROM o_pilote p WHERE p.pl# = 1) lrv WHERE lrv.column_value= **refVol1;**

B: delete from table_pListRefVols where column_value = refVol1;

C: A est faut

Module M3.3, section 2, partie 2 : QUIZ

Considérons les types PILOTE_T, AVION_T et AVION_T. Considérons les tables O_PILOTE, O_AVION et O_VOL : définies dans la section Module M3.3, section 2, partie 1. Considérons la requête suivante :

Supposons la requête suivante : DELETE FROM PILOTE WHERE PL#=1;

➢ **Question 6 : Caractérisez ce que devient les pointeurs vers le pilote supprimé dans la table O_VOL**

A: Ils sont automatique mis à NULL

B: Ils sont IS DANGLING

C: A est faut

Module M3.3, section 2, partie 3 : Les liens d'association

Module M3.3, section 2, partie 3 : Les liens d'association

Plan

- *Généralités*
- *Etapes de mise en place d'une application objet relationnelle*
- *Définition du schéma conceptuel avec MERISE, UML, Entité/Relation, …*
- *Définition du schéma types*
- *Création des types*
- *Création des tables et définition des contraintes d'intégrités*
- *Création d'indexes*
- *Insertion des lignes des tables ayant des liens d'association*
- *Mise à jour des lignes des tables ayant des liens d'association*
- *Suppression des lignes des tables ayant des liens d'association*
- **Consultation des tables ayant des liens d'association**
- **Utilisation des opérateurs sur les collections**
- **Autres considérations**

Consultation des tables ayant des liens d'association

- **Donner le numéro du vol, le nom du pilote et le type d'avion du Vol 100 avec déréférencement implicite. Opérateur .**

  SELECT v6.vol#, v6.refpilote.plnom, v6.refavion.avtype **FROM** o_vol v6 **WHERE** v6.vol#=100;

VOL#	PILOTE.PLNOM	AVION.AVTYPE
100	ZEMBLA	A320

- **Informations sur l'avion participant au VOL 100 avec déréférencement explicite. Opréteur DEREF**

  SELECT **DEREF**(v6.refAvion) **FROM** o_vol v6 **WHERE** vol#=100;

```
DEREF(V6.REFAVION)(AV#, AVTYPE, LOCALISATION, CAPACITE, REMARQUE, ALISTREFVOLS)
--------------------------------------------------------------------------------
AVION_T(1, 'A320', 'Nice', 350, 'En Service', LISTREFVOLS_T(0000220208BFA7B8DC8E
324945A3FE36B5EC8DFAC54DF0012326504E34A82BC8813325281C))
```

Consultation des tables ayant des liens d'association

- **Donner la liste des vols d'un Pilote connaissant son numéro et son nom.**

```
SELECT v.column_value.vol#, v.column_value.villeDepart
 FROM TABLE(SELECT p6.pListRefVols FROM o_pilote p6 WHERE p6.pl#= 1 and p6.plnom='ZEMBLA' ) v;
```

```
COLUMN_VALUE.VOL# COLUMN_VALUE.VILLEDE
----------------- --------------------
              100 Nice
              110 Paris
```

```
SELECT p6.pListRefVols(1).vol#  FROM o_pilote p6 WHERE p6.pl#= 1 AND p6.plnom='ZEMBLA' ;
  ERREUR Ó la ligne 1 :
  ORA-00904: "P6"."PLISTREFVOLS" : identificateur non valide
```

Utilisation des opérateurs sur les collections

Quelsques opérateurs sur les Nested Table

- CURSOR
- **CAST(MULTISET…)**
- TABLE
- **SUBMULTISET**,
- CARDINALITY
- **MEMBER OF**
- IS [NOT] EMPTY
- **IS A SET,**
- COLLECT,
- **MULTISET {UNION | INTERSECT | EXCEPT}**
- POWERMULITSET,
- **POWERMULTISET_BY_CARDINALITY,**
- SET,

Utilisation des opérateurs sur les collections

L'opérateur CURSOR

- L'opérateur **CURSOR** permet de restituer un ensemble d'objets issus d'une Nested Table
- **Exemple 1 :** Donner la liste des vols de chaque Pilote. En utilisant les opérateurs CURSOR et TABLE

SELECT **CURSOR**(
SELECT p6.plnom, v.column_value.vol#, v.column_value.villeDepart
FROM **TABLE** (p6.pListRefVols) v)FROM o_pilote p6;

```
CURSOR(SELECTP6.PLNO
--------------------
CURSOR STATEMENT : 1

CURSOR STATEMENT : 1

PLNOM                            COLUMN_VALUE.VOL# COLUMN_VALUE.VILLEDE
-------------------------------- ----------------- --------------------
ZEMBLA                                         100 Nice
ZEMBLA                                         110 Paris

CURSOR STATEMENT : 1

CURSOR STATEMENT : 1

aucune ligne sÚlectionnÚe
```

BY NC ND

Utilisation des opérateurs sur les collections

- **L'opérateur CAST(MULTISET …)**
  - **Syntaxe : CAST({MULTISET(<requête>) | expr} AS nested_type)**
  - Cet opérateur permet de transformer (convertir) le résultat d'une "requête" en une « nested table ».
  - **Exemple 1 :** Donner la liste des références de vols d'un Pilote connaissant son numéro en utilisant les opérateurs CAST et Multiset

```
SELECT CAST ( MULTISET( SELECT REF(v) FROM o_vol v WHERE v.refPilote.pl# IN (1,2)  ) AS   listRefVols_t )
FROM dual;
CAST(MULTISET(SELECTREF(V)FROMO_VOLVWHEREV.REFPILOTE.PL#IN(1,2))ASLISTREFVOLS_T)
--------------------------------------------------------------------------------
LISTREFVOLS_T(0000280209BFA7B8DC8E324945A3FE36B5EC8DFAC54DF0012326504E34A82BC881
3325281C010157CC0000, 0000280209A6FC5F9DDDB54C918F49A5BAD9869EC24DF0012326504E34
A82BC8813325281C010157CC0001)
```

Utilisation des opérateurs sur les collections

- **L'opérateur CAST(MULTISET …)**
  - **CAST et MULTSET peuvent être utilsés en insertion**

```
INSERT INTO o_piloteSELECT pilote_t(    4, 'GAGARIN',adresse_t(1, 'rue Arago',6000, 'NICE'),
      '0493825084', '12-JUN-47', 12000.50,
      CAST (MULTISET(SELECT REF(v) FROM o_vol v WHERE v.pilote#=4)AS set_o_vol_t )
   )FROM dual;
```

Utilisation des opérateurs sur les collections

L'opérateur CARDINALITY

- L'opérateur **CARDINALITY** permet renvoyer la **cardinalité** d'une **collection de type Nested Table**
- **Syntaxe : CARDINALITY (nested_table)**

- **Exemple 1 :** Donner la cardinalité de la liste des vols attachés à chaque pilote

```
SELECT PL#, PLNOM, CARDINALITY(pListRefVols) FROM o_pilote;
```

```
        PL# PLNOM        CARDINALITY(PLISTREFVOLS)
---------- ------------ -------------------------
         1 ZEMBLA                               2
         2 ZORRO                                0
```

Not

anr
INVESTIR L'AVENIR
BY NC ND
DATUM ACADEMY

Utilisation des opérateurs sur les collections

L'opérateur MEMBER OF

- L'opérateur MEMBER OF permet de vérifier qu'une instance d'un type est membre d'une **collection de type Nested Table**
- **Syntaxe : Expression [NOT] MEMBER [OF] nested_table**
- **Exemple 1 :** Vérifier si une instance de cours est dans la collection listeCours

SELECT cursus_id, titre, listeCours **FROM** O_CURSUS WHERE COURS_T(103, 'UML', 'Booch', 'Jackson') **MEMBER OF** listeCours;

CURSUS_ID : 3
TITRE : M1 INFO
LISTECOURS(COURS_ID, LIBELLE, RESPONSABLE, INTERVENANT) :
LISTE_DE_COURS_T(COURS_T(103, 'UML', 'Booch', 'Jackson'))

Utilisation des opérateurs sur les collections

L'opérateur IS [NOT] EMPTY

- L'opérateur EMPTY permet de vérifier qu'une **collection de type Nested Table est vide ou non**
- **Syntaxe : nested_table** IS [NOT] EMPTY
- **Exemple 1 :** Vérifie si une collection nested table est ou non vide

SELECT cursus_id, titre, listeCours **FROM** O_CURSUS **WHERE** listeCours **IS EMPTY**;

CURSUS_ID : 4
TITRE : M2 ISI
LISTECOURS(COURS_ID, LIBELLE, RESPONSABLE, INTERVENANT) : LISTE_DE_COURS_T()

Utilisation des opérateurs sur les collections

- **L'opérateur** IS [NOT] A SET
  - L'opérateur IS A SET permet de vérifier qu'une **collection de type Nested Table N'A PAS DE DOUBLON**
  - **Syntaxe : nested_table** IS [NOT] A SET
  - **Exemple 1 :** Vérifie si une collection nested table n'a pas de doublons

SELECT cursus_id, titre, listeCours FROM O_CURSUS WHERE listeCours IS A SET;

CURSUS_ID TITRE LISTECOURS(COURS_ID, LIBELLE, RESPONSABLE, INTERVENANT)
1 M2 MBDS
LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date'), COURS_T(102, 'XML', 'Jacobson', 'Smirnoff'))

2 M1 MIAGE
LISTE_DE_COURS_T(COURS_T(101, 'Java', 'Mc Nelly', 'Buffa'))

3 M1 INFO
LISTE_DE_COURS_T(COURS_T(103, 'UML', 'Booch', 'Jackson'))

Utilisation des opérateurs sur les collections

L'opérateur COLLECT

- **L'opérateur COLLECT** prend en paramètre une colonne de n'importe quel type et construit une collection de type Nested Table. L'application de la fonction CAST au résultat permet d'opérer la conversion. **S'il y a des doublons ils sont conservés**. Pour éliminer les doublons, précéder la colonne de DISTINCT
- **Syntaxe :** COLLECT (column)
- **Exemple 1 :** Construction d'une collection avec Collect

```
CREATE TYPE LISTNOMCOURS_T AS TABLE OF varchar2(50)
/

SELECT  CAST (COLLECT(titre) AS LISTNOMCOURS_T )
FROM O_CURSUS;

CAST(COLLECT(TITRE)ASLISTNOMCOURS_T)
LISTNOMCOURS_T('M2 MBDS', 'M1 MIAGE', 'M1 INFO', 'M2 ISI')
```

Utilisation des opérateurs sur les collections

- **L'opérateur MULTISET {UNION | EXCEPT | INTERSECT}**
  - L'opérateur MULTISET {UNION | INTERSECT | EXCEPT} permet d'effectuer l'union, l'intersection ou la différence entre deux collections de type NESTED TABLE
  - **Syntaxe**

```
nestedTable1 MULTISET {UNION
                      | EXCEPT
                      | INTERSECT
                      } [{ALL | DISTINCT}] nestedTable2
```

  - **Exemple 1 : différence entre la Nested Table listeCours et LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date'))**

```
SELECT listeCours MULTISET EXCEPT LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date')) "resultat« FROM O_CURSUS;
```

resultat(COURS_ID, LIBELLE, RESPONSABLE, INTERVENANT)
LISTE_DE_COURS_T(COURS_T(102, 'XML', 'Jacobson', 'Smirnoff'))
LISTE_DE_COURS_T(COURS_T(101, 'Java', 'Mc Nelly', 'Buffa'))
LISTE_DE_COURS_T(COURS_T(103, 'UML', 'Booch', 'Jackson'))
LISTE_DE_COURS_T()

Module M3.3, section 2, partie 3 : Les liens d'association

Utilisation des opérateurs sur les collections

L'opérateur POWERMULTISET

- L'opérateur POWERMULTISET prend en paramètre une collection de type Nested Table afin de construire une collection de collection contenant des sous-collections construites avec chaque élément ou une combinaison d'éléments de la collection en paramètre. Supposons que nous avons une Collection COLS avec 2 éléments 'A' et 'B'. Powermultiset(COLS) renvoie la collection de collection suivantes : **CollectionDeCollections(**collection(A), collection(B) et Collection(A, B)**)**.
  PowerMultiSet permet de construire tous les ensembles unique possible combinant les éléments d'un ensemble.
- **Syntaxe :** POWERMULTISET (expression)
- **Exemple 1 :** Construction d'une collection avec Collect

```
-- Création d'un collection de collection de cours
CREATE or replace TYPE ListeDeListeDeCOURS_T AS TABLE OF LISTE_DE_COURS_T
/
SELECT CAST (POWERMULTISET(listeCours) AS ListeDeListeDeCOURS_T )
FROM O_CURSUS where cursus_id<3;
```

CAST(POWERMULTISET(LISTECOURS)ASLISTEDELISTEDECOURS_T)(COURS_ID, LIBELLE, RESPON

```
LISTEDELISTEDECOURS_T(LISTE_DE_COURS_T(COURS_T(100, 'SGBDR', 'Codd', 'Date')), L
ISTE_DE_COURS_T(COURS_T(102, 'XML', 'Jacobson', 'Smirnoff')), LISTE_DE_COURS_T(C
OURS_T(100, 'SGBDR', 'Codd', 'Date'), COURS_T(102, 'XML', 'Jacobson', 'Smirnoff'))
LISTEDELISTEDECOURS_T(LISTE_DE_COURS_T(COURS_T(101, 'Java', 'Mc Nelly', 'Buffa'))
```

Note : La colonne nested table ListeCours ne doit contenir de liste vide.

Utilisation des opérateurs sur les collections

- **L'opérateur** POWERMULTISET_BY_CARDINALITY
  - L'opérateur **POWERMULTISET_BY_CARDINALITY** prend en paramètre une collection de type Nested table ainsi qu'une cardinalité afin de construire une collection de collection avec des Nested Table ayant **la dite cardinalité**
  - **Syntaxe :** POWERMULTISET_BY_CARDINALITY(expression, cardinality)

Utilisation des opérateurs sur les collections

- **L'opérateur** POWERMULTISET_BY_CARDINALITY
  - **Exemple 1 :** Construction d'une collection avec Collect

CREATE or replace TYPE **ListeDeListeDeCOURS1_T** AS TABLE OF LISTE_DE_COURS1_T
/
SELECT **CAST (**POWERMULTISET_BY_CARDINALITY **(listeCours, 1)** AS ListeDeListeDeCOURS1_T)
FROM O_CURSUS1 where cursus_id<3;

```
CAST(POWERMULTISET_BY_CARDINALITY(LISTECOURS,1)ASLISTEDELISTEDECOURS1_T)(COURS_ID, LIBELLE, RESPONSABLE, INTERVENANT)
--------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------
LISTEDELISTEDECOURS1_T(LISTE_DE_COURS1_T(COURS1_T(100, 'SGBDR', 'Codd', 'Date')), LISTE_DE_COURS1_T(COURS1_T(102, 'XML', 'Jacobson'
, 'Smirnoff')))
LISTEDELISTEDECOURS1_T(LISTE_DE_COURS1_T(COURS1_T(101, 'Java', 'Mc Nelly', 'Buffa')))
```

Utilisation des opérateurs sur les collections

- **L'opérateur** SET
  - L'opérateur SET renvoie une collection après avoir supprimé les doublons
  - **Syntaxe :** SET (column)
  - **Exemple 1 :**

    SELECT SET(pListRefVols) FROM o_pilote;

    SET(pListRefVols)
    SET_o_vol_T(00002202081E3F202E19524F849FE204B98CA2E631A01125D890944C3C9F4F69C4FDA047FB**)**

BY NC ND

Autres Considérations

- La gestion **des liens d'associations** via les références en s'appuyant sur REF et les Nested Tables "**devrait" favoriser les performances** dans un contexte navigationnel
- Il s'agit **d'une sorte d'implémentation de la jointure** si on connecte deux tables par l'égalité
- La **programmation d'applications objets est mieux adaptée** dans un langage L3G plutôt que SQL seul

Module M3.3, section 2, partie 3 : QUIZ

- **Question 1 : Soit la requête suivante : SELECT CAST (MULTISET(SELECT REF(v) FROM o_vol v WHERE v.refPilote.pl# IN (1,2)) AS listRefVols_t) FROM dual; que fait elle**

  A: Cette requête construit la liste des vols des pilotes Nr 1 ét 2

  B: Cette requête construit la liste pointeurs vers les vols des pilotes Nr 1 ét 2

  B: MULTISET prend en paramètre une colonne ou une expression

  C: MULTISET prend en paramètre une requête SQL

- **Question 2 : Soit la requête suivante : SELECT CAST(COLLECT(REF(v)) AS listRefVols_t) FROM o_vol v WHERE v.refPilote.pl# IN (1,2); que fait elle**

  A: Cette requête construit la liste des vols des pilotes Nr 1 ét 2

  B: Cette requête construit la liste pointeurs vers les vols des pilotes Nr 1 ét 2

  B: COLLECT prend en paramètre une colonne ou une expression

  C: COLLECT prend en paramètre une requête SQL

Module M3.3, section 2, partie 3 : QUIZ

➢ **Question 3 : Soit la requête suivante : que fait elle**

SELECT CURSOR(SELECT p6.plnom, v.column_value.vol#, v.column_value.villeDepart
FROM TABLE (p6.pListRefVols) v**)** FROM o_pilote p6;

A: Cette requête renvoie les informations sur chaque pilote et ses vols
B: Cette requête construit la liste des vols des pilotes Nr 1 ét 2
B: CURSOR s'applique sur les champs NESTED TABLE
C: Sans CURSOR il est impossible d'accéder aux éléments d'une collection

➢ **Question 4 : Que fait la requête SELECT suivante :**

CREATE or replace TYPE ListeDeListeDeVOLS_T AS TABLE OF LISTREFVOLS_T;
SELECT p6.plnom, CAST (POWERMULTISET(p6.pListRefVols) AS ListeDeListeDeVOLS_T) FROM o_pilote p6;

A: Cette requête permet de construire une collection de collection avec les références vers les vols de chaque pilote
B: Il y a une collection de collection pour chaque Pilote
C: Les éléments de chaque collection de collection sont des collections de type OF LISTREFVOLS_T avec 1, 2, …N éléments de la collection des pointeurs vers les Vols d'un pilote
D: Impossible, cette requête ne marche pas. Conversion impossible

Module M3.3: Bilan et Exercices

- Bilan

- Exercices

Module M3.4 : Lien d'héritage et le Mapping objet des tables relationnelles SQL2 via les vues

Module M3.4 : Les liens d'héritage et le Mapping objet des tables relationnelles SQL2 via les vues

Plan

- Module M3.4, section 1 : Les liens d'héritage
- Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

Module M3.4, section 1 : Les liens d'héritage

Module M3.4, section 1 : Les liens d'héritage

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Plan

- Généralités
- Les sous-types
- Surcharge des méthodes et des attributs
- Types non instanciables
- Les types et les méthodes finales
- Conversion de types
- Les nouveaux opérateurs liés à l'héritage

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.4, section 1 : Les liens d'héritage

Généralités

- Soit **deux types A et B, on dit que A hérite de B, ssi l'ensemble des caractéristiques statiques et dynamiques de B sont aussi implicitement celles de A**. On dit aussi que A est un fils de B ou que B est père de A.

- L'Héritage est le mécanisme **de transmission des propriétés d'un type vers les types dérivés**.

- Oracle supporte depuis la version 9**, l'héritage simple**

Module M3.4, section 1 : Les liens d'héritage

Les sous-types

- **Syntaxe**

CREATE OR REPLACE TYPE nomType UNDER superType (
[inheritanceClauses] {sousProgammeClause | MapOrderClause }

)
[{[NOT] FINAL] | [[NOT] INSTANCIABLE]}]

Mot clé	Description
UNDER	Pour indiquer le type père
inheritanceClauses	Propriétés d'héritage au niveau des fonctions et des procédures
FINAL	Type ne pouvant plus être étendu (pas de sous-types)
INSTANCIABLE	Type pouvant avoir des instances

Module M3.4, section 1 : Les liens d'héritage

Les sous-types

- **Exemple de création d'un sous-type**

```
CREATE OR REPLACE TYPE Personne_t AS OBJECT(
  numInsee    varchar2(30),
  nom         varchar2(20),
  Prenom      varchar2(20),
  DateNaiss    date,
  MEMBER  PROCEDURE  afficher
) NOT FINAL;
/

CREATE OR REPLACE TYPE Employee_t UNDER Personne_t (
  salaire     number(7,2)
);
/
```

- **NOTE :** Si l'option NOT FINAL n'est pas précisé, le type est FINAL. On ne peut pas créer des sous-types à partir de lui

Surcharge des méthodes

- La surcharge permet de **définir plusieurs méthodes ayant un même nom** mais offrant des comportements différents

- **Exemple**

```
CREATE OR REPLACE TYPE Position as OBJECT(
  x      number(5,2),
  y      number(5,2)
);
/
CREATE OR REPLACE TYPE Employee2_t UNDER Personne_t (
  salaire                number(7,2),
  MEMBER PROCEDURE setSalaire(sal IN number),
  MEMBER PROCEDURE setSalaire(sal IN varchar2),
  MEMBER PROCEDURE afficher(coord position)
);
/
```

Module M3.4, section 1 : Les liens d'héritage

Redéfinition des méthodes

- **Le mot clé [NOT] OVERRIDING permet de contrôler la redéfinition d'une méthode d'un type au niveau d'un de ses sous-types**
- **Exemple**

```
CREATE OR REPLACE TYPE Employee3_t UNDER Personne_t (
  salaire                number(7,2),
  OVERRIDING MEMBER PROCEDURE afficher
);
/
```

- **Limites de la redéfinition**
  - On **ne peut redéfinir** une **méthode finale**
  - Les méthodes **MAP et ORDER ne peuvent être redéfini**
  - Une **méthode statique d'un sous-type ne peut redéfinir une méthode du type père**
  - Une méthode **membre d'un sous-type ne peut redéfinir** une méthode **statique du type père**
  - Le prototype de la méthode dans le sous-type doit être équivalent à celui de la méthode au niveau du type père

Types non instanciables

- Le mot clé **[NOT] INSTANTIABLE permet de définir l'équivalent d'une <u>méthode abstraite</u> ou d'une classe Abstraite en Java.** Une méthode de ce genre n'a pas de corps elle doit être redéfinie dans les types fils
- **Si un type a une méthode NOT INSTANTIABLE, il doit être NOT INSTANTIABLE et NOT FINAL**. Un type peut aussi être défini NOT INSTANCIABLE même s'il n'a pas de méthode NOT INSTANCIABLE
- **Exemple de définition d'un type et d'une méthode non instanciable**

```
CREATE OR REPLACE TYPE FIGURE_T  AS OBJECT(
        x        number(4),
        y        number(4),
        NOT INSTANTIABLE MEMBER FUNCTION surface RETURN  NUMBER,
         MEMBER PROCEDURE deplacer(x1 IN NUMBER,        y1 IN NUMBER)
        ) NOT INSTANTIABLE NOT FINAL;
```

Types non instanciables

- **Exemple de définition d'un type et d'une méthode à partir d'un type non instanciable**

```
CREATE OR REPLACE TYPE CERCLE_T UNDER FIGURE_T (
        rayon   NUMBER(4),
        OVERRIDING MEMBER  FUNCTION surface RETURN NUMBER
);

CREATE OR REPLACE TYPE BODY CERCLE_T  IS

        OVERRIDING  MEMBER  FUNCTION surface RETURN NUMBER IS
        BEGIN
                RETURN rayon*rayon*Pi;
        END;
END;
/
```

Conversion de types

- A l'image des systèmes à objets, une instance d'un sous-type **peut apparaître partout ou une instance d'un type père apparaît**.
- **Exemple : pose des contraintes**

```
CREATE TABLE o_personne of personne_t(
    constraint pk_o_personne primary key(numInsee),
    Prenom Constraint nl_o_personne_prenom not null,
    datenaiss Constraint nl_o_personne_datenaiss not null
);

INSERT INTO o_personne
VALUES(personne_t('122222', 'Dong', 'Jing Tong', to_date('10/10/1982', 'DD/MM/YYYY')));

INSERT INTO o_personne
VALUES(employee_t('222222', 'Dong', 'Fei Fei', to_date('10/10/1992', 'DD/MM/YYYY'),10000.0));
Select * from o_personne;
?
```

Conversion de types

Limitation des conversions implicites

- Lors de la création il est possible d'interdire toute substitution

```
CREATE TABLE o_personne2 OF personne_t
NOT SUBSTITUTABLE AT ALL LEVELS;
```

- Il est possible de limiter la substitution sur certains TYPE

```
CREATE TABLE o_personne3 OF personne_t
IS OF ( EMPLOYEE_T );
```

Module M3.4, section 1 : Les liens d'héritage

➢ Autre cas : Un employé est forcement une personne

- **Exemple**

```
Set serveroutput on
Set serveroutput on
DECLARE
   pers1 Personne_t := personne_t('122222',  'Dong', 'Jing Tong', to_date('10/10/1982', 'DD/MM/YYYY'));
   pers2 Personne_t :=null;
   emp1 Employee_t:= employee_t('222222',  'Dong', 'Fei Fei', to_date('10/10/1992', 'DD/MM/YYYY'),10000.0);
   emp2 Employee_t:=null;
   emp3 Employee_t:=null;
BEGIN
   pers2 := emp1;                                    -- COMMENT1: Cela marche car tout employé est forcement une personne
   dbms_output.put_line('nom='|| pers2.nom);         -- COMMENT2: Cela marche car "nom" est commun à pers2 et emp1
   --dbms_output.put_line('salaire='||pers2.salaire); -- COMMENT3: Impossible car seule la partie commune est visible avec "pers2" sans casting
   --emp2:=pers2                                     -- COMMENT4: Impossible sans casting. Toute personne n'est employé
   emp2:=TREAT (pers2 as employee_t);                -- COMMENT5: Cela marche grâce au Casting car on n'avait bien affecté emp1 à pers1
   dbms_output.put_line('salaire='||emp2.salaire);   -- COMMENT6: Cela marche, le salaire n'a pas été perdu avec COMMENT1. Oracle gère des pointeurs
   --emp3:=pers1;                                    -- COMMENT7: Impossible sans casting. Toute personne n'est 1 employé
END;
/
```

Les nouveaux opérateurs liés à l'héritage

- Certains **nouveaux opérateurs** ou fonctions sont introduits avec l'héritage : **IS OF *type*, SYS_TYPEID, TREAT**

- ***IS OF type*** permet de déterminer si un objet est d'un type donné

SELECT value(op) **FROM** o_personne op **WHERE** value(op) **is of (employee_t);**

VALUE(OP)(NUMINSEE, NOM, PRENOM, DATENAISS)

EMPLOYEE_T('222222', 'Dong', 'Fei Fei', '10/10/90', 10000)

Les nouveaux opérateurs liés à l'héritage

- **SYS_TYPEID : permet de renvoyer le numéro d du type d'une instance**

```
SELECT  op.nom, SYS_TYPEID(VALUE(op))  TYPE_ID FROM o_personne op
```

NOM	TYPE_ID
Dong	01
Dong	02

BY NC ND

Les nouveaux opérateurs liés à l'héritage

- **TREAT Cas 1:** permet de convertir le type d'une expression en un sous type donné. Il est par exemple possible de convertir les instances d'un type de base en des instances de ses sous-type

```
SELECT TREAT(value(op) as employee_t)
FROM o_personne op;
```

TREAT(VALUE(OP)ASEMPLOYEE_T)(NUMINSEE, NOM, PRENOM, DATENAISS, SALAIRE)
EMPLOYEE_T('222222', 'Dong', 'Fei Fei', '10/10/90', 10000)

Les nouveaux opérateurs liés à l'héritage

- **TREAT Cas 2: permet aussi de poser des contraintes d'intégrité sur les champs des types fils en cas de gestion de la hiérarchie de types dans une seule table**

  - **Exemple : Le salaire d'un employé doit être compris entre 10000 et 70000.**

```
ALTER TABLE o_personne
add constraint chk_o_personne_employe_salaire
CHECK(TREAT(object_value AS employee_t).salaire between 10000 and 70000);

-- Object_value permet de designer une instance d'employé. C'est une colonne virtuelle d'une table objet.

-- Vérification
INSERT INTO o_personne
VALUES(employee_t('333333', 'RACKHAM', 'Lerouge', to_date('10/10/1992', 'DD/MM/YYYY'),5000.0));

*
ERREUR Ó la ligne 1 :
ORA-02290: violation de contraintes (MOPOLO.CHK_O_PERSONNE_EMPLOYE_SALAIRE)
```

Les nouveaux opérateurs liés à l'héritage

- **TREAT Cas 3: permet aussi de poser des indexes sur les champs des types fils en cas de gestion de la hiérarchie de type dans une seule table**

  - **Exemple : Créer un index non unique sur la colonne salaire**

```
Create index idx_o_personne_emp_salaire on o_personne op
(TREAT(VALUE(op) AS employee_t).salaire);
```

Ou bien

```
Create index idx_o_personne_emp_salaire on o_personne op
(TREAT(object_value AS employee_t).salaire);
```

Les nouveaux opérateurs liés à l'héritage

- **TREAT Cas 3: permet aussi de poser des indexes sur les champs des types fils en cas de gestion de la hiérarchie de type dans une seule table**

```
-- verification
Set autotrace on
SELECT * FROM o_personne op
where TREAT(value(op) as employee_t).salaire
<11000;
```

cr

```
NUMINSEE                NOM             PRENOM          DATENAIS
------------------------ --------------- --------------- --------
222222                  Dong            Fei Fei         10/10/92


Plan d'exÚcution
----------------------------------------------------------
Plan hash value: 3767178368

-----------------------------------------------------------------------------------------
| Id  | Operation                | Name                    | Rows  | Bytes | Cost (%CPU)| Time
|
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                         |     1 |    63 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| O_PERSONNE              |     1 |    63 |     2   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN       | IDX_O_PERSONNE_EMP_SALAIRE |     1 |       |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------
```

Module M3.4, section 1 : QUIZ

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

- **Question 1 : Cochez ce qui caractérise l'héritage sous SQL3 Oracle**

  A: L'héritage est le principe qui permet de partager une instance d'un type entre plusieurs sous-type

  B: Oracle supporte l'héritage multiple

  C: Oracle supporte l'héritage simple uniquement

  D: L'héritage est le principe qui permet à un type d'hériter des propriétés d'un autre type

- **Question 2 : Cochez ce qui caractérise l'héritage sous SQL3 Oracle**

  A: Un type fils peut surcharger une méthode d'un type père

  B: Pour hériter d'un type il doit être NOT FINAL

  C: Un type défini avec le mot clé NOT INSTANCIABLE NOT FINAL devrait avoir au moins une méthode NOT INSTACIABLE

  D: Il n'est pas possible en SQL3 de définir l'équivalent d'une classe abstraite C++ ou Java

cr

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.4, section 1 : QUIZ

- **Soit le type suivant**

```
Create or replace type vehicule_t as object (
lieuParkage varchar2(30),
nomVehicule varchar2(100),
NOT INSTANTIABLE  member procedure demarrer,
NOT INSTANTIABLE  member procedure stoper
) not instantiable not final;
/
```

- **Question 3: cochez ce qui fonctionne**

A: declare v Vehicule_t; begin null; end;

B: declare v Vehicule_t:=vehicule_t('Garage', 'Voiture'); begin null; end;

C: declare v Vehicule_t:=vehicule_t(); begin null; end;

D:declare v Vehicule_t:=null; begin null; end;

Module M3.4, section 1 : QUIZ

➢**Question 4 : Cochez ce qui est juste par rapport à cette commande**

```
Create or replace type voiture_t UNDER vehicule_t(
    nombreDePorte number(2),
    OVERRIDING member procedure demarrer,
    OVERRIDING member procedure stoper,
    member procedure stoper(param in number)
    );
```

A: La commande permet de créer un type appelé voiture_t qui hérite du abstrait vehicule_t

B: OVERRIDING permet surcharger les méthodes «démarrer» et «stopper» du type père «vehicule_t»

C: OVERRIDING permet la réécriture des méthodes «démarrer» et «stopper» du type père «vehicule_t»

D: La méthode « member procedure stoper(param in number) » n'est pas autorisée ici

Module M3.4, section 1 : QUIZ

➢**Question 5 : Cochez ce qui est juste par rapport à cette commande**

```
Create or replace type bateauARame_t UNDER vehicule_t(
nombreDeRames number(2),
member procedure stoper(param in number)
) not instantiable not final;
```

A: La méthode « member procedure stoper(param in number)» **surcharge** la méthode « stoper » de vehicule_t

B: La méthode « member procedure stoper(param in number)» **réécrit** la méthode « stoper » de vehicule_t

C: Les méthodes « stoper » et « demarrer » sont pour le type bateauARame_t «NOT INSTANTIABLE»

D: La méthode member procedure stoper(param in number) n'est pas autorisée ici

E: Il est obligatoire de définir le type bateauARame_t comme étant « not instantiable not final »

Module M3.4, section 1 : QUIZ

- **Question 6: cochez ce que fait l'opérateur TREAT**
  A: L'opérateur TREAT a le même rôle que l'opérateur IS OF TYPE
  B: Permet de rechercher les instances d'un sous type en cas de gestion de la hiérarchie de types dans une seule table créée au niveau du type racine
  C: Permet aussi de poser des contraintes d'intégrité sur les champs des types fils en cas de gestion de la hiérarchie de types dans une seule table créée au niveau du type racine
  D: Permet aussi de poser des indexes sur les champs des types fils en cas de gestion de la hiérarchie de type dans une seule table créée au niveau du type racine

- **Question 7: cochez ce fait l'opérateur** IS OF TYPE
  A: L'opérateur TREAT a le même rôle que l'opérateur IS OF TYPE
  B: Permet de rechercher les instances d'un sous type en cas de gestion de la hiérarchie de types dans une seule table créée au niveau du type racine
  C: Permet aussi de poser des contraintes d'intégrité sur les champs des types fils en cas de gestion de la hiérarchie de types dans une seule table créée au niveau du type racine
  D: Permet de déterminer si un objet est d'un type donné en cas de gestion de la hiérarchie de types dans une seule table créée au niveau du type racine

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

Plan

- Généralités
- Les Types de vues
- Création et manipulation de vues relationnelles
- Création et manipulation de vues objets
- L'opérateur MAKE_REF
- Le TRIGGER INSTEAD OFF

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

Généralités

- **Les vues sont des objets définis à partir des tables existantes et/ou des ADTs. Elles permettent :**
  - De **masquer la complexité** d ’une requête,
  - D **’augmenter la sécurité** des tables de base
  - De **masquer des informations** à certains utilisateurs

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ Généralités

- **On distingue 2 types de vues, définies sur les tables (objets, relationnelles) ou autres vues (objets, relationnelles) :**

  - **vue relationnelle** : une vue relationnelle est une vue construite sur les colonnes d'une table relationnelle, objet ou sur d'autres vues

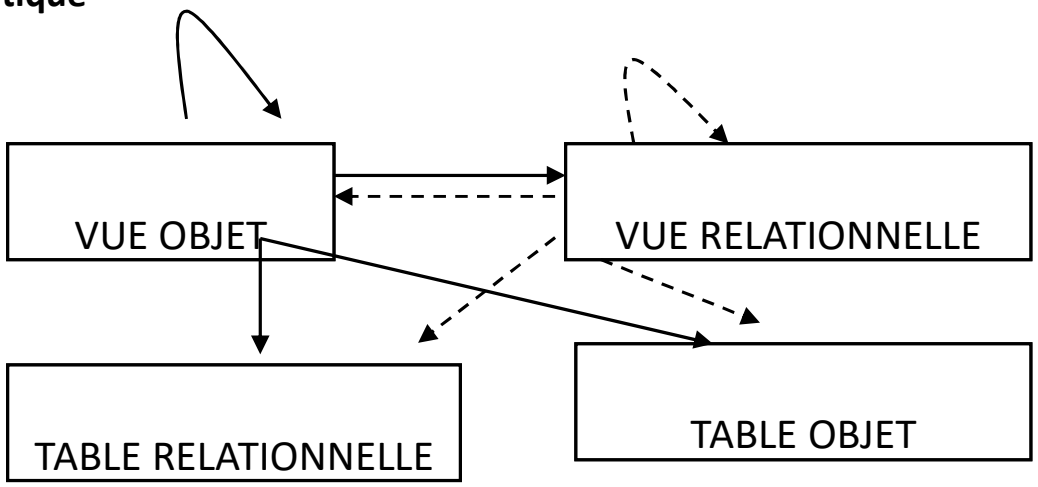
  - **vue objet :** une vue objet est une "table objet virtuelle" dont les instances sont identifiées par des OID. Les colonnes d'une telle vue sont définies sur un type de l'utilisateur

Les types de vues

- **Avantages <u>des vues objets</u> :**
  - Accès **aux tables relationnelles SQL2 par les applications Orientées Objets** (O.O)
  - Coexistence **entre applications relationnelles sql2 et applications relationnelles SQL3**.

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

- Les types de vues
  - **réprésentation schématique**



Une VUE XXX peut être construite à partir de ...

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢Création et manipulation de vues relationnelles

- Une **vue relationnelle** SQL2 peut être construite sur : **une vue relationnelle,** une vue objet, **une table relationnelle** ou une table objet. A la différence d'une vue Objet, elle ne possède pas d'OID

- **Exemple de création d'une vue relationnelle SQL2 à partir d'une table relationnelle**

```
CREATE TABLE rpilote7 (
        PL#             NUMBER(4),
        nom             VARCHAR2(12),
        adresse         adresse_t,
        Tel             VARCHAR2(12),
        dnaiss          DATE,
        salaire         NUMBER(7,2),
        pListRefVols2   listRefVols_t)
        NESTED TABLE pListRefVols2 STORE AS store_pListRefVols2;
```

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢Création et manipulation de vues relationnelles

- **Exemple de création d'une vue relationnelle à partir d'une TABLE relationnelle**

```
CREATE OR REPLACE VIEW pilote_v (plnum, plnom, adresse, tel)  AS
SELECT  Pl#, nom, adresse, Tel
FROM rpilote7 ;
```

- **Exemple de création d'une vue relationnelle à partir d'une TABLE OBJET**

```
CREATE OR REPLACE VIEW v_o_pilote_nice AS
SELECT * FROM o_pilote p6
WHERE p6.adresse.ville ='NICE';
```

NOTE : **o_pilote** est une table objet mais pas la **vue v_o_pilote_nice**

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢Création et manipulation de vues OBJETS

➢Syntaxe de création d'une vue object

CREATE OR REPLACE [FORCE] VIEW nomVue
OF nomTypeObjet
WITH OBJECT IDENTIFIER
[DEFAULT | (attribut1, attribut2, …)]
AS sousRequête;

Mots et paramètres	Description
OF nomTypeObjet	création d'une vue objet sur le type nomTypeObjet
WITH OBJECT IDENTIFIER	permet de spécifier la ou les colonnes utilisées comme clé pour identifier de façon unique les lignes de la vue objet. Cette clause est obligatoire s'il s'agit d'une vue sur une table objet ou une vue objet.

Création et manipulation de vues OBJETS

Création d'une vue objet à partir d'une table OBJET

- **Exemple**

```
-- Création de la vue sur pilotes à partir d'une table Objet
CREATE OR REPLACE  VIEW o_view_pilote2
    OF r_pilote_t
WITH OBJECT IDENTIFIER DEFAULT AS
SELECT op.pl#, op.nom, op.tel, op.adresse, op.dnaiss
FROM o_pilote op;
```

NOTE : o_pilote est supposée être une table Objet

BY NC ND

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ Création et manipulation de vues OBJETS

- **Création de vues OBJETS sur des tables RELATIONNELLES**
  - L'objectif est de pouvoir **faire cohabiter les applications relationnelles existantes et les nouvelles applications Objets**
  - Pour chaque table relationnelle **il est nécessaire de créer un type de données**. Ces types peuvent contenir des liens d'association (REF ou collection de type Nested tables). Des attributs d'une table peuvent être regroupés sous un type particulier. Une hiérarchie de types peut aussi être construite
  - Des **vues objets seront ensuite créés sur ces types**. Une hiérarchie des vues peut être créés pour simuler la hiérarchie d'héritage
  - Nous allons dans l'exemple qui suit introduire des OID sur les tables relationnelles **RPILOTE8 et RVOL8**.

-- Etape 1: Création table la relationnelle RPILOTE8

```
CREATE TABLE rpilote8(
        pl#             NUMBER(4) constraint pk_rpilote8_pl# primary key ,
        nom             VARCHAR2(20),
        tel             VARCHAR2(20),
        NUMERO          NUMBER(4),
        RUE             VARCHAR2(20),
        CODE_POSTAL     NUMBER(5),
        VILLE           VARCHAR2(20),
        dnaiss          DATE
);
```

Création et manipulation de vues OBJETS

Création de vues objets sur des tables relationnelles

-- Insertion de ligne dans la table RPILOTE8

Insert into **rpilote8** values (1, ‘Gagarin’, ‘00723344667’, 1, ‘Rue du cosmos’, 1000, ‘Klouchino’, to_date(’09-03-1934’, ‘DD-MM-YYYY’));

Insert into **rpilote8** values (2, ‘Zigmund’, ‘004923344667’, 1, ‘Lumumba Strasse’, 8262, ‘Morgenröthe’, to_date(’13-02-1937’, ‘DD-MM-YYYY’));

Insert into **rpilote8** values (3, ‘Armstrong’, ‘00111111111’, 1, ‘Avenue de l’’espace’, 45895, ‘Wapakoneta’,
to_date(’ 05-08-1930’, ‘DD-MM-YYYY’));

Insert into **rpilote8** values (4, ‘Baudry’, ‘0033612122322’, 1, ‘Av. Champs Elysées’, 75000, ‘Paris’, to_date(’ 06-03-1946’, ‘DD-MM-YYYY’));

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

Création et manipulation de vues OBJETS

Création de vues objets sur des tables relationnelles.

-- Etape 1: Création table la relationnelle RVOL8

```
CREATE TABLE RVol8 (
Vol#        NUMBER(4) constraint pk_rvol8_vol#
            primary key,
Pilote#     NUMBER(4) constraint
fk_rvol8_pilote# REFERENCES rpilote8(pl#),
VD           VARCHAR2(20),
VA          VARCHAR2(20),
dateVol     date
)
/
```

Création et manipulation de vues OBJETS

- **Création de vues objets sur des tables relationnelles.**

-- Etape 1: Création table la relationnelle RVOL8

-- Insertion de lignes dans la table relationnelle RVOL8

Insert into **rvol8** values (100, 1, ‘Paris’, ‘Nice’, to_date(‘10-10-1965’, ‘DD-MM-YYYY’));

Insert into **rvol8** values (110, 1, ‘Nice’, ‘Bordeaux’, to_date(‘10-10-1966’, ‘DD-MM-YYYY’));

Insert into **rvol8** values (120, 2, ‘Moscou’, ‘Paris’, to_date(‘10-10-1968’, ‘DD-MM-YYYY’));

Insert into **rvol8** values (130, 3, ‘Paris’, ‘Newyork’, to_date(‘10-12-1970’, ‘DD-MM-YYYY’));

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ Création et manipulation de vues OBJETS

▪ Création de vues objets sur des tables relationnelles

```
-- Etape 2 : Création des types
-- Suppression des types avant création s'ils existent déjà
    DROP TYPE r_pilote8_t  force;
    DROP TYPE r_vol8_t  force;
    DROP TYPE listRefVol8_t force;

-- Créate du type listRefvol8_t : collection de vols
    CREATE OR REPLACE TYPE r_vol8_t
    /
    CREATE OR REPLACE TYPE listRefVol8_t AS TABLE OF
    REF r_vol8_t
    /
```

➢ Création et manipulation de vues OBJETS

▪ Création de vues objets sur des tables relationnelles

```
-- Création du type r_pilote8_t
-- Les colonnes numero, rue, code_postal, ville sont
-- regroupées sous le type adresse_t. Le champs
-- listRefVols est ajouté pour gérer la liste des référénces
-- vers les vols d'un pilote.

CREATE TYPE r_pilote8_t as object(
pl#             NUMBER(4) ,
nom             VARCHAR2(20),
tel             VARCHAR2 (20),
adresse         adresse_t,
dnaiss          DATE,
listRefVols     listRefVol8_t)
/
```

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ Création et manipulation de vues OBJETS

- **Création de vues objets sur des tables relationnelles**

- Etape 2 : Création des types

-- Création du type r_vol8_t. La référence refPilote est ajouté pour stocker le lien
-- vers le pilote qui assure le vol.

```
CREATE OR REPLACE  TYPE  r_Vol8_t AS OBJECT(
    Vol#          NUMBER(4),
    Pilote#       NUMBER(4),
    VD            VARCHAR2(20),
    VA            VARCHAR2(20),
    dateVol       date,
    refPilote     REF r_pilote8_t
)
/
```

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ Création et manipulation de vues OBJETS

▪ Création de vues objets sur des tables relationnelles

- Etape 3 : Création de vues objets

-- Création des vues o_v_pilote8 et o_v_vol8
-- Ces deux vues ont des références circulaires. Trois étapes sont nécessaires pour les créer :

1. Créer la vue o_v_vol8 avec référence NULL. Pas de référence à la vue o_v_pilote8
2. Créer la vue o_v_pilote8 avec référence à o_v_vol8
3. Recréer la vue o_v_vol8 avec référence à o_v_pilote8

-- 1.Création de la vue o_v_vol8 avec la REF à null

```
CREATE OR REPLACE  VIEW o_v_vol8 OF r_vol8_t
WITH OBJECT IDENTIFIER(vol#) AS
SELECT rv.vol#, rv.pilote#, rv.vd, rv.va, rv.dateVol, null
FROM rvol8 rv;
```

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ Création et manipulation de vues OBJETS

▪ Création de vues objets sur des tables relationnelles

- **Etape 3 : Création de vues objets**

-- 2. Création de la vue o_v_pilote8 avec construction de la collection de REF vers les vols d'un pilote

```
CREATE OR REPLACE  VIEW o_v_pilote8 OF r_pilote8_t
WITH OBJECT IDENTIFIER(pl#) AS
SELECT rp.pl#, rp.nom, rp.tel, adresse_t(rp.numero, rp.rue, rp.code_postal, rp.ville), rp.dnaiss,
CAST(MULTISET(select ref(v) from o_v_vol8 v where v.pilote#=rp.pl#) AS listRefVol8_t)
FROM rPilote8 rp;
```

-- 3. Recréation de la vue o_v_vol8 avec la REF au pilote qui assure le VOL.

```
CREATE OR REPLACE  VIEW o_v_vol8 OF r_vol8_t
WITH OBJECT IDENTIFIER(vol#) AS
SELECT rv.vol#, rv.pilote#, rv.vd, rv.va, rv.dateVol, MAKE_REF(o_v_pilote8, rv.pilote#)
FROM rvol8 rv;
-- Note : La function MAKE_REF renvoie le Ref d'un objet (voir plus loin)
```

Création et manipulation de vues OBJETS

Remarque sur les vues objets

- l'OID permet de référencer les lignes accessibles à travers la vue
- un index unique doit être associé aux colonnes jouant le rôle d'identifiant pour les tables relationnelles

```
ALTER TABLE rpilote8
ADD CONSTRAINT pk_rpilote8 PRIMARY KEY(pl#);
```

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ Création et manipulation de vues OBJETS

- **Consultation et mise à jour à travers des vues objets**

-- Insertion d'un pilote à travers la vue objet
INSERT INTO o_v_pilote8 (pl#, nom, tel, adresse, dnaiss) **VALUES (5, 'Tintin', '0493825084',**
adresse_t(1, 'rue de la Gare',6000, 'Bruxelles'),
to_date('12-06-1947', 'DD-MM-YYYY'));

SELECT ovp.pl#, ref(ovp) FROM o_v_pilote8 ovp;

➢ Création et manipulation de vues OBJETS

➢ Consultation et mise à jour à travers des vues objets

```
       PL#
----------
REF(OVP)
--------------------------------------------------------------------------------
         1
00004A038A0046D41ECA8592C94F71B6F79A97591512F500000014260100010001002900000000000
090604002A00078401FE0000000A02C1020000000000000000000000000000000000000000

         2
00004A038A0046D41ECA8592C94F71B6F79A97591512F500000014260100010001002900000000000
090604002A00078401FE0000000A02C1030000000000000000000000000000000000000000

         3
00004A038A0046D41ECA8592C94F71B6F79A97591512F500000014260100010001002900000000000
090604002A00078401FE0000000A02C1040000000000000000000000000000000000000000

         4
00004A038A0046D41ECA8592C94F71B6F79A97591512F500000014260100010001002900000000000
090604002A00078401FE0000000A02C1050000000000000000000000000000000000000000

         5
00004A038A0046D41ECA8592C94F71B6F79A97591512F500000014260100010001002900000000000
090604002A00078401FE0000000A02C1060000000000000000000000000000000000000000
```

Création et manipulation de vues OBJETS

- **Consultation et mise à jour à travers des vues objets**

-- Insérer un vol à travers la vue objet o_v_vol8

```
Insert into o_v_vol8 (vol#, pilote#, vd, va, dateVol)
values (140, 5, 'Lyon', 'Nice',
to_date('10-12-2000', 'DD-MM-YYYY'));
```

Création et manipulation de vues OBJETS

-- Informations sur les vols et leurs pilotes via le REF

SELECT ovv.vol#, ovv.vd, ovv.dateVol, ovv.**refPilote**.nom
FROM **o_v_vol8 ovv** order by ovv.**refPilote**.nom;

VOL#	VD	DATEVOL	REFPILOTE.NOM
130	Paris	10/12/70	Armstrong
110	Nice	10/10/66	Gagarin
100	Paris	10/10/65	Gagarin
140	Lyon	10/12/00	Tintin
120	Moscou	10/10/68	Zigmund

BY NC ND

➢Création et manipulation de vues OBJETS

➢Consultation et mise à jour à travers des vues objets

```
-- Informations sur les vols du pilote 1 via la collection de REF
select lrv.column_value.vol#, lrv.column_value.vd, lrv.column_value.dateVol, lrv.column_value.refPilote.nom
FROM TABLE(select ovp.listRefVols
  FROM o_v_pilote8 ovp  where ovp.pl#=1) lrv;
```

```
COLUMN_VALUE.VOL# COLUMN_VALUE.VD      COLUMN_V COLUMN_VALUE.REFPILO
----------------- -------------------- -------- --------------------
              100 Paris                10/10/65 Gagarin
              110 Nice                 10/10/66 Gagarin
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.4, section 2 : Mapping objet des tables relationnelles SQL2 via les vues

➢ L'opérateur MAKE_REF

- L'opérateur **MAKE_REF** **rend une référence vers une ligne d'une vue objet si la ligne existe. Dans le cas contraire rend la prochaine référence libre**. Il est nécessaire de passer à MAKE_REF le nom de la vue et les valeurs des colonnes clés.

- **Syntaxe**

MAKE_REF (nomVueObjet, key[key, …])

- **Exemple 1: Renvoi la référence du pilote nr 8 s'il existe**

SELECT **MAKE_REF(o_v_pilote8 , 2)** **FROM** dual;

```
MAKE_REF(O_V_PILOTE8,2)
-------------------------------------------------------------------------------
00004A038A0046D41ECA8592C94F71B6F79A97591512F500000014260100010001002900000000000
090604002A00078401FE0000000A02C1030000000000000000000000000000000000000000
```

SELECT **ref(ovp)** **FROM** o_v_pilote8 ovp

WHERE ovp.pl#=2;

```
REF(OVP)
-------------------------------------------------------------------------------
00004A038A0046D41ECA8592C94F71B6F79A97591512F500000014260100010001002900000000000
090604002A00078401FE0000000A02C1030000000000000000000000000000000000000000
```

BY NC ND

L'opérateur MAKE_REF

- Si l'objet identifié par la clé existe **c'est la référence existante qui est renvoyée** sinon, une nouvelle référence est attribuée.

- **Exemple 2**

  SELECT **MAKE_REF(o_v_pilote8, 6)** FROM dual;

```
MAKE_REF(O_V_PILOTE8,6)
--------------------------------------------------------------------------------
00004A038A0046D41ECA8592C94F71B6F79A97591512F5000000142601000100010029000000000
090604002A00078401FE0000000A02C1070000000000000000000000000000000000000000
```

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR
anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Le trigger INSTEAD OF

- Le trigger INSTEAD OF **permet de mettre à jour les tables sous-jascentes à une vue Complexe**. Ce qui n'était pas possible jusqu'à la version 7 d'Oracle

- Le trigger INSTEAD OF **est toujours exécuté à la place de l'ordre SQL sous-jascent**(INSERT, UPDATE, DELETE).

- Dans **l'exemple ci-dessous, l'insertion dans la vue provoquera une insertion dans la table relationnelle** rpilote8 et dans la table objet o_pilote8

- **Création d'un table objet sur le type de la vue objet**

```
create table o_pilote8 of r_pilote8_t(constraint pk_o_pilote8_pl# primary key (pl#))
Nested table listRefVols store as storeListRefVol;
```

Le trigger INSTEAD OF

- **Exemple:**

```
CREATE OR REPLACE TRIGGER
tr_update_ville_pil_v INSTEAD  OF INSERT ON o_v_pilote8
DECLARE
   adr        adresse_t;
BEGIN
   insert into rpilote8 values (:new.pl#, :new.nom, :new.tel,     :new.adresse.numero, :new.adresse.rue, :new.adresse.code_postal,
   :new.adresse.ville, :new.dnaiss);
   insert into o_pilote8 values (:new.pl#, :new.nom, :new.tel, :new.adresse, :new.dnaiss , listRefVol8_t());
END;
/
-- Une ligne va être insérée dans rpilote8 et o_pilote8 avec ce qui suit
INSERT INTO o_v_pilote8 (pl#, nom, tel, adresse, dnaiss) VALUES ( 7, 'Milou', '0493825084',
      adresse_t(1, 'rue de la Gare',6000, 'Bruxelles'),
      to_date('12-06-1947', 'DD-MM-YYYY'));

Select count(*) from rpilote8 where pl#=7; -- résutat : 1
Select count(*) from o_pilote8 where pl#=7; -- résutat : 1
```

Module M3.4, section 2 : QUIZ

➢ **Question 1 : Cochez ce qui caractérise le mieux une vue objet**

A: Une vue objet permet de rendre objet, une table objet

B: Une vue objet permet de rendre objet, une table relationnelle

C: Une vue objet sert d'abord à masquer la complexité

D: Une vue objet sert d'abord à masquer des colonnes à certains utilisateurs

➢ **Question 2 : Que faut il faire pour rendre objet, une table relationnelle appelée tableR**

A: Il faut juste faire : CREATE VIEW v_tableR AS SELECT * FROM tableR;

B: Il faut juste migrer la table tableR vers l'objet (import/export)

C: Il faut créer un type (par exemple tableR_t) mappant la structure de tableR; puis créer une vue objet comme suit : CREATE VIEW v_o_tableR of tableR_t WITH OBJECT IDENTIFIER(primaryKeyDeTableR) AS as select * from tableR;

D: Il faut créer un type (par exemple tableR_t) mappant la structure de tableR; puis créer une vue objet comme suit : CREATE or replace VIEW v_o_tableR of tableR_t WITH OBJECT IDENTIFIER DEFAULT AS SELECT * FROM tableR;

Module M3.4, section 2 : QUIZ

- **Considérons les tables relationnelles suivantes**

```
create table etudiant (
idEtu number(4) primary key,
nom varchar(30)
);

insert into etudiant values(1, 'King');
insert into etudiant values(2, 'Adams');

create table notes (
idEtu number(4) references etudiant,
matiere varchar2(30), note number(4,2),
primary key (idEtu, matiere )
);

insert into notes values(1, 'Java', 11);
insert into notes values(1, 'C++', 12);
insert into notes values(2, 'Big data', 13);
```

Module M3.4, section 2 : QUIZ

➢ **Considérons les types suivants**

```
create or replace type etudiant_t as object (
idEtu number(4),
nom varchar(30)
);
/

create or replace type notes_t as object (
idEtu number(4),
matiere varchar2(30),
note number(4,2),
refEtu REF etudiant_t
);
/
```

Module M3.4, section 2 : QUIZ

- **Considérons les vues suivantes**

```
CREATE OR REPLACE VIEW v_o_etudiant OF etudiant_t WITH OBJECT IDENTIFIER (IdEtu) AS SELECT * FROM etudiant;

CREATE OR REPLACE VIEW v_o_notes of NOTES_T WITH OBJECT IDENTIFIER (idEtu, matiere) AS SELECT n1.idEtu, n1.matiere, n1.note,
make_ref(v_o_etudiant,n1.idEtu )  FROM notes n1;
```

- **Question 3 : au vu des tables relationnelles, des types et des vues créés précédemment : cochez ce que fait la requête suivante :** SELECT ref(oe) FROM v_o_etudiant;

A: Cette requête renvoie la référence vers chaque étudiant inséré dans la table relationnelle ETUDIANT. Les références sont stockées dans la vue v_o_etudiant

B: Cette requête renvoie la référence vers chaque étudiant inséré dans la table ETUDIANT. Les références sont générées dynamiquement

C: Cette requête renvoie une erreur. En effet, dans une table relationnelle les lignes ne sont pas identifiées par des REF , même si on s'appui sur une vue

D: A et B sont faut

Module M3.4, section 2 : QUIZ

- **Question 4 : au vu des tables relationnelles, des types et des vues créés précédemment : cochez ce que fait la requête suivante :** SELECT vn.refEtu.nom, vn.matiere, vn.notes FROM v_o_notes vn;

  A: Cette requête renvoie une erreur. En effet, dans une table relationnelle les lignes ne sont pas identifiées par des REF, même si on s'appui sur une vue

  B: Cette requête renvoie le nom et les notes de chaque étudiante par matière sans faire de jointure

  C: Cette requête renvoie le nom et les notes de chaque étudiante par matière avec une jointure entre les tables relationnelles etudiant et notes

  D: C est faut

- **Question 5 : au vu des tables relationnelles, des types et des vues créés précédemment : cochez ce que fait la requête suivante :** SELECT value(oe) FROM v_o_etudiant oe;

  A: Cette requête permet d'afficher les instances d'étudiants insérées dans la table relationnelle ETUDIANT

  B: Cette requête permet d'afficher les valeurs d'étudiants insérées dans la table relationnelle ETUDIANT

  C: Cette requête renvoie une erreur. En effet, il est impossible de manipuler des objets compexes via des vues

  D: A et B sont faut

Module M3.4, section 2 : QUIZ

➢ **Question 6 : Que fait cette requête**

SELECT MAKE_REF(V_O_ETUDIANT, ClePrimaireDeLaTableEtudiant) FROM dual;

Considérons les lignes insérées précédemment dans la table relationnelle ETUDIANT. Les étudiants 1 et 2 exiistent

A: Si ClePrimaireDeLaTableEtudiant vaut 10, la requête renvoie une référence même si aucun étudiant a ce numéro

B: Si ClePrimaireDeLaTableEtudiant vaut 1, la requête renvoie une référence vers l'étudiant Nr 1 de la table ETUDIANT

C: MAKE_REF est un générateur de REF

D: MAKE_REF est un générateur d'OID

E: MAKE_REF est un générateur de ROWID

Module M3.4: Bilan et Exercices

- Bilan

- Exercices

Module M3.5 : Gestion des objets volumineux

Module M3.5 : Gestion des objets volumineux

Plan

- Module M3.5, section 1 : Gestion des Objets Volumineux
- Module M3.5, section 2 : Gestion des Objets Volumineux
- Module M3.5, section 3 : Gestion des Objets Volumineux
- Module M3.5, section 4 : Gestion des Objets Volumineux

Module M3.5, section 1 : Gestion des Objets Volumineux

Module M3.5, section 1 : Gestion des Objets Volumineux

- **Plan**
  - **Généralités**
  - **Les LOBs internes**
  - *Les LOBs externes ou BFILE*
  - *Les LOBs Locators*
  - *Le Package DBMS_LOB partie 1*
  - *Le Package DBMS_LOB partie 2*

Module M3.5, section 1 : Gestion des Objets Volumineux

➢ Généralités

- **Les LOBs :** Large Objects ont été introduits par Oracle **depuis la version 8.0**.
- Les LOBs sont adaptés pour **les applications multimédia**
- On distingue **deux classes de LOBs** :
  - **Les LOBs internes** (CLOB pour Caractère LOB, NCLOB pour National Caractère LOB for the Data Base et BLOB pour Binary LOB)
  - **Et les LOBs externes BFILEs**
- Les **LOBs** vont se substituer progressivement aux **LONG et LONG RAW**
- La taille **d'un CLOB ou BLOB**
  - 4 Go

Module M3.5, section 1 : Gestion des Objets Volumineux

➢ Généralités

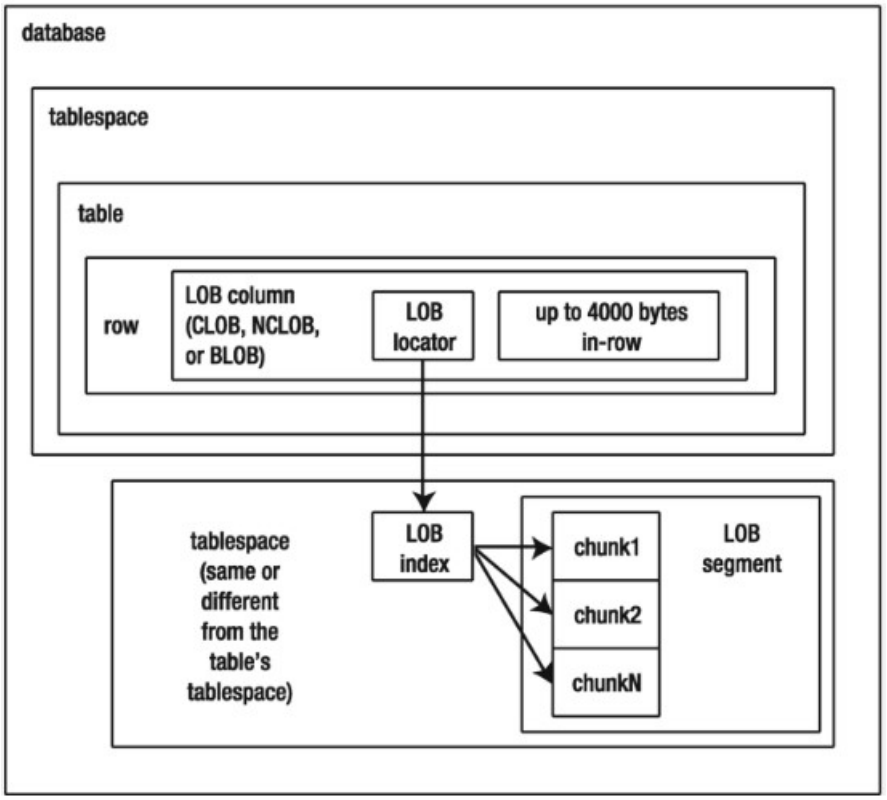
- **Différénces entre LOB et LONG/LONG RAW**
  - **Les types LONG RAW ET LONG existent toujours mais les LOB offrent plus de fonctionalités**
  - **Les LOBs offrent les avantages suivants :**
    - ✓ Plusieurs **colonnes de types LOBs dans une table sont possibles**. Par contre un seul LONG/LONG RAW par table
    - ✓ Les **LOBS** peuvent **être attributs d 'ADT (pas les LONG et LONG ROW)**
    - ✓ Les LOBs stockent uniquement dans la table LE POINTEUR vers les emplacements des données (celles-ci étant stockées dans un segment séparé). **Par contre, les LONGs stockent directement les données dans la table .**
    - ✓ Un **LOB (CLOB, BLOB)** peut atteindre **4 Go** alors qu'un **LONG/LONG RAW** est limité à **2Go**

Module M3.5, section 1 : Gestion des Objets Volumineux

➢ Généralités

- **Un LOB (objet volumineux) se décompose en 3 parties**
  - **Lob Locator (Handle)** :Pointeur vers une CASE du LOB Index. Cette CASE contient l'adresse du premier CHUNK (Block) d'un LOB (Data)
  - **LOB INDEX (ou Lob Locators) :**Ensemble de CASE contenant les adresses des CHUNCK (Block) de chaque LOB
  - **LOB Segment :** Segment contenant les données du LOB (les DATA)

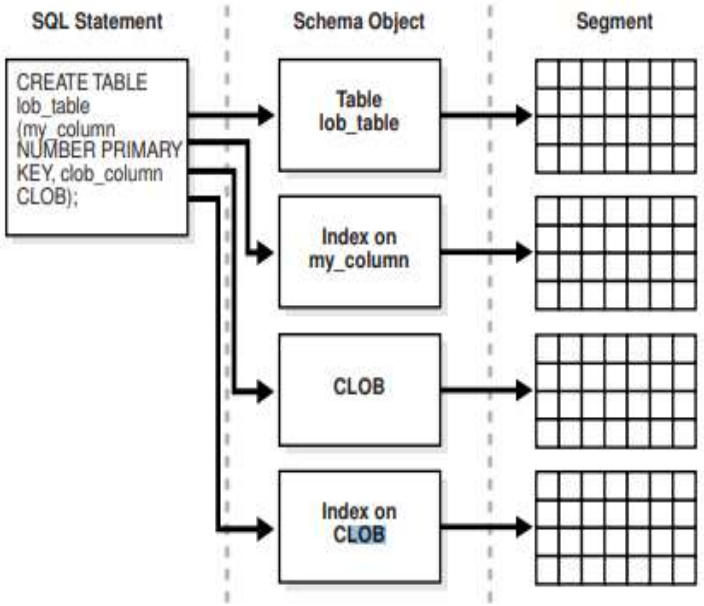
NOTE : Le Package DBMS_LOB permet de manipuler les LOBs



Module M3.5, section 1 : Gestion des Objets Volumineux

➢Généralités

- **Lors de la création d'une table contenant un LOB plusieurs segments créées :**
  - Un segment pour le LOB INDEX
  - Un segment pour les données du LOB
  - Bien d'autres segments sont aussi créés (Segment de la table contenant le LOB, segment d'index sur les colonnes Primay Key, Unique Key, etc.)



Les LOB internes

- Ce sont **des LOB de type BLOB, CLOB ou NCLOB**. Ils sont directement stockés dans la base AU CAS, DANS UN SEGMENT SEPARE

- Un lob peut être défini comme étant : **un attribut d'un ADT, une colonne d'une table, une variable SQL, une variable hôte d'un programme, une variable PL/SQL, un paramètre ou un résultat d'une fonction**

Les LOB internes

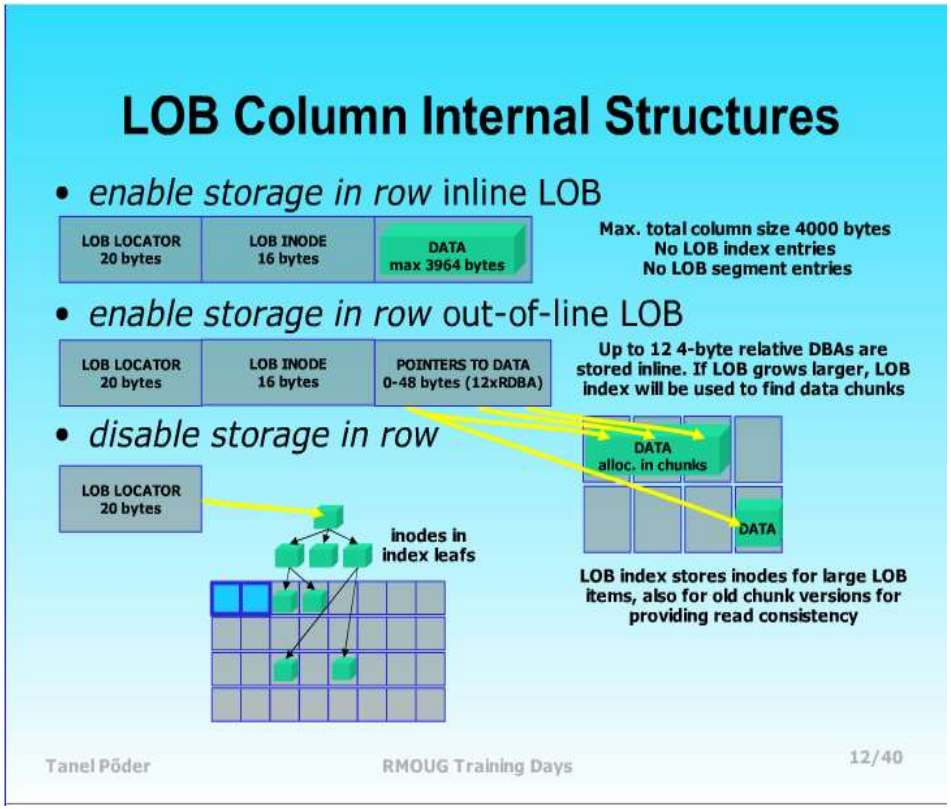
Gestion du stockage des LOBs internes

- Les LOBs et leur Index peuvent être **stockés dans des segments séparés** de celui de la table qui les contient
- **Un Tablespace séparé peut être affecté au LOB et / ou à son Index**. Par défaut il résident dans le même tablespace que la table
- Si un tablespace S est associé à un LOB mais pas à son index, ce dernier résidera dans S
- Plusieurs LOBs d'une même table peuvent être attachés à un même tablespace

Les LOB internes

Gestion du stockage des LOBs internes

- ENABLE STORAGE IN ROW est active et le lob<4K alors stockage dans la ligne (pas de Lob Index)
- ENABLE STORAGE IN ROW est active et le lob=4K*12 chunck alors stockage hors de la ligne (pas de Lob Index)
- ENABLE STORAGE IN ROW N'EST PAS ACTIVE alors externalisation dans un segment séparé + LOB Index



Les LOB internes

Contrôle du stockage des LOBs internes

Les paramètres :

PCTVERSION, FREEPOOLS, CACHE, NOCACHE, LOGGING, NOLOGGING, FILESYSTEM_LIKE_LOGGING, CHUNK, ENABLE/ DISABLE STORAGE IN ROW, BASICFILES / SECUREFILES, RETENTION, MAXFILE, COMPRESS / NOCOMPRESS , FREELISTS / FREELIST GROUP, DEDUPLICATE , KEEP_ DUPLICATES, ENCRYPT / DECRYPT permettent de contrôler le stockage des LOBs

Module M3.5, section 1 : Gestion des Objets Volumineux

➢ Les LOB internes

▪ Contrôle du stockage des LOBs internes

- **Syntaxe de creation de tables avec des lob internes**

```
Create table nomTableAvecLob(      colLob {clob | blob | nclob}, …
)
[LOB (colLob1, [clob2], …) STORE AS [ SECUREFILE | BASICFILE ]] [lobSegmentName]
[([TABLESPACE tablespaceName]
      [STORAGE([INITIAL size] [NEXT size] [MINEXTENTS n ] [MAXSIZE size]
                                    FREELISTS n] [FREELIST GROUPS n])]
      [ { ENABLE | DISABLE } STORAGE IN ROW]
      [CHUNK integer
      [PCTVERSION integer
      [RETENTION [ { MAX | MIN integer | AUTO | NONE } ]
      [FREEPOOLS integer
      [{DEDUPLICATE | KEEP_DUPLICATES}]
      [{COMPRESS [ {HIGH | MEDIUM | LOW} ] | NOCOMPRESS }]
      [{{ ENCRYPT [ USING 'encrypt_algorithm' ] | DECRYPT}]
      [{ CACHE | NOCACHE | CACHE READS } [{ LOGGING | NOLOGGING | FILESYSTEM_LIKE_LOGGING }] } ]
)]
```

LOB internes

Contrôle du stockage des LOBs internes

- **BASICFILE / SECUREFILE** : **BasicFile est le mode gestion du stockage des objets volumineux avant la 11g**. **C'est le mode par défaut**. C'est le mode à utiliser sur les LOBs dans **les tablespaces non ASSM**. La gestion en mode **secureFiles permet une gestion rapide, l'encryption, la compression et la deduplication** des LOBs

- **MAXSIZE** : Taille maximale du LOB dans la clause storage à ne pas dépasser

Module M3.5, section 1 : Gestion des Objets Volumineux

➢ Les LOB internes

▪ Contrôle du stockage des LOBs internes

- **PCTVERSION** : vaut 10% par défaut, espace réservé pour conserver les versions précédentes des LOBs afin d'assurer la lecture CONSISTANTE. Le positionner à 20% en cas de MAJ fréquentes. **Utile uniquement si BasicFiles. Utiliser RETENSION pour les SECUREFILES**

- **CHUNK** : nombre de blocks lus ou écrits d'un coup lors de l'appel des méthodes OCIlobWrite, OCIlobRead, DBMS_LOB.write ou DBMS_LOB.read
  - ✓ Note : Intitial et Next doivent être supérieurs à CHUNK
  - ✓ **Utile uniquement si BasicFiles**

- **FREEPOOLS** : Indique le nombre de Freelist Group. Vous devez être en mode gestion automatique des segments rollback . **Utile uniquement si BasicFiles**

- **FREELISTS / FREELIST GROUP** : spécifie le nombres de processus Freelists ou freelist group pour un segment. **Utile uniquement si BasicFiles**

Les LOB internes

Contrôle du stockage des LOBs internes

- **CACHE/NOCACHE/CACHE READS** : CACHE permet conserver un LOB souvent accédé dans le buffer cache(lecture/écriture), idem pour CACHE READS(lecture uniquement)
- **LOGGING/NOLOGGING** génère ou non la trace dans le fichier REDO LOG
- **FILESYSTEM_LIKE_LOGGING** : Génère la trace dans les REDO uniquement pour les méta-données des SecureFiles Lob
- **ENABLE (DISABLE) STORAGE IN ROW** : autorise, lorsque les données d'un LOB sont < à 4Ko, à écrire directement dans la ligne qui contient le LOB pour des raisons de PERFORMANCE

Les LOB internes

Contrôle du stockage des LOBs internes

- **RETENTION** :Est l'alternative à PCTVERSION pour conserver l'état avant d'un LOB en modification. La durée de retention est alignée sur celle du paramètre UNDO_RETENTION de la base. Les données rollback sont stockées dans le LOB.Valeurs possibles de RETENTION :

  MAX: si le lob atteint **MaxSize, a**lors les zones du Lob occupées par les données rollback seront réutilisées en cas d'ajout ou de modificatiobn dans le LOB

  MIN n : Conserver les données Rollbacks pendant n seconde

  AUTO: Permet de conserver les données undo pour les lectures consistantes seules. Pas de flashback db

  NONE: Pas de données Undo ni en mode flashback ni en mode lecture consistante

- **COMPRESS / NOCOMPRESS** : active ou désactive la compression intelligente des **secureFiles** Lobs

Les LOB internes

Contrôle du stockage des LOBs internes

- **DEDUPLICATE / KEEP_ DUPLICATES** : Deduplicate permet une déduplication intelligente des **SecureFiles** Lob. Si plusieurs lignes, plusieurs partitions d'un secureFiles Lob ont une même données, alors ils partageront le block qui la contient. Pas de partage si Keep_duplicate. Attention, une licence sur l'option advanced security est nécessaire

- **ENCRYPT / DECRYPT** : Permet un **cryptage** ou un **non cryptage** intelligent des **secureFiles** Lob

Les LOB internes

- **Gestion de LOB en mode BasicFiles**

```
CREATE  OR REPLACE
  TYPE LPILOTE_T AS OBJECT(
  pl#            NUMBER(4),
  nom            VARCHAR2(20),
  dnaiss         DATE,
  adr            VARCHAR2(20),
  salaire        NUMBER(7,2),
  cv_texte       CLOB,
  cv_anime       BLOB
  );
/
```

Les LOB internes

```
Drop table lpilote;
CREATE TABLE LPILOTE OF LPILOTE_T(
      CONSTRAINT pk_lpilote PRIMARY KEY(pl#)
)
LOB (cv_texte, cv_anime)  STORE AS basicfile(
    STORAGE (NEXT 10K)
    DISABLE STORAGE IN ROW
    CHUNK 4
);
```

Les LOB internes

Gestion de LOB en mode BasicFiles

```
ALTER TABLE LPILOTE
MODIFY LOB (cv_texte)
(
  CACHE
  PCTVERSION 20
  STORAGE (NEXT 10K)
   INDEX  (STORAGE (next 12K ))
);
```

B

BASICFILE

INSTITUTE OF TECHNOLOGY UNIVERSITÉ CÔTE D'AZUR

Les LOB internes

Gestion de LOB en mode secureFiles

- Le paramètre d'initialisation **DB_SECUREFILE** doit avoir la bonne valeur
  - ✓ **PERMITTED** – création de Lob en mode secureFile autorisé (jusque Oracle 11c)
  - ✓ **PREFERRED** – création de Lob en mode secureFile si BASICFILE n'est explicitement noté
  - ✓ **ALLWAYS** : Tous les Lob créés dans le file Système seront créés en mode secureFiles
  - ✓ **NEVER** – Tout Lob créé comme secureFile est converti en basicFile
  - ✓ **IGNORE** – ignore le mot clé secureFile et les options du mode secureFiles

anr INVESTIR L'AVENIR CC BY NC ND DATUM ACADEMY

Les LOB internes

- **Gestion de LOB en mode secureFiles**

```
CREATE  OR REPLACE TYPE SPILOTE_T AS
  OBJECT(
  pl#              NUMBER(4),
  nom              VARCHAR2(20),
  dnaiss           DATE,
  adr              VARCHAR2(20),
  salaire           NUMBER(7,2),
  cv_texte          CLOB,
  cv_anime          BLOB
  );
/
```

Les LOB internes

```
Drop table spilote;
CREATE TABLE SPILOTE OF SPILOTE_T(
CONSTRAINT pk_spilote PRIMARY KEY(pl#))
LOB (cv_texte, cv_anime)  STORE AS securefile(
STORAGE (NEXT 10K)
RETENTION AUTO
KEEP_DUPLICATES
COMPRESS
);
```

Les LOB internes

Initialisation des LOBs internes

- Lors d'une insertion ou d'une MAJ, un LOB interne peut être initialisé à NULL (rien)
- L'initialisation à vide se fait à travers l'opérateur EMPTY_CLOB ou EMPTY_BLOB
- Un LOB initialisé à NULL ne pourra être manipulé que s'il a été mis à vide auparavant

Les LOB internes

Initialisation des LOBs internes

- **Exemple**

```
INSERT INTO LPILOTE
      VALUES (12, 'Guela', '12-DEC-70',
      'Le Cap', 2000.0, EMPTY_CLOB(), NULL);
```

- **Avant d'utiliser un LOB initialisé à NULL il l'initialiser à vide**

```
UPDATE lpilote
SET cv_anime = EMPTY_BLOB()
WHERE pl#=12;
```

Module M3.5, section 1 : QUIZ

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ **Question 1 : Cochez ce qui est un LOB interne**

A: NCLOB
B: LONG
C: BLOB
D: CLOB
E: BFILE

➢ **Question 2 : Taille d'un LOB interne ?**

A: 2 G bytes
B: 4 G bytes
C: 8 G bytes
D: 16 G bytes
E: 32 G bytes

➢ **Question 3 : Définition d'un LOB interne ?**

A: Stocke dans les segments de la BD les pointeurs vers fichiers
B: Stocke dans les segments de la BD les objets volumineux
C: Stocke dans les segments de la BD les noms de fichiers MM**
D: Aucune des affirmations précédentes n'est juste

➢ **Question 4 : Cochez ce qui caractérise un LOB interne**

A: Une table peut contenir un et un seul LOB interne
B: : Une table peut contenir plusieurs LOB interne
C: Le nombre maximum de LOB interne par table est limité à 3000
D: un LONG ROW est aussi LOB interne

****MM: Multimédia**

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.5, section 2 : Gestion des Objets Volumineux

Module M3.5, section 2 : Gestion des Objets Volumineux

- **Plan**
  - *Généralités*
  - *Les LOBs internes*
  - **Les LOBs externes ou BFILE**
  - **Les LOBs Locators**
  - *Le Package DBMS_LOB partie 1*
  - *Le Package DBMS_LOB partie 2*

Les LOB externes

- Le type **BFILE (File Based LOB)** spécifie que le LOB sera stocké **en dehors de la base de données Oracle.**
- Le système de fichiers d'Oracle **garde en mémoire le chemin d'accès** de ce fichier.
- Les BFILE sont vus comme étant des données binaires

Les LOB externes

L'objet DIRECTORY

- Un nouvel objet appelé « **DIRECTORY** », **permet l'administration** des accès et la manipulation de fichiers
- Il permet également de gérer toutes les utilisations des FILEs stockés sur le serveur Oracle.
- Les **privilèges** autorisés pour l'objet « DIRECTORY » sont : READ, WRITE, DELETE et CREATE DIRECTORY
- **Syntaxe**

  CREATE OR REPLACE **DIRECTORY** nomDirectory AS 'PathAvecNomFichier';

  NOTE : NomDirectory => nom du répertoire dans la base

Les LOB externes

- Voici les **étapes pour installer** des objets de type **FILE** et des objets « DIRECTORY » :
  - 1. **Créer le répertoire sous l'OS** (en tant qu'utilisateur oracle) et lui donner les accès.
  - 2. **Copier les fichiers à l'intérieur de ce répertoire**
  - 3. **Créer la table oracle contenant le type BFILE**
  - 4. **Créer l'objet DIRECTORY** et lui donner les droits.
  - 5. **Insérer les lignes de la table** en associant le fichier OS.

Les LOB externes

- **Exemple** :
  - 1. **Création du répertoire sous L'OS**
    mkdir 'C:\ORADATA\DBCOUR\LOB\FILES\';
  - 2. **Copier le fichier dans ce répertoire** :
    exemple ORACLEGAL3.DOC
  - 3. **Créer la table contenant le BFILE**

```
CREATE TABLE PROJET(
        PROJ#       NUMBER(4),
        ProjNom     VARCHAR2(20),
        ProjFile    BFILE
);
```

Les LOB externes

Exemple

- **4. Créer un objet DIRECTORY**

```
CREATE OR REPLACE DIRECTORY bfile_dir
        AS 'C:\ORADATA\DBCOUR\LOB\FILES\';
```

- **5. Initialiser un BFILE grâce à la méthode**
  BFILENAME('nomDirectory', 'nomFichier');

```
INSERT INTO projet
VALUES(102, 'Objectif lune ',
BFILENAME('BFILE_DIR', 'ORACLEGAL3.DOC'));
```

- **NOTE :**
  Pas de vérification de la présence du fichier en insertion

Les LOB externes

Règles d'utilisation des directory

- Une directory **ne doit pas pointer sur les Répertoires des Fichiers** Oracle(fichiers de données, de controles our Redo Log) : RISQUE d'ACCIDENTS
- Contrôler **l'affectation du privilège CREATE ANY DIRECTORY**
- **Ne pas utiliser les commandes DROP ou REPLACE** sur les objets qui pointent vers un BFILE en cours de manipulation. Risque de perte de pointeurs
- Le retrait des privilèges à un utilisateur sur une Directory provoque l'échec de ses actions sur un BFILE

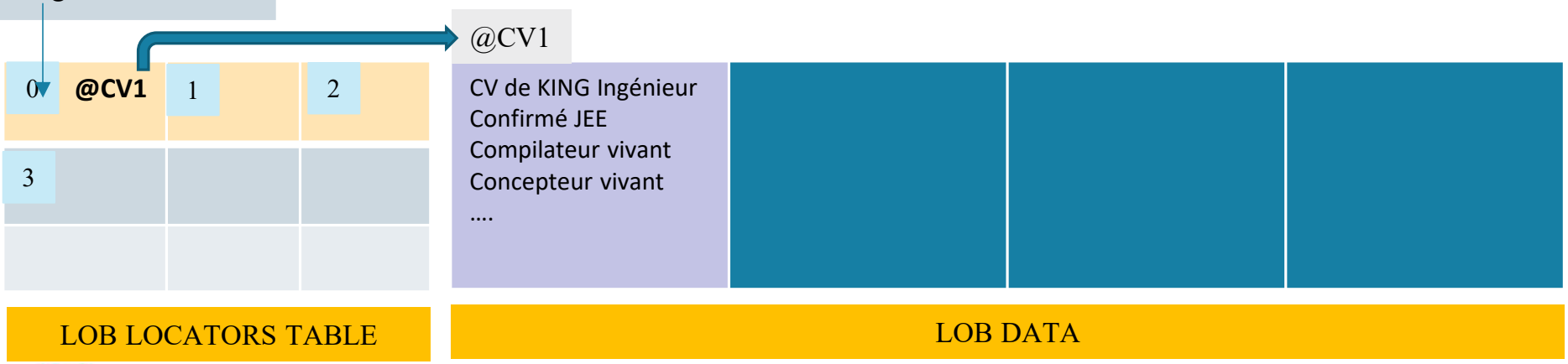
Les LOBs Locators

- **On distingue deux types de Lobs Locators :**
  - les LOBs **Locators internes**
  - les LOBs **Locators externes**
- Un **Lob Locator est tableau de pointeurs** vers les données (DATA) d'un LOB externe ou interne.
  - S'il s'agit d'un BFILE (Lob Externe)**, la donnée est un nom de fichier**
  - S'il s'agit d'un CLOB, BLOB ou NCLOB **c'est une CASE contenant la position de la valeur** du LOB dans son segment de données

Les LOBs Locators internes

1
KING
12/12/2010
CV 0

BLOB=LOB LOCATORS + LOB DATA



Les LOBs Locators

Opérations sur les LOBs Locators

- Initialisation **des colonnes devant contenir des LOCATORS**. Via INSERT et UPDATE en utilisant les Builtins :
  - ✓**EMPTY_CLOB()** RETURN CLOB;
  - ✓**EMPTY_BLOB()** RETURN BLOB;
  - ✓**BFILENAME(directory_alias, filename)** RETURN BFILE;

- **Récupération des LOBs** dans des Variables HOTEs
  SELECT nomLob **INTO** variableLob **FROM** nomTable **WHERE** primaryKey=xxx;

- **Manipulations de LOBs** à partir du **package DBMS_LOB**

Module M3.5, section 2 : QUIZ

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

- **Question 1 : Cochez ce qui est un LOB externe**
  - A: NCLOB
  - B: LONG
  - C: BLOB
  - D: CLOB
  - E: BFILE

- **Question 2 : Taille d'un LOB externe ?**
  - A: 2 G bytes
  - B: 4 G bytes
  - C: 8 G bytes
  - D: Taille d'un pointeur
  - E: Taille d'un fichier

- **Question 3 : Définition d'un LOB externe ?**
  - A: Stocke dans les segments de la BD pointeurs vers fichiers
  - B: Stocke dans les segments de la BD les objets volumineux
  - C: Stocke dans les segments de la BD les noms de fichiers MM**
  - D: Aucune des affirmations précédentes n'est juste

- **Question 4 : Cochez ce qui caractérise un LOB externe**
  - A: Une table ne peut contenir un et un seul LOB externe
  - B: : Une table peut contenir plusieurs LOB externe
  - C: Une directory Oracle doit être créée pour pointer vers un répertoire physique d'un OS contenant des fichiers
  - D: L'initialisation d' un LOB externe se fait avec la fonction BFILE_NAME(nomDirectoryOracle, nomFichier)

****MM: Multimédia**

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.5, section 2: QUIZ

➢ **Question 5 : Définition d'un Lob Locator interne ?**

A: Tableau de pointeurs. Chacun d'eux contient les noms de fichiers contenant les objets volumineux

B: Tableau de pointeurs. Chacun d'eux pointe vers un fichier externe contenant un objet volumineux

C: Tableau de pointeurs indexé. Chacun d'eux pointe vers un objet volumineux

D: Tableau de pointeurs. Chacun d'eux pointe vers un objet volumineux

➢ **Question 6 : Définition d'un Lob Locator externe?**

A: Tableau de pointeurs. Chacun d'eux contient les noms de fichiers contenant les objets volumineux

B: Tableau de pointeurs. Chacun d'eux pointe vers un fichier externe contenant un objet volumineux

C: Tableau de pointeurs indexé. Chacun d'eux pointe vers un objet volumineux

D: Tableau de pointeurs. Chacun d'eux pointe vers un objet volumineux

Module M3.5, section 2 : QUIZ

- **Question 7 : Cochez ce qui permet d'initialiser un lob interne de type BLOB**
  A: EMPTY_CLOB()
  B: EMPTY_BLOB()
  C: BFILENAME(directory_alias, filename)
  D: NULL

- **Question 8 : Cochez ce qui permet d'initialiser un lob interne de type CLOB/NCLOB**
  A: EMPTY_CLOB()
  B: EMPTY_BLOB()
  C: BFILENAME(directory_alias, filename)
  D: NULL
- **Question 9 : Cochez ce qui permet d'initialiser un lob externe**
  A: EMPTY_CLOB()
  B: EMPTY_BLOB()
  C: BFILENAME(directory_alias, filename)
  D: NULL

Module M3.5, section 3 : Gestion des Objets Volumineux

Module M3.5, section 3 : Gestion des Objets Volumineux

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Plan

- *Généralités*
- *Les LOBs internes*
- *Les LOBs externes ou BFILE*
- *Les LOBs Locators*
- **Le Package DBMS_LOB partie 1**
- *Le Package DBMS_LOB partie 2*

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Le Package DBMS_LOB partie 1

LOB PL/SQL Package (DBMS_LOB)

- La plupart des opérations utilisées sur les objets BFILEs, CLOB, BLOB et NCLOB sont contenues dans la **boite à outils 'DBMS_LOB'**.
- **Ces outils utilisent les valeurs de deux constantes :**
  - ✓ DBMS_LOB.MAX_BLOB_SIZE = 4 Go.
  - ✓ DBMS_LOB.MAX_CLOB_SIZE = 4 Go.
- **Les fonctions et procédures de ce package peuvent être classées en deux groupes :**
  - ✓ **Mutators** : Write, fdelete, copy, erase.....
  - ✓ **Observers** : Compare, read, exist, ... (interviennent en consultation).

Le Package DBMS_LOB partie 1

Classification des méthodes

- **Méthodes de MAJ des BLOB, CLOB et NCLOB**

✓Append()	Ajoute le contenu du source vers Destination
✓Copy()	copie un LOB source vers un autre
✓ERASE()	Efface toute les parties d'un LOB
✓LOADFROMFILE()	Charge bfile vers un lob interne
✓LOADCLOBFROMFILE()	Charge un Bfile vers clob
✓LOADBLOBFROMFILE()	Charge un bfile vers un blob
✓TRIM()	Réduit la valeur d'un LOB à la taille spécifiée
✓WRITE	Ecriture des données dans un LOB à partir d'un OFFSET
✓WRITEAPPEND	Ecriture des données dans un LOB à la fin du fichier

Le Package DBMS_LOB partie 1

Classification des méthodes

- **Méthodes de Lecture et de contrôle des LOB internes et Externes**

✓ compare	Compare deux LOBs ou deux parties de LOBs
✓ getLength()	Rend la longueur d'un LOB
✓ instr()	Rend la position de la Nième occurrence d'un modèle dans le LOB
✓ read()	Lecture des données dans un LOB à partir de la position OFFSET
✓ substr()	Rend une "partie" du LOB à partir de la position OFFSET

Le Package DBMS_LOB partie 1

Classification des méthodes

- **Méthodes spécifiques aux BFILE**

✓ getLength()	Rend la longueur d'un fichier
✓ fileClose()	Fermeture de tous les fichiers ouverts
✓ fileExist()	Teste si le fichier existe bien dans sa directory
✓ fileGetName()	Rend le nom Oracle de la directory et le nom du fichier
✓ fileIsOpen()	Test si un fichier est ouvert
✓ fileOpen()	Ouverture file

Le Package DBMS_LOB partie 1

Classification des méthodes

- **Les types de données en entrée ou en sortie des méthodes sur les BLOB**
  - ✓ **BLOB** : source ou destination des Binary LOB
  - ✓ **RAW** : source "raw buffer" et destination "raw buffer" (BLOB)
  - ✓ **CLOB** : source ou destination des Character LOB
  - ✓ **VARCHAR2** :source/destination des buffers de caractères (utile pour manipuler les CLOB et NCLOB)
  - ✓ **INTEGER** : spécifie la taille du buffer ou du LOB, l'offset du LOB

Le Package DBMS_LOB partie 1

La méthode READ

- Cette méthode **permet de lire des données dans les LOB** internes ou externes
- **Trois prototypes** sont disponibles selon le type de LOB : BLOB, CLOB/NCLOB, BFILE
  - ✓ 1. procedure **READ**(lob_loc IN **BLOB**, amount IN OUT BINARY_INTEGER, offset IN TEGER, buffer OUT RAW)
  - ✓ 2. procedure **READ**(lob_loc IN **CLOB** CHARACTER SET ANY_CS, amount IN OUT BINARY_INTEGER, offset IN TEGER, buffer OUT VARCHAR2 CHARACTER SET LOB_LOC%CHARSET)
  - ✓ 3. procedure **READ**(lob_loc IN **BFILE**, amount IN OUT BINARY_INTEGER, offset IN TEGER, buffer OUT RAW)

Module M3.5, section 3 : Gestion des Objets Volumineux

➢ Le Package DBMS_LOB partie 1

▪ La méthode READ

- **Les paramètres**

Nom	Description
lob_loc	Locator du LOB à lire
amount	Quantité des données à lire
offset	Position à partir du début du CLOB ou BLOB
buffer	Buffer dans lequel est stocké le résultat

- **Les exceptions pouvant survenir**

VALUE_ERROR
INVALID_ARGVAL
NO_DATA_FOUND
Pour les BFILEs seuls : UNOPENED_FILE, NO_EXIST_DIRECTORY, NOPRIV_DIRECTORY, INVALID_OPERATION

Module M3.5, section 3 : Gestion des Objets Volumineux

➢ **Le Package DBMS_LOB partie 1**

- **La méthode READ**
  - **Exemple de lecture d'un BLOB**

```
CREATE OR REPLACE PROCEDURE readCvAnime IS
        lobCvAnime      BLOB;
        buffer          RAW(32767);
        qte             BINARY_INTEGER := 32767;
        position        INTEGER := 1;
        BEGIN
                SELECT cv_anime INTO lobCvAnime
                FROM  lpilote  WHERE pl#=1;
                        LOOP
                        DBMS_LOB.READ(lobCvAnime, qte, position,

                        buffer);
                        /* Insérer les Manipulations ICI*/
                        position := position+qte;
                END LOOP;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No More Data');
END;
/
```

Le Package DBMS_LOB partie 1

- **La méthode READ**
  - **Exemple de lecture dans un BFILE**

```
CREATE OR REPLACE PROCEDURE readProjetFile IS
   bfProjetFile     BFILE;
   buffer           RAW(32767);
   qte              BINARY_INTEGER := 32767;
   position         INTEGER := 1;
   BEGIN
            SELECT projFile INTO bfProjetFile
            FROM  projet  WHERE proj#=102;
```

```
-- ouverture du fichier (Lecture seule sinon erreur)
IF DBMS_LOB.FILEEXISTS(bfProjetFile)=1 THEN
        DBMS_OUTPUT.PUT_LINE('Fichier Ouvert ! ');
        DBMS_LOB.FILEOPEN(bfProjetFile,
                        DBMS_LOB.FILE_READONLY);
ELSE
        DBMS_OUTPUT.PUT_LINE('Fichier inexistant');
END IF;
--position := position+qte;
-- à suivre
```

Le Package DBMS_LOB partie 1

La méthode READ

- **Exemple de lecture d'un BFILE**

```
LOOP
        DBMS_LOB.READ(bfProjetFile, qte, position, buffer);
        position := position+qte;
        /* Insérer les Manipulations ICI*/
  END LOOP;
  EXCEPTION
        WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Fin du LOB');
        DBMS_LOB.FILECLOSE(bfProjetFile);
  END;
  /
```

Le Package DBMS_LOB partie 1

La méthode WRITE

- Cette méthode **permet d'écrire des données dans un LOB** interne à partir d'un OFFSET
- **Deux prototypes** sont disponibles selon le type de LOB : BLOB, CLOB/NCLOB, BFILE
  - ✓ 1. procedure **WRITE**(lob_loc IN OUT **BLOB**, amount IN IN BINARY_INTEGER, offset IN TEGER, buffer IN RAW)
  - ✓ 2. procedure **WRITE**(lob_loc IN OUT **CLOB** CHARACTER SET ANY_CS, amount IN BINARY_INTEGER, offset IN TEGER, buffer IN VARCHAR2 CHARACTER SET LOB_LOC%CHARSET)

Module M3.5, section 3 : Gestion des Objets Volumineux

➢ **Le Package DBMS_LOB partie 1**

- **La méthode WRITE**
  - **Les paramètres**

Nom	Description
lob_loc	Locator du LOB interne ou écrire
amount	Nombre de bytes à écrire ou écrits
offset	Position (à partir du début du CLOB ou BLOB) à partir de laquelle on va écrire
buffer	Buffer contenant les données à écrire

  - **Les exceptions pouvant survenir**

VALUE_ERROR
INVALID_ARGVAL

Module M3.5, section 3 : Gestion des Objets Volumineux

➢Le Package DBMS_LOB partie 1

▪ La méthode WRITE

- **Exemple d'écriture dans un BLOB**

```
CREATE OR REPLACE PROCEDURE writeCvTexte IS
        lobCvtexte          CLOB;
        buffer    VARCHAR2(32767);
        qte                 BINARY_INTEGER := 32767;
        position INTEGER:=1;
        BEGIN
                SELECT cv_texte INTO lobCvTexte
                FROM  lpilote  WHERE pl#=12;
                -- préparer des données dans buffer puis écrire
                buffer := 'Il ya un debut a tout';
                DBMS_LOB.WRITE(lobCvTexte, qte, position, buffer);
EXCEPTION
                WHEN VALUE_ERROR THEN
                DBMS_OUTPUT.PUT_LINE('Erreur de Valeurs');
END;
/
```

Module M3.5, section 3 : QUIZ

➢**Question 1 : Cochez ce qui caractérise le package DBMS_LOB**

A: Les fonctions du package DBMS_LOB ne sont pas surchargées et ne sont pas surchargeables

B: Le package DBMS_LOB fournit des fonctions de mises à jour et de lecture des lob internes

C: Le package DBMS_LOB fournit des fonctions de mises à jour et de lecture des lob externes

D: Le package DBMS_LOB fournit des fonctions test sur les LOB internes et les lob externes

➢**Question 2 : Que fait la fonction du package DBMS_LOAD**.LOADFROMFILE

DBMS_LOB.LOADFROMFILE (dest_lob IN OUT NOCOPY BLOB, src_file IN BFILE, amount IN INTEGER, dest_offset IN INTEGER := 1, src_offset IN INTEGER := 1);

A: Cette fonction permet de supprimer une partie d'un LOB interne

B: Cette fonction permet de récuper le contenu d'un fichier (image, vidéo, texte) du système de fichier vers un CLOB/BLOB dans la base de données

C: Cette fonction permet mettre à jour un CLOB/BLOB dans la base de données

D: Il est possible d'indiquer la position à partir de laquelle il faut écrire dans un CLOB/BLOB

Module M3.5, section 3 : QUIZ

➢ **Question 3 : Cochez les fonctions faisant partie du package DBMS_LOB**

A: DBMS_LOB.LOADCLOBFROMFILE

B: DBMS_LOB.FILEGETNAME

C: DBMS_LOB.WRITEAPPEND

D: DBMS_LOB.LOADCLOBFROMFILE_FROMSECUREFILE

Question 4 : Cochez les fonctions qui permettent de modifier des LOB de type CLOB

A: DBMS_LOB.WRITEAPPEND

B: DBMS_LOB.LOADFROMFILE

C: DBMS_LOB.OPEN

D: DBMS_LOB.READ

Module M3.5, section 4 : Gestion des Objets Volumineux

Module M3.5, section 3 : Gestion des Objets Volumineux

- **Plan**
  - *Généralités*
  - *Les LOBs internes*
  - *Les LOBs externes ou BFILE*
  - *Les LOBs Locators*
  - *Le Package DBMS_LOB partie 1*
  - **Le Package DBMS_LOB partie 2**

Module M3.5, section 4 : Gestion des Objets Volumineux

➢ **Le Package DBMS_LOB partie 2**

- **Exemple de mise à jour d'un CLOB avec WriteAppend et Update**

```
Drop table myEmp;

create table myEmp(
    insee    number(10) primary key,
    nom      varchar2(20),
    sal      number(7,2),
    cv       clob);

DELETE FROM myEmp;

INSERT INTO myEmp VALUES (1, 'Tintin', 10000, 'MON CV');

COMMIT;

set serveroutput on
```

```
DECLARE
num_var INTEGER;
cv1     CLOB;
read_amount INTEGER;
read_offset INTEGER;
buffer VARCHAR2(1000);
BEGIN
-- Etape 1 : stockage du CLOB dans la variable locale cv1
SELECT cv INTO cv1 FROM myEmp WHERE insee = 1;

read_amount := dbms_lob.getlength(cv1);
read_offset := 1;
```

Module M3.5, section 4 : Gestion des Objets Volumineux

- **Le Package DBMS_LOB partie 2**

  - **Exemple de mise à jour d'un CLOB (suite)**

```
-- Lecture du contenu du LOB vers la variable Buffer
dbms_lob.read(cv1, read_amount,  read_offset, buffer);

-- Affichage du LOB contenu dans CV1 depuis buffer
dbms_output.put_line('cv1 value: ' || buffer);
-- L'information affichée sera : 'MON CV

-- Etape 2: Verrouiller le CV à mettre à jour plus tard
SELECT CV into CV1 FROM  myEmp WHERE  insee = 1 for update;

-- cv1 contient toujours la valeur affectée à l'étape 1
buffer := null;
read_amount := dbms_lob.getlength(cv1);

-- vérification
dbms_lob.read(cv1, read_amount, read_offset, buffer);
dbms_output.put_line('cv11 value: ' || buffer);
-- L'information affichée sera : MON CV

-- Etape 3:  Ecriture dans un lob avec writeAppend
buffer := 'BlablaBlablaBlablaBlablaBlablaBlablaBlablaBlablaBlabla
BlablaBlablaBlablaBlablaBlablaBlablaBlablaBlablaBlabla
BlablaBlablaBlablaBlablaBlablaBlablaBlablaBlabla
BlablaBlablaBlabla';
```

Module M3.5, section 4 : Gestion des Objets Volumineux

Le Package DBMS_LOB partie 2

- **Exemple de mise à jour d'un CLOB (suite)**

```
-- Récupérer le CV dans la base avec SELECT. Il  ya« MON CV »

SELECT cv INTO cv1 FROM myEmp WHERE insee = 1;

--Mettre à jour le CV avec CV1 dans la base sans faire de UPDATE
dbms_lob.writeappend(cv1, length(buffer), buffer);

-- Récupérer le CV dans la base avec SELECT
SELECT cv INTO cv1 FROM myEmp WHERE insee = 1;

read_amount := dbms_lob.getlength(cv1);

read_offset := 1;

dbms_lob.read(cv1, read_amount, read_offset,buffer);
dbms_output.put_line('cv12 value: ' || buffer);
-- On verra "MON CVBLABLA....."
```

```
-- Etape 5 : UPDATE du CV dans le base par CV1
UPDATE myemp SET cv= cv1 WHERE insee=1;

-- Récupérer le CV dans la base avec SELECT
SELECT cv INTO cv1 FROM myEmp WHERE insee = 1;

read_amount := dbms_lob.getlength(cv1);

read_offset := 1;

dbms_lob.read(cv1, read_amount, read_offset,buffer);

dbms_output.put_line('cv13 value: ' || buffer);
-- On verra "MON CVBLABLA....."

-- ERROR: ORA-01403: no data found
Commit;
END;
/
```

Module M3.5, section 4 : Gestion des Objets Volumineux

- **Le Package DBMS_LOB partie 2**
  - **Exemple de mise à jour d'un CLOB à partir d'un Bfile**

    -- **Création du répertoire sous L'OS**

    -- 1. Création du répertoire sous L'OS
    mkdir 'C:\APP\ORACLE\oradata\orcl\BFILES\';

    -- Y déposer les fichiers exemple cv1Miloud.txt, cv2Boumlid.txt, cv3Milou.txt

-- 3. Créer la table contenant le BFILE

```
Drop table bfiles;
drop table clobs;
create table bfiles(
        id      number (5),
        text    bfile);
```

-- 4. Créer un objet DIRECTORY

```
drop directory bfile_dir;

CREATE OR REPLACE DIRECTORY bfile_dir  AS
'C:\APP\ORACLE\oradata\orcl\BFILES\';
```

Le Package DBMS_LOB partie 2

- **Exemple de mise à jour d'un CLOB à partir d'un Bfile**

```
-- 5. Insertion de bfiles
delete from bfiles;
INSERT INTO bfiles VALUES(1, BFILENAME('BFILE_DIR', 'cv1Miloud.txt'));
INSERT INTO bfiles VALUES(2, BFILENAME('BFILE_DIR', 'cv2Boumlid.txt'));
INSERT INTO bfiles VALUES(3, BFILENAME('BFILE_DIR', 'cv3Milou.txt'));

create table clobs(
        id      number (5),
        text            clob
);
```

Module M3.5, section 4 : Gestion des Objets Volumineux

- **Le Package DBMS_LOB partie 2**
  - **Exemple de mise à jour d'un CLOB à partir d'un Bfile**

```
DECLARE
  cursor c is
  select id, text from bfiles;
  v_clob clob;
  lg number(38);
BEGIN
  for j in c
  loop
  Dbms_Lob.FileOpen ( j.text,
  Dbms_Lob.File_Readonly );
  lg:=Dbms_Lob.GETLENGTH(j.text);
  insert into clobs ( id, text )
  values ( j.id, empty_clob() )
  returning text into v_clob;
    Dbms_Lob.LoadFromFile(
      dest_lob => v_clob,
      src_lob => j.text,
      amount => lg,
      dest_offset => 1,
      src_offset => 1
    );
    --amount => 4294967296, /* = 4Gb */
    Dbms_Lob.FileClose ( j.text );
    end loop;
    commit;
END;
/
```

Module M3.5, section 4 : Gestion des Objets Volumineux

➢ Le Package DBMS_LOB partie 2

- **Exemple de mise à jour d'un CLOB à partir d'un Bfile**

```
-- vérification
col text format a50
select * from clobs;
```

ID	TEXT
1	Nom : Miloud Prenom :Mohamed date naissance :10/10/1980 Compétence :Ingénie
2	Nom : Boumlid Prenom : Foudelle date naissance :10/10/1982 Compétence :Plan
3	Nom : Milou Prenom : Tintin date naissance :10/10/1978 Compétence :Directeur

Le Package DBMS_LOB partie 2

Les vues du dictionnaire Oracle

- **Les vues sur les LOBS** : ALL_LOBS, DBA_LOBS, USER_LOBS
- **Les vues sur les directories** : ALL_DIRECTORIES, DBA_DIRECTORIES

DESCRIBE DBA_LOBS

Name	Null?	Type
OWNER	NOT NULL	VARCHAR2(30)
TABLE_NAME	NOT NULL	VARCHAR2(30)
COLUMN_NAME		VARCHAR2(400
SEGMENT_NAME	NOT NULL	VARCHAR2(30)
INDEX_NAME	NOT NULL	VARCHAR2(30)
CHUNK		NUMBER
PCTVERSION	NOT NULL	NUMBER
CACHE		VARCHAR2(3)
LOGGING		VARCHAR2(3)
IN_ROW		VARCHAR2(3)

Module M3.5, section 4 : Gestion des Objets Volumineux

➢ Le Package DBMS_LOB partie 2

▪ Les vues du dictionnaire Oracle

- **DESCRIBE DBA_LOBS**

Name	Description
…	
CHUNK	Unité de transfert dans un LOB : Multiple d'un Bloc
PCTVERSION	Zone de lecture consistante
CACHE	Cache mémoire pour le LOB
IN_ROW	Vrai si stockage dans le segment contenant le LOB
…	

- **DESCRIBE DBA_DIRECTORIES**

Name	Null?	Type
OWNER	NOT NULL	VARCHAR2(30)
DIRECTORY_NAME	NOT NULL	VARCHAR2(30)
DIRECTORY_PATH		VARCHAR2(4000)

Module M3.5, section 4 : Gestion des Objets Volumineux

➢ **Le Package DBMS_LOB partie 2**

- **Les vues du dictionnaire Oracle**

  - **Lister les informations sur les LOBs**

```
SELECT TABLE_NAME,    COLUMN_NAME, SEGMENT_NAME,  INDEX_NAME, CHUNK, PCTVERSION,    CACHE, IN_ROW
FROM    dba_lobs  WHERE table_name IN ('LPILOTE', 'PROJET');
```

TABLE_NAME	COLUMN_NAME	*SEGMENT_NAME*	INDEX_NAME	CHUNK	*PCTVERSION*	CACHE	*IN_ROW*
LPILOTE	CV_TEXTE	*SYS_LOB0000003236C00*	SYS_IL0000003236C000	2048	007$ 07$ *20*	YES	*NO*
LPILOTE	CV_ANIME	*SYS_LOB0000003236C00*	SYS_IL0000003236C000	2048	008$ 08$ *10*	NO	*NO*

BY NC ND

Le Package DBMS_LOB partie 2

Les vues du dictionnaire Oracle

- **Lister** les informations sur toutes **les Directories** de votre base.

```
SELECT OWNER, DIRECTORY_NAME, DIRECTORY_PATH
FROM dba_directories;
```

OWNER	DIRECTORY_NAME	DIRECTORY_PATH
SYS	BFILE_DIR2	F:\oradata\DBCOUR\LOB\files\
SYS	BFILE_DIR	C:\ORADATA\DBCOUR\LOB\FILES

Le Package DBMS_LOB partie 2

- **Lecture Consistante et LOB :**
  - Oracle assure que deux processus peuvent agir sur un même CHUNCK d'un LOB. L'un en lecture l'autre en écriture.
- **Le nombre maximum de fichiers BFILE ouverts est contrôlé via le paramètre d'initialisation**
  - SESSION_MAX_OPEN_FILES=valeur
  - Le développeur doit, s'il a ouvert un fichier le fermer. SINON RISQUE DE DEPASSEMENT du PARAMETRE SESSION_MAX_OPEN_FILES

Le Package DBMS_LOB partie 2

- **Pas de LOB distribué**, mais réplication des lobs aujourd'hui possible
- **Pas de chargements de LOB avec SQLLOADER** en V8. Utiliser dbms_lob.loadfromfile()
- **Les LOBs ne peuvent apparaître** : dans des requêtes avec Group by, Order by, select distinct, join
- **Pas d'appel des méthodes du package DBMS_LOB dans le code PL/SQL du poste client (V8)**, maintenant supporté

Module M3.5, section 4 : QUIZ

➢ **Question 1 : Cochez les vues du dictionnaires qui donnent les informations sur les objets volumineux de tous les utilisateurs**

A: USER_LOBS

B: DBA_LOBS

C: CBD_LOBS

D: ALL_DIRECTORIES

➢ **Question 2 : Cochez les vues du dictionnaires qui donnent les informations sur les directories créées dans la base**

A: CDB_DIRECTORIES

B: DBA_DIRECTORIES

C: ALL_DIRECTORIES

D: USER_DIRECTORIES

Module M3.5 : Bilan et Exercices

- Bilan
- Exercices

UNIVERSITÉ CÔTE D'AZUR ESTIA INSTITUTE OF TECHNOLOGY

DATUM ACADEMY

Module M3.6 : PLSQL étendu aux objets complexes et Mapping Objet relationnel dans Java/JDBC

FUN FRANCE UNIVERSITÉ NUMÉRIQUE anr INVESTIR L'AVENIR MIAGE RÉSEAU DES MIAGE DE FRANCE Nice MBDS BIHAR

Module M3.6 : PLSQL étendu aux objets complexes et Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Plan

- **Module M3.6, section 1** : PLSQL étendu aux objets complexes
- **Module M3.6, section 2** : PLSQL étendu aux objets complexes
- **Module M3.6, section 3, partie 1** : Mapping Objet relationnel dans Java/JDBC
- **Module M3.6, section 3, partie 2** : Mapping Objet relationnel dans Java/JDBC

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 1 : PLSQL étendu aux objets complexes

Module M3.6, section 1 : PLSQL étendu aux objets complexes

ESTIA INSTITUTE OF TECHNOLOGY

UNIVERSITÉ CÔTE D'AZUR

Plan

- Généralités
- Les types
- Les instances
- Les collections en PL/SQL permanentes
- Les collections en PL/SQL temporaires
- *Les call out de PL/SQL*
- *Bulk SQL et Bulk binding*

anr

INVESTIR L'AVENIR

CC BY NC ND

DATUM ACADEMY

Module M3.6, section 1 : PLSQL étendu aux objets complexes

➢ Généralités

- Version disponible du **coté du serveur** Oracle et **coté outils clients**
- Supporte la programmation **Orienté Objet**
- **Améliore les performances** grâce à la centralisation du code sur le moteur base de données
- **Augmente la portabilité** grâce à la centralisation du code coté serveur

Module M3.6, section 1 : PLSQL étendu aux objets complexes

➢ Les types

- **Oracle Objet distingue deux sortes de types :**
  - *.Les types définis en SQL (**types permanents**): Les ADT définis à ce niveau peuvent avoir des méthodes*
  - *Les types définis en PL/SQL (**types temporaires**) : leur durée **de vie est limitée à l 'exécution du programme**.*

- **Définition d'un type permanent en SQL**

```
SQL>CREATE OR REPLACE TYPE pAdresse_t AS OBJECT(
        Numero          NUMBER(4),
        Rue             VARCHAR2(20),
        Code_Postal     NUMBER(5),
        Ville           VARCHAR2(20)
);
/
```

Ce type pourra être utilisé dans la déclaration de variables PL/SQL

Module M3.6, section 1 : PLSQL étendu aux objets complexes

➢ Les types

- **Caractéristiques d'un type temporaire en PL/SQL**
  - Les types **temporaires sont définis grâce au mot clé TYPE**. Il sera possible de définir des "Types Tables", des Types RECORD, des Types VARRAY et des Types REF CURSOR
  - **Les différentes Syntaxes**
    - ✓ **TYPE nomType IS VARRAY**(limit) OF typeDesElements;
    - ✓ **TYPE nomType IS TABLE** OF typeDesElements ;
    - ✓ **TYPE nomType IS RECORD** (déclarationDesChamps, …);
    - ✓ **TYPE nomType REF CURSOR** OF typeDesElements ;
  - **Les éléments d'un VARRAY ou d'un Type table peuvent pas être** : BOOLEAN, NCHAR, NVARCHAR2, des types définis par l'utilisateur, des REF CURSOR, des TYPES TABLES ou des VARRAY

Module M3.6, section 1 : PLSQL étendu aux objets complexes

Les types

- **Définition d'un type temporaire en PL/SQL**
  - **Exemple**

```
DECLARE
    TYPE numList_t IS TABLE OF NUMBER;
    TYPE jourSeamine_t IS VARRAY(7) OF VARCHAR2(13);
    TYPE tabAdresse_t IS VARRAY(2) OF PAdresse_t;
    TYPE TAdresse_t IS   RECORD(
                Numero          NUMBER(4),
                Rue             VARCHAR2(20),
                Code_Postal     NUMBER(5),
                Ville           VARCHAR2(20));
    p  tadresse_t;
BEGIN
  p.numero :=10;
  dbms_output.put_line('p.numero ='|| p.numero );
END;
/
```

NOTE : adresse_t est est type permanent

Les instances

- **Instantiation d'un ADT permanent**

```
-- création de la table objet Adresse
CREATE TABLE Adresse OF pAdresse_t ;
  -- type abstrait défini préalablement

INSERT INTO Adresse
VALUES(pAdresse_t(1, 'rue Arago',06000, 'NICE') ;
```

Les instances

- **Instantiation d'un ADT temporaire**

```
DECLARE
    TYPE TAdresse_t IS  RECORD(
                Numero  NUMBER(4),
                Rue             VARCHAR2(20),
                Code_Postal     NUMBER(5),
                Ville           VARCHAR2(20));
  tadr1 TAdresse_t:= TAdresse_t(1,'rueArago',06000,'NICE');
  tadr2  TAdresse_t:=NULL;  -- initialisation à NULL
  BEGIN
    IF tadr2 IS NULL THEN
        tadr2.Rue :='5th Avenue'; -- EXCEPTION
        dbms_output.put_line('tadr2.Rue  : '||tadr2.Rue );
    END IF;
  END ;
/
```

Les collections permantes en PL/SQL

- Les collections permanentes sont créés coté serveur du SGBD Oracle

- Exemple de manipulation d'une collection permanente de type NESTED TABLE

```
SQL> CREATE  OR REPLACE TYPE avion9_t as object (
                Av#             NUMBER(4),
                AvType          VARCHAR2(45),
                Loc             VARCHAR2(20),
                Cap             NUMBER(4)
);
/

SQL>CREATE OR REPLACE TYPE avion9_set_t AS TABLE OF avion9_t;
```

BY NC ND

Les collections permantes en PL/SQL

- Exemple de manipulation d'une **collection permanente**

- Création **d'une table avec une colonne de type table**

```
CREATE TABLE hangar9 (
    hangar#          NUMBER(2),
    avions9          avion9_set_t
  )
NESTED TABLE avions9  STORE AS storeAvions9;
```

BY NC ND

Module M3.6, section 1 : PLSQL étendu aux objets complexes

➢ Les collections permantes en PL/SQL OBJET

- Exemple de manipulation d'une **collection permanente**

```
DECLARE
  tabav9  avion9_set_t;
BEGIN
  tabav9 := avion9_set_t(
                    avion9_t(1, 'Concorde', 'Paris', 100),
                    avion9_t(2, 'A300', 'Berlin', 300),
                    avion9_t(3, 'A320', 'Paris', 200)
            );
  INSERT INTO  hangar9
  VALUES (1, avion9_set_t(avion9_t(4, 'Caravelle',       'Paris', 250), avion9_t(5, 'A300', 'Nice', 300)));

  INSERT INTO  hangar9 VALUES(2, tabav9);
END;
/
```

Les collections temporaires en PL/SQL OBJET

- Une collection **temporaire est définie dans un programme PLSQL** et est visible dans ce programme uniquement
- Exemple de manipulation d'une **collection temporaire**

```
DECLARE
  TYPE tavion9_set_t IS VARRAY(5) of avion9_t ;
  tabav9  tavion9_set_t;
BEGIN
  tabav9 := tavion9_set_t(
    avion9_t(1, 'Concorde', 'Paris', 100),
    avion9_t(2, 'A300', 'Berlin', 300),
    avion9_t(3, 'A320', 'Paris', 200));
  For i in tabav9.first .. tabav9.last
  loop
    dbms_output.put_line('tabav9('||i||')='||tabav9(i).avtype);
  End loop;
END;
/
```

Module M3.6, section 1 : QUIZ

➢ **Question 1 : Cochez les types qui sont des types permanents**

A: TYPE tableauAdresse_t IS VARRAY(2) OF Adresse_t;-- Adresse_t existe

B: TYPE ListeDeNombres_t IS TABLE OF NUMBER;

C: TYPE Adresse2_t IS RECORD(Numero NUMBER(4), ...):

D: create or replace type Employe2_t as object (empno number(4), …);

E: create or replace ListeDeNombres2_t IS TABLE OF NUMBER;

➢ **Question 2 : Cochez les types qui sont des types temporaires**

A: TYPE tableauAdresse_t IS VARRAY(2) OF Adresse_t;-- Adresse_t existe

B: TYPE ListeDeNombres_t IS TABLE OF NUMBER;

C: TYPE Adresse2_t IS RECORD(Numero NUMBER(4), ...):

D: create or replace type Employe2_t as object (empno number(4), …);

E: create or replace ListeDeNombres2_t IS TABLE OF NUMBER;

Module M3.6, section 1 : QUIZ

➢ **Question 3 : Cochez ce qui est possible**

A: BEGIN TYPE ListeDeNombres_t IS TABLE OF NUMBER; END;

B: BEGIN CREATE OR REPLACE Employe2_t as object (empno number(4), …); END;

C: DECLARE TYPE ListeDeNombres_t IS TABLE OF NUMBER; BEGIN null; END;

D: DECLARE CREATE OR REPLACE Employe2_t as object (empno number(4), …); BEGIN null; END;

E: Un type temporaire est disponible uniquement dans le programme PLSQL ou il est créé

G: Le mot clé TYPE permet de définir un type temporaire ou permanent

Module M3.6, section 1 : QUIZ

➢ **Question 4 : PLSQL et la persistance. Cochez ce qui est possible après avoir étudié le code qui suit**

```
CREATE OR REPLACE TYPE Etudiant2_t as object(idEtu number(4), nomEtu varchar2(30));
CREATE TABLE o_etudiant2 of Etudiant2_t (primary key(idEtu));
insert into o_etudiant2 values (1, 'King');
insert into o_etudiant2 values (2, 'Milou');
set serveroutput on
declare
refEtu REF Etudiant2_t;
etu1 Etudiant2_t;
begin
SELECT ref(etu) INTO refEtu FROM o_etudiant2 etu WHERE etu.idEtu=1;
dbms_output.put_line('nomEtu='||refEtu.nomEtu); --L1
--utl_ref.select_object(refEtu, etu1); --L2
--dbms_output.put_line('nomEtu='||etu1.nomEtu); --L3
end;
/
```

A: **refEtu**.nomEtu permet effectivement d'accéder au nom de l'étudiant Nr 1

B: La réponse en A ne marche pas, il faut faire **deref(refEtu).**nomEtu pour accéder au nom de l'étudiant Nr 1

C: Le code PL/SQL ci-dessus ne fonctionne pas. La navigation avec les variables REF n'est pas possible en PLSQL

D: Commentez L1 et décommentez L2 et L3 ça ira

Module M3.6, section 2 : PLSQL étendu aux objets complexes

Module M3.6, section 2 : PLSQL étendu aux objets complexes

Plan

- *Généralités*
- *Les types*
- *Les instances*
- *Les collections permanentes en PL/SQL*
- *Les collections temporaire,,s en PL/SQL*
- **Les call out de PL/SQL**
- **Bulk SQL et Bulk binding**

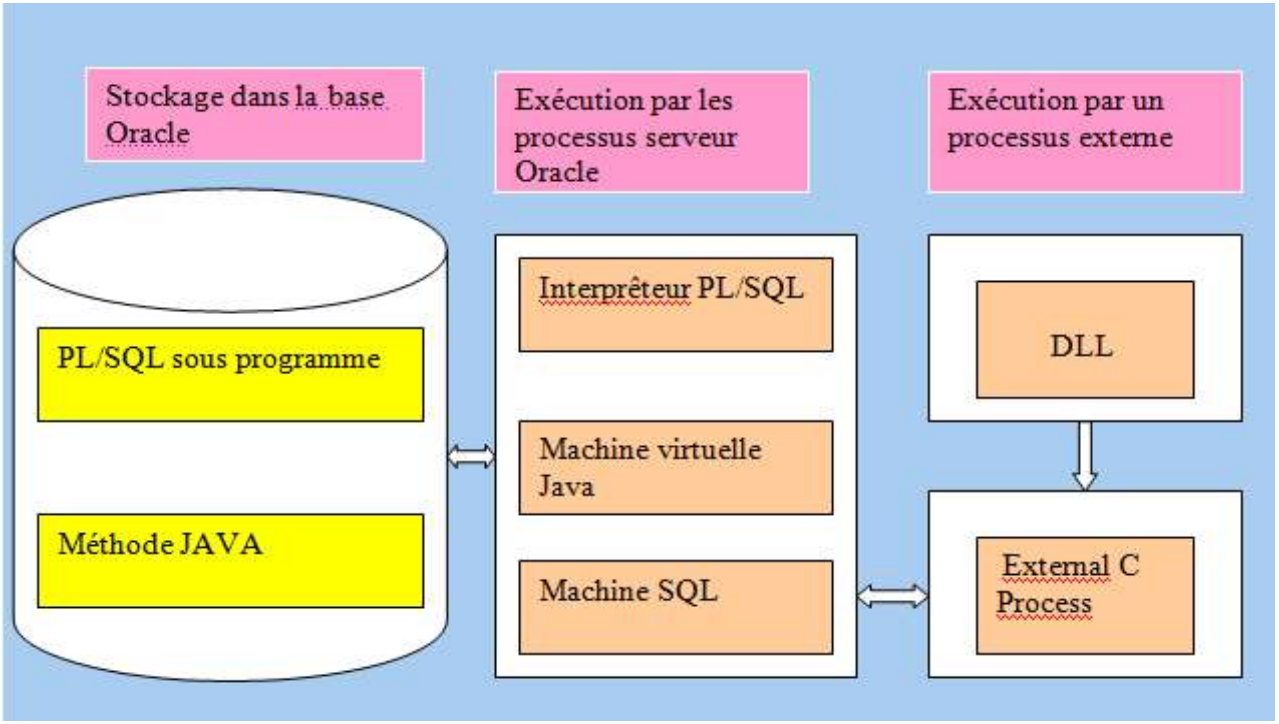
Les CALL OUT de PL/SQL OBJET

- Il est possible d'exécuter **un code d'un L3G** (généralement C, C++, PASCAL, COBL, …) **à partir d'un code PL/SQL**

- Ce **code L3G s'exécute dans un espace d'adressage séparé**. **Il a un accès contrôlé grâce aux objets LIBRARY.**

Les CALL OUT de PL/SQL OBJET

- La base de données Oracle et les procédures externes



Les CALL OUT de PL/SQL OBJET

La base de données Oracle et les procédures externes

- Les procédures stockées Oracle sont forcement écrites en : PL/SQL ou Java, ce code **s'exécute dans la SGA (S**ystem **G**lobal **A**rea**)** Oracle

- Toutefois il est possible qu'une procédure stockées PL/SQL invoque une procédure externe écrite en C, C++, Cobol, Pascal, …
  - ✓ Ce code, pour des raisons de sécurité s'exécute dans une **zone mémoire indépendante de la SGA** (extproc)

Les CALL OUT de PL/SQL OBJET

Mise en œuvre des procédures externes étape par étape

- **Etape 1** : Fixer les variables d'environnements
- **Etape 2** : Identifier la DLL contenant la procédure externe
- **Etape 3** : Publier la procedure externe
- **Etape 4** : Utiliser la procedure externe

Les CALL OUT de PL/SQL OBJET

Etape 1 : Fixer les variables d'environnements

- L'administrateur doit configurer la base pour exécuter les procédures externes écrites en C
- **Configurer l'agent extproc** dans les fichiers **listener.ora** et tnsnames.ora
- **Démarrer un listener EXCLUSIVEMENT** pour les procédures externes
- **Le listener doit préciser les variables d'environnement** tels que :
  - ✓ ORACLE_HOME,
  - ✓ ORACLE_SID,
  - ✓ LD_LIBRARY_PATH

Module M3.6, section 2 : PLSQL étendu aux objets complexes

➢ Les CALL OUT de PL/SQL OBJET

- **Etape 1 : Fixer les variables d'environnements**

Dans TNSNAMES.ORA

```
EXTPROC_CONNECTION_DATA =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST =
   localhost)(PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SID = PLSExtProc)
    )
  )
```

➢ Les CALL OUT de PL/SQL OBJET

- **Etape 1 : Fixer les variables d'environnements (suite)**

DANS LISTENER.ORA

```
  (SID_DESC =
    (SID_NAME = PLSExtProc)
    (ORACLE_HOME = C:\APP\ORACLE\db_1)
    (PROGRAM = extproc)
    (ENVS=
  "EXTPROC_DLLS=ONLY:
  'D:\Cours\oracleSql3\callBack\TestCallExternalC\TestCallExt
  ernalC.dll "
    )
   )
--version C++
'D:\Cours\oracleSql3\callBack\TestCallExternalCPP\TestCallEx
  ternalCPP.dll';
```

- NOTA : Il faut ensuite arrête et redémarrer le listener

Module M3.6, section 2 : PLSQL étendu aux objets complexes

➢Les CALL OUT de PL/SQL OBJET

- **Etape 2** : Identifier la DLL contenant la procédure externe

<u>Créer un objet library dans la base de données</u>

Sqlplus tpsql3/tpsql3@orcl

```
-- Avec une dll C
CREATE OR REPLACE LIBRARY  TestCallExternalC
  IS 'D:\Cours\oracleSql3\callBack\TestCallExternalC\TestCallExternalC.dll';

-- Avec une dll C++
CREATE OR REPLACE LIBRARY  TestCallExternalCPP
  IS 'D:\Cours\oracleSql3\callBack\TestCallExternalCPP\TestCallExternalCPP.dll’;
```

Nota : Il est utile de donner le chemin complet vers la librairie

Les CALL OUT de PL/SQL OBJET

- **Etape 3 : Publier la procedure externe**
  - **La procédure doit être publiée à travers une procédure PL/SQL qui indique le langage d'implémentation et la signature C:** int c_calcFactorial (int n)

```
CREATE OR REPLACE FUNCTION plsql_c_calcFactorial (n BINARY_INTEGER)
return BINARY_INTEGER
AS LANGUAGE C
NAME "c_calcFactorial"
LIBRARY TestCallExternalC;
/
```

Les CALL OUT de PL/SQL OBJET

Etape 4 : Utiliser la procédure externe

```
Sqlplus tpsql3/tpsql3@orcl

Set serveroutput on

Declare
    Fact        number:=0;
    i           number:=5;
Begin
  Fact:= plsql_c_calcFactorial(i);
  Dbms_output.put_line('Factoriel de '|| I || '= '|| fact);
End;
/
```

Factoriel de 5= 120
Procédure PL/SQL terminée avec succès.

Bulk SQL et Bulk binding

- Lors d'un **SELECT INTO, INSERT, UPDATE, DELETE** manipulant plusieurs lignes les données sont échangées LIGNE par LIGNE entre les moteurs PLSQL et SQL

- **Bulk SQL** permet d'augmenter les performances lors des échanges entre le moteur SQL et le moteur PLSQL en permettant le transferts des LIGNES en un seul ECHANGE

- Les instructions BULK SQL sont :
  - FORALL
  - BULK COLLECT
  - Une combinaison des deux

Bulk SQL et Bulk binding

FORALL

- Permet d'envoyer **de PLSQL vers SQL plusieurs Instructions SQL en mode Batch** (**en un seul bloc**). Au lieu d'une instruction à la fois
- Permet ainsi **d'augmenter** fortement **les performances en évitant des compilations multiples**

Module M3.6, section 2 : PLSQL étendu aux objets complexes

- **Bulk SQL et Bulk binding**
  - **FORALL**
    - **Exemple avec FOR .. LOOP**

```
DROP TABLE emp_temp;

CREATE TABLE emp_temp AS SELECT * FROM emp;

DECLARE
 TYPE NumList IS VARRAY(20) OF NUMBER;
 depts NumList := NumList(10, 20, 30); -- department numbers
BEGIN
 FOR i IN depts.FIRST..depts.LAST LOOP
 DELETE FROM emp_temp
 WHERE deptno = depts(i);
 END LOOP;
END;
/
```

NOTE : Trois DELETE sont envoyés un à un (**trois fois**). Ce qui est lent

Module M3.6, section 2 : PLSQL étendu aux objets complexes

- **Bulk SQL et Bulk binding**
  - **FORALL**
    - Exemple avec FOR ALL

```
DROP TABLE emp_temp2;

CREATE TABLE emp_temp2 AS SELECT * FROM emp;

DECLARE
    TYPE NumList IS VARRAY(20) OF NUMBER;
    depts NumList := NumList(10, 20, 30); -- department numbers
BEGIN
    FORALL i IN depts.FIRST..depts.LAST
            DELETE FROM emp_temp2
            WHERE deptno = depts(i);
END;
/
```

NOTE : **Trois DELETE sont envoyés en une seule fois**

Module M3.6, section 2 : PLSQL étendu aux objets complexes

Bulk SQL et Bulk binding

- **FORALL**
  - **Mesure de performance**

```
DROP TABLE emp_temp;
CREATE TABLE emp_temp AS SELECT * FROM emp;
DROP TABLE emp_temp2;
CREATE TABLE emp_temp2 AS SELECT * FROM emp;

DECLARE
   TYPE NumList IS VARRAY(20) OF NUMBER;
   depts NumList := NumList(10, 20, 30);
   t1 INTEGER; t2 INTEGER; t3 INTEGER;
BEGIN
    t1 := DBMS_UTILITY.get_time;
    FOR i IN depts.FIRST..depts.LAST LOOP
          DELETE FROM emp_temp WHERE deptno = depts(i);
   END LOOP;
```

```
t2 := DBMS_UTILITY.get_time;
FORALL i IN depts.FIRST..depts.LAST
     DELETE FROM emp_temp2 WHERE deptno = depts(i);
 t3 := DBMS_UTILITY.get_time;

DBMS_OUTPUT.PUT_LINE('Temps en Secondes');
DBMS_OUTPUT.PUT_LINE('FOR LOOP: ' || TO_CHAR((t2 - t1)/100));
 DBMS_OUTPUT.PUT_LINE('FORALL: ' || TO_CHAR((t3 - t2)/100));
END;
/
```

Temps en Secondes

FOR LOOP: ,01
FORALL: 0

PL/SQL procedure successfully completed.

Module M3.6, section 2 : PLSQL étendu aux objets complexes

➢ Bulk SQL et Bulk binding

▪ BULK COLLECT

- Permet **de renvoyer de SQL vers PLSQL** le résultat de l'execution d'une requête en **mode Batch (en un seul bloc).** Au lieu de plusieurs transferts
- Cette clause peut apparaître
  - ✓ **Dans un SELECT INTO :** SELECT … BULK COLLECT INTO …
  - ✓ **Dans un FETCH :** FETCH cursorsName BULK COLLECT …
  - ✓ **Dans la clause RETURNING** : en cas INSERT, UPDATE ou DELETE RETURNING … BULK COLLECT INTO …

Module M3.6, section 2 : PLSQL étendu aux objets complexes

➢ Bulk SQL et Bulk binding

- **BULK COLLECT**

✓ Exemple avec BULK COLLECT (plus rapide que les curseurs)

```
set serveroutput on

DECLARE
TYPE empnos_type IS TABLE OF emp.empno%TYPE;
TYPE enames_type IS TABLE OF emp.ename%TYPE;
bcempnos empnos_type ; bcenames enames_type ;
TYPE empSet IS TABLE OF emp%ROWTYPE;
bcemps empSet;
```

```
begin
    SELECT empno, ename
    BULK COLLECT  INTO bcempnos , bcenames
    FROM emp ORDER BY empno;

    dbms_output.put_line('bcempnos(1)='|| bcempnos(1));
    dbms_output.put_line('bcenames(1)='|| bcenames(1));

    SELECT *
    BULK COLLECT INTO bcemps
    FROM emp ORDER BY ename,hiredate;
    dbms_output.put_line('bcemps(1).ename=‘||
        bcemps(1).ename);
end;
/
```

Module M3.6, section 2 : PLSQL étendu aux objets complexes

Bulk SQL et Bulk binding

- **FORALL et BULK COLLECT**

```
DROP TABLE emp_temp3;
CREATE TABLE emp_temp3 AS SELECT * FROM emp;

Set serveroutput on
DECLARE
      TYPE deptnoList_t IS TABLE OF NUMBER;
      deptnos deptnoList_t := deptnoList_t(10,20,30);
      TYPE empsDeleted_t IS TABLE OF emp%ROWTYPE;
      empsDeleted empsDeleted_t ;
BEGIN
      FORALL j IN deptnos.FIRST..deptnos.LAST
      DELETE FROM emp_temp3
      WHERE deptno = deptnos(j)
      RETURNING EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO
      BULK COLLECT INTO empsDeleted;
      DBMS_OUTPUT.PUT_LINE ('Deleted ' || SQL%ROWCOUNT || ' rows:');
     DBMS_OUTPUT.PUT_LINE('empsDeleted(1).ename='||empsDeleted(1).ename );
END;
/
```

Module M3.6, section 2 : QUIZ

➢ **Question 1 : Cochez la ou les bonnes définition d'un CALL OUT**

A: Un CALL OUT est la possibilité qui existe sous oracle d'importer des programmes écris dans des L3G tels que C, C++, …

B: Un CALL OUT est la possibilité qui existe sous oracle d'exporter des programmes écrits dans des L3G tels que C, C++, …

C: Un CALL OUT est la possibilité qui existe sous oracle d'exécuter des programmes écrits dans des L3G tels que C, C++, …

D: A, B sont faut

➢ **Question 2 : Cochez ce qui est juste**

A: Le code L3G d'un CALL OUT s'exécute dans la SGA Oracle

B: Le code L3G d'un CALL OUT s'exécute dans une SGA spéciale appelée EXTPROC

C: Le code L3G d'un CALL OUT s'exécute dans une SGA spéciale appelée PROCEDURE EXTERNE

D: Il faut indiquer le nom de la library contenant le code L3G (C, C++, …) dans le fichier LISTENER.ORA

E: La sécurité du SGBD est la raison majeure de la séparation des espaces d'exécution du code L3G

F: Seul le code PLSQL et/ou JAVA peuvent s'exécuter dans la SGA Oracle

Module M3.6, section 2 : QUIZ

➢**Question 3 :**

Supposons une librairie C contenant une fonction appelée "c_calcFactorial"

```
CREATE OR REPLACE FUNCTION plsql_c_calcFactorial (n BINARY_INTEGER)
return BINARY_INTEGER
AS LANGUAGE C
NAME "c_calcFactorial"
LIBRARY TestCallExternalC;
/
```

Cochez ce qui est juste

A: "c_calcFactorial" est le nom de la function contenu dans la librairie C

B: plsql_c_calcFactorial est le nom de la function contenu dans la librairie C

C: plsql_c_calcFactorial est le nom de publication de la function "**c_calcFactorial**" dans PLSQL

D: Il est POSSIBLE d'appeler directement "**c_calcFactorial"** dans des programmes PLSQL

E: Il est IMPOSSIBLE d'appeler directement "**c_calcFactorial"** dans des programmes PLSQL, il faut passer par **plsql_c_calcFactorial**

Module M3.6, section 2 : QUIZ

➢ **Question 4 : Que fait FORALL**

A: Permet d'envoyer de PLSQL vers SQL plusieurs Instructions SQL en mode Batch (en un seul bloc)

B: Permet d'augmenter fortement les performances

C: Les requêtes peuvent être : SELECT, UPDATE, INSERT, DELETE

D: Les requêtes peuvent être : CREATE .., ALTER …, DROP …

E: : Les requêtes peuvent être : UPDATE, INSERT, DELETE

F: FORALL est plus rapide que FOR

G: Il s'agit de requête dont la seule différence est l'indice de la boucle

H: Avec FORALL on compile une fois et on exécute plusieurs fois

Module M3.6, section 2 : QUIZ

➢ **Question 5 : Que fait BULK COLLECT**

A: Permet d'éviter l'utilisation des CURSEURS

B: Permet d'augmenter fortement les performances

C: Les requêtes peuvent être : SELECT, UPDATE, INSERT, DELETE

D: Les requêtes peuvent être : CREATE .., ALTER …, DROP …

E: : Les requêtes peuvent être : UPDATE, INSERT, DELETE

F: **BULK COLLECT** est plus rapide que d'utiser les curseurs

G: **BULK COLLECT** construit dynamiquement une collection

H: Peut être utilisé en combinaison avec FORALL

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

Module M3.6, section 3, partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Plan

- **Les différentes approches pour accéder aux objets complexes d'une base Oracle depuis Java**
- **Architecture des drivers Oracle JDBC et le SGBD Oracle**
- **Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées partie 1**
- *Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées partie 2*
- *Mapping des collections de type VARRAY*
- *Mapping des références (REF)*
- *Mapping des collections de type Nested table*
- *Mapping des Objets volumineux (LOB)*

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3, partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

- **Les différentes approches pour accéder aux objets complexes d'une base Oracle depuis Java**
  - Via Jdeveloper
  - Via SQLJ
  - Via TOPLINK
  - JDBC
- Note : nous présentons dans cette section uniquement l'intégration via JDBC

anr INVESTIR L'AVENIR

CC BY NC ND

DATUM ACADEMY

Module M3.6, section 3, partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Les différents types de drivers JDBC

- **Pilotes de type 1:** Pilotes accédant à une base de données par l'intermédiaire d'une autre technologie (**on parle de passerelle**). **Exemple, les passerelles JDBC-ODBC**

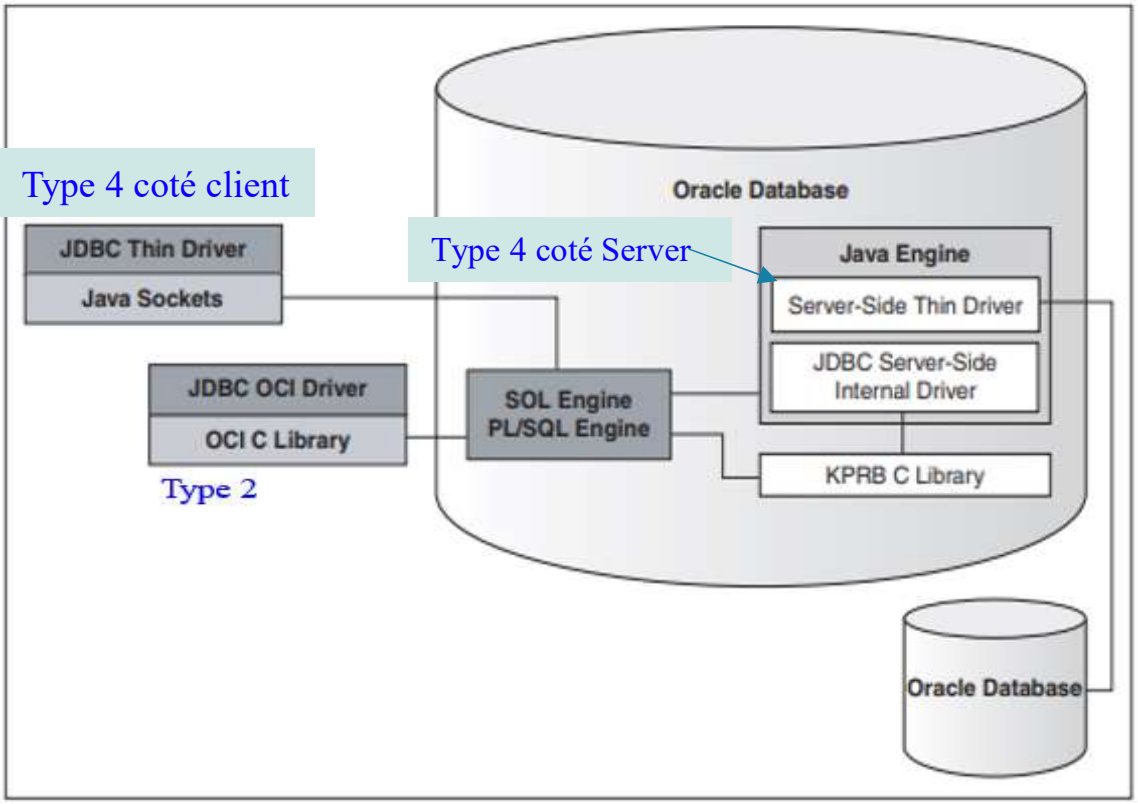
- **Pilotes de type 2:** Il s'agit de pilote JDBC invoquant les méthodes natives (JNI) du SGBD(API écrites en un autre langage que Java. Exemple: C, C++, etc.)

- **Pilotes de type 3:** Pilotes convertissant les appels JDBC en un protocole indépendant du SGBD. Un serveur convertit ensuite ceux-ci dans le protocole du SGBD requis (modèle à 3 couches, couches réseaux)

- **Pilotes de type 4:** Pilotes convertissant **les appels JDBC directement en un protocole réseau** (socket) exploité par le SGBD (exemple : SQLNET pour Oracle). **Il s'agit d'un Driver full Java**

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

➢Architecture des drivers Oracle JDBC et le SGBD Oracle



Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

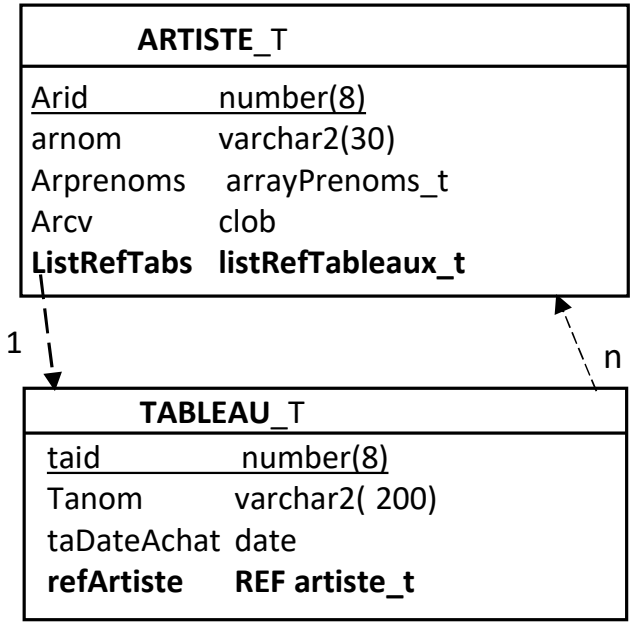
- Cette approche permet de faire un mapping 1:1 entre un type Oracle et une classe Java de même structure

- Cette approche nécessite de suivre les étapes suivantes :
  - **Etape 1 : Définir vos types objets** dans la base de données Oracle
  - **Etape 2 : Créer dans la base des tables** objets ou relationnelles utilisant ces types
  - **Etape 3 : Créer sous java des classes** correspondantes aux types définis dans l'étape 1
  - **Etape 4 : Lier les types Oracle aux classes Java** correspondantes et charger les objets de la base vers les classes java

Module M3.6, section 3, partie 1 : Mapping Objet relationnel dans Java/JDBC

➢ **Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées**

- **Etape 1 : Définir vos types objets dans la base de données Oracle**
  - Considérons le schéma des types ci-contre



listRefTableaux_t : Type pour gérer la collection de REF vers les tableaux de chaque artiste.

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

▪ Etape 1 : Définir vos types objets dans la base de données Oracle

```
-- Suppression des tables et des types
Drop table o_tableau cascade constraints;
Drop table o_artiste cascade constraints;
Drop type tableau_t force;
Drop type artiste_t force;
Drop type arrayPrenoms_t force;
Drop type listRefTableaux_t ;

-- Déclaration Forward des Types afin d'éviter l'interblocage dû au referencement mutuel entre types
CREATE OR REPLACE TYPE artiste_T
/
--Création du type varray (arrayPrenoms_t) pour stocker les prénoms
Create or replace type arrayPrenoms_t as varray(4) of varchar2(30)
/
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 1 : Définir vos types objets dans la base de données Oracle**

```
-- Création du type tableau_t
CREATE OR REPLACE type tableau_t AS OBJECT(
   taid              number(8),
   tanom             varchar2(200),
   tadateAchat       date,
   refArtiste        REF artiste_t)
/

-- Création du type table (listRefTableaux_t) : Les
-- éléments de cette liste sont des références vers des
-- tableaux.
CREATE OR REPLACE TYPE listRefTableaux_t  AS TABLE
   OF REF tableau_t
/
```

```
-- Création du type artiste_t
CREATE OR REPLACE TYPE artiste_t AS OBJECT(
   arid              number(8),
   arnom             varchar2(200),
   arprenoms         arrayPrenoms_t,
   arcv              clob,
   listRefTabs       listRefTableaux_t)
/
```

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR
anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 2 : Créer dans la base des tables objets ou relationnelles utilisant ces types**

```
-- Création de la table objet o_tableau
CREATE TABLE O_tableau OF tableau_t( Constraint pk_o_tableau_taid PRIMARY KEY(taid))
/

-- Création de la table o_artiste
CREATE TABLE O_artiste OF artiste_t( Constraint pk_o_artiste_arid PRIMARY KEY(arid))
NESTED TABLE listRefTabs STORE AS storeListRefTabs;
/
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 2 : Créer dans la base des tables objets ou relationnelles utilisant ces types**

```
-- INSERTION DES LIGNES
DECLARE
   refArt1        REF Artiste_t;
   refArt2        REF Artiste_t;
   refTab1        REF Tableau_t;
   refTab2        REF Tableau_t;
   refTab3        REF Tableau_t;
BEGIN
--  1: Insertion des artistes (liste REF tableaux vide)
INSERT INTO o_artiste oa
VALUES(1, 'VAUDIER',ARRAYPRENOMS_T('Benjamin',
   'Ben'), 'CV BEN', LISTREFTABLEAUX_T() )
RETURNING REF(oa) INTO refArt1;
```

```
INSERT INTO o_artiste oa
VALUES(2, 'CEZANNE',ARRAYPRENOMS_T('Paul'), 'CV CEZANNE',
LISTREFTABLEAUX_T() ) RETURNING REF(oa) INTO refArt2;

-- 2: Insertion des tableaux avec maj de la Ref vers l'artiste
INSERT INTO o_tableau ot VALUES(1, 'Faire le contraire',
to_date('10-10-2012', 'DD-MM-YYYY'),refArt1 )
RETURNING REF(ot) INTO refTab1;

INSERT INTO o_tableau ot VALUES(2, 'Les joueurs de cartes',
to_date('31-12-1901', 'DD-MM-YYYY'),refArt2)
RETURNING REF(ot) INTO refTab2;
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 2: Créer dans la base des tables objets ou relationnelles utilisant ces types**

```
-- 2: Insertion des tableaux avec maj de la Ref vers l'artiste (suite)
INSERT INTO o_tableau ot values(3, 'Le garçon au Gilet rouge',
to_date('10-10-1900', 'DD-MM-YYYY'),refArt2) returning ref(ot) into refTab3;

-- 3: Mise à jour de la liste des références vers les artistes
INSERT INTO TABLE(select oa.listRefTabs  from o_artiste oa where oa.arid=1) VALUES (refTab1);
INSERT INTO TABLE(select oa.listRefTabs from o_artiste oa where oa.arid=2) VALUES (refTab2);
INSERT INTO TABLE(select oa.listRefTabs  from o_artiste oa where oa.arid=2) VALUES (refTab3);
END;
/
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Artiste.java
// Cette classe doit implémenter les méthodes de l'interface Sqldata
package galerie;

import java.io.BufferedReader;
import java.io.IOException;
import java.sql.*;
import oracle.sql.ARRAY;

public class Artiste implements SQLData{
   private String sql_type ;
   private int arid;
```

```
private String arnom;
private ARRAY arprenoms;
private Clob arcv;
private ARRAY listRefTabs;
public Artiste (){}

public Artiste(String sql_type, int arid,String arnom, ARRAY
arprenoms, Clob arcv, ARRAY listRefTabs ){
                this.sql_type=sql_type;
                this.arid=arid ;
                this.arnom=arnom;
                this.arprenoms=arprenoms ;
        this.arcv=arcv;
       this.listRefTabs=listRefTabs;   }
```

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Artiste.java Cette classe doit implémenter les méthodes de
// l'interface Sqldata (suite)

public int getArid() { return arid; }
public void setArid(int arid) {this.arid = arid; }
public String getArnom() {return arnom;}
public void setArnom(String arnom) { this.arnom = arnom;}
public Clob getArcv() {return arcv;}
public void setArcv(Clob arcv) {this.arcv = arcv;}
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Artiste.java Cette classe doit implémenter les méthodes de
//'interface Sqldata (suite)

public void setArprenoms(ARRAY arprenoms) {this.arprenoms = arprenoms;}
public ARRAY getArprenoms() {return arprenoms;}
public ARRAY getListRefTabs() {return listRefTabs;}
public void setListRefTabs(ARRAY listRefTabs) {this.listRefTabs = listRefTabs;}
public String getSQLTypeName()throws SQLException{return sql_type;}
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

- **Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées**
  - **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Artiste.java. Cette classe doit implémenter les méthodes de l'interface Sqldata
// Lire dans le flot dans l'ordre.
    public void readSQL(SQLInput stream, String typeName) throws SQLException{
        sql_type=typeName;
        arid=stream.readInt();
        arnom=stream.readString();
        arprenoms=(ARRAY) stream.readArray();
        arcv=stream.readClob();
        listRefTabs=(ARRAY) stream.readArray();
        System.out.println("") ;
    }
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

- **Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées**
  - **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Artiste.java. Cette classe doit implémenter les méthodes de l'interface Sqldata
//Ecrire dans le flot dans l'ordre.
   public void writeSQL(SQLOutput stream) throws SQLException{
          stream.writeInt(arid);
          stream.writeString(arnom);
          stream.writeArray(arprenoms);
          stream.writeClob(arcv);
          stream.writeArray(listRefTabs);
   }
```

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

- **Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées**
  - **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

// Création de la classe Java Artiste.java. Cette classe doit implémenter les méthodes de l'interface Sqldata
//Afficher les inforrmation sur un artiste.

```
public void display() throws SQLException, IOException{
    System.out.println("");
    System.out.println("{");
    System.out.println("arid = "+this.getArid());
    System.out.println("arnom = "+this.getArnom());
    this.displayArprenoms();
    this.displayArcv();
    this.displayInfoTableauxArtiste();
    System.out.println("}");
    System.out.println("") ; }
```

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

- **Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées**
  - **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

// Création de la classe Java Artiste.java. Cette classe doit implémenter les méthodes de l'interface Sqldata
//Afficher les inforrmation sur les tableaux d'un artiste.

```
public void displayInfoTableauxArtiste() throws SQLException, IOException{
      // affichage des infos sur les tableaux
    Ref [] lesRefDesTableaux= (Ref[])this.getListRefTabs().getArray();
      System.out.println("<Tableaux:");
     for (Ref refTableau : lesRefDesTableaux) {
         Tableau tablo = (Tableau) refTableau.getObject();
         //tablo.display();
         System.out.println("   [taid="+tablo.getTaid()+" tanom="+tablo.getTanom()+"]");
      }
      System.out.println(">");
   }
```

Module M3.6, section 3 , partie 1 : Mapping Objet relationnel dans Java/JDBC

- **Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées**
  - **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
-- Création de la classe Java Artiste.java. Cette classe doit implémenter les méthodes de l'interface Sqldata
//Afficher les inforrmation sur le prénoms d'un artiste.
 public void displayArprenoms() throws SQLException{
      // affichage des prénoms
       String [] lesArprenoms= ( String [])this.getArprenoms().getArray();
       for (int i=0; i<lesArprenoms.length;i++){
         System.out.println("Prenoms["+i+"]="+lesArprenoms[i]);
       }

     }
}
```

Module M3.6, section 3 , partie 1 : QUIZ

➢ **Question 1 : Cochez ce qui correspond à un driver JDBC**

A: Un driver JDBC est un composant logiciel permettant à une application Java d'interagir avec une base de données

B: Il s'agit d'une API standard que doit implémenter chaque éditeur de SGBD

C: Les drivers JDBC sont analogues aux drivers ODBC

D: La traduction de «Driver» en français est «Pilote»

E: Le standard est respecté à 100% par tous les éditeurs de SGBD

F: Il existe 5 types de drivers JDBC

➢ **Question 2 : Cochez les types de drivers JDBC supportées par Oracle**

A: Type 1

B: Type 2

C: Type 3

D: Type 4

Module M3.6, section 3 , partie 1 : QUIZ

➢ **Question 3 : Cochez ce qui correspond à un driver JDBC de type 4**

A: Les drivers JDBC de type 4 sont des drivers JDBC FULL Java

B: Ils utilisent le middle ware réseau et les sockets

C: Ils peuvent exister entre deux serveurs du SGBD

D: Aucune des affirmations précédentes n'est juste

➢ **Question 4 : Cochez ce qui correspond à un driver JDBC de type 2**

A: Ce sont des drivers JDBC **NON** FULL Java

B: Il s'agit de pilote JDBC invoquant les méthodes natives (JNI) du SGBD écrites dans un autre langage

C: Ils existent toujours et uniquement entre deux serveurs du SGBD

D: Aucune des affirmations précédentes n'est juste

Module M3.6, section 3 , partie 1 : QUIZ

➢**Question 5 : Cochez ce qui caractérise au mieux une classe spécialisée**

A: Une classe spécialisée doit avoir obligatoirement le même nom que le type Oracle correspondant

B: Une classe spécialisée peut avoir un nom quelconque. Il existe des mécanismes pour le faire correspondre au type Oracle

C: Le nombre maximum de champs et de méthodes de cette classe est limité à 1000

D:L'affirmation faite en C est fausse

E: Elle doit posséder obligatoirement un champ appelé sql_type contenant le nom du type équivalent Oracle

➢**Question 6 : Cochez le contrat idéal pour une classe spécialisée**

A: Doit avoir des méthodes de manipulation des CLOB, BLOB, BFILE

B: Doit avoir des méthodes de manipulation des types VARRAY et NESTED TABLE

C: Doit avoir des getters et des setters

D: Dois avoir des méthodes de manipulation des REF

E: Doit implémenter les méthodes de l'interface Sqldata (readSQL et writeSQL)

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY

UNIVERSITÉ CÔTE D'AZUR

Plan

- *Les différentes approches pour accéder aux objets complexes d'une base Oracle depuis Java*
- *Architecture des drivers Oracle JDBC et le SGBD Oracle*
- *Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées partie 1*
- **Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées partie 2**
- **Mapping des collections de type VARRAY**
- **Mapping des références (REF)**
- **Mapping des collections de type Nested table**
- **Mapping des Objets volumineux (LOB)**

anr INVESTIR L'AVENIR

CC BY NC ND

DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

➢ Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Tableau.java Cette classe
// doit implémenter les méthodes de l'interface Sqldata
package galerie;
//import static com.oracle.jrockit.jfr.DataType.ARRAY;
import java.sql.*;
import oracle.sql.ARRAY;
import oracle.sql.REF;
import java.io.BufferedReader;

public class Tableau implements SQLData{
        private String sql_type ;
        private int taid;
        private String tanom;
```

```
private Date tadateAchat;
    private Ref refArtiste;
        public Tableau (){}
  public Tableau(String sql_type,int taid,String tanom,
  Date tadateAchat, REF refArtiste ){
        this.sql_type=sql_type;
        this.taid=taid ;
        this.tanom=tanom;
        this.tadateAchat=tadateAchat;
         this.refArtiste=refArtiste;
  }
```

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Tableau.java Cette classe
// doit implémenter les méthodes de l'interface Sqldata

public String getSQLTypeName()throws SQLException{
                    return getSql_type();
  }
public void readSQL(SQLInput stream, String typeName) throws
  SQLException{
          setSql_type(typeName);
          taid=stream.readInt();
          tanom=stream.readString();
          tadateAchat=stream.readDate();
         setRefArtiste((Ref) stream.readRef());
  }
```

```
public void writeSQL(SQLOutput stream) throws
  SQLException{
          stream.writeInt(taid);
          stream.writeString(tanom);
          stream.writeDate(tadateAchat);
          stream.writeRef(getRefArtiste());
  }

public int getTaid() {
      return taid;
  }
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Tableau.java.Cette classe doit implémenter les méthodes de
// l'interfaceSqldata
public void setTaid(int taid) {this.taid = taid;}
public String getTanom() {return tanom;}
public void setTanom(String tanom) {this.tanom = tanom;}
public Date getTadateAchat() {return tadateAchat;}
public void setTadateAchat(Date tadateAchat) {this.tadateAchat = tadateAchat;}
public Ref getRefArtiste() {return refArtiste;}
public void setRefArtiste(Ref refArtiste) {this.refArtiste = refArtiste;}
public String getSql_type() {return sql_type;}
public void setSql_type(String sql_type) {this.sql_type = sql_type;}
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

➢Mapping des objets complexes Oracle avec Java/JDBC avec des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Tableau.java.Cette classe doit implémenter les méthodes de
// l'interfaceSqldata
public void display() throws SQLException, java.io.IOException{
    System.out.println("");
    System.out.println("{");
     System.out.println("taid = "+this.getTaid());
     System.out.println("tanom = "+this.getTanom());
      System.out.println("tadateAchat = "+this.getTadateAchat());
      this.displayInfoEmpDeptFromRef();
      System.out.println("}");
      System.out.println("");
    }
```

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées

- **Etape 3 : Créer sous java des classes correspondantes aux types définis dans l'étape 1**

```
// Création de la classe Java Tableau.java.Cette classe doit implémenter les méthodes de
// l'interfaceSqldata

// Affichage des informations sur l'artiste ayant peint le tableau
public void displayInfoArtisteFromRef() throws SQLException{
        Ref refArtiste1 = this.getRefArtiste();
        Artiste art1 = (Artiste) refArtiste1.getObject();
        System.out.println("Nom du peintre = "+art1.getArnom());
      }
}
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées

- **Etape 4 : Lier les types Oracle aux classes Java correspondantes et charger les objets de la base vers les classes java**

```
// Création de la classe Java Galerie.java
// Cette classe charge le driver jdbc et fait mapping entre
// les types Oracle et les classes Java équivalentes
package galerie;
import java.io.IOException;
import java.sql.*;
public class Galerie{
public static void main(String[] argv) throws SQLException,
   ClassNotFoundException{ try {
   Class.forName("oracle.jdbc.driver.OracleDriver" );
   Connection conn = DriverManager.getConnection(
   "jdbc:oracle:thin:@144.21.67.201:1521/PDBBDS1.631174089.orac
   lecloud.internal", "MOPOLO", "xxxxx");

   Statement stmt = conn.createStatement();
   }
```

```
java.util.Map mapOraObjType =
  conn.getTypeMap();

// Le nom du type abstrait Oracle doit être précédé
// du nom de son propriétaire. Voilà le mapping objet
// relationnel.
  mapOraObjType.put((Object)"MOPOLO.TABLEAU
  _T",(Object)Class.forName("galerie.Tableau" ));

  mapOraObjType.put((Object)"MOPOLO.ARTISTE_
  T"(Object)Class.forName("galerie.Artiste" ));
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées

- **Etape 4 : Lier les types Oracle aux classes Java correspondantes et charger les objets de la base vers les classes java**

```
// classe Java Galerie.java

// Requête SQL à exécuter
   String sqlTableau = "SELECT value(ot) FROM o_tableau ot";

   // Exécution de la requête
   ResultSet resultsetTableau = stmt.executeQuery(sqlTableau);

   //Affichage des informations sur les tableaux

   System.out.println("********INFOS TABLEAUX *****************");
   while(resultsetTableau.next()){
         //Récupération et affichage d'un tableau
         //System.out.println(resultsetEmp.getObject(1));
         Tableau tablo = (Tableau)resultsetTableau.getObject(1, mapOraObjType);
         tablo.display();
   }
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées

- **Etape 4 : Lier les types Oracle aux classes Java correspondantes et charger les objets de la base vers les classes java**

```
// classe Java Galerie.java

// accès aux objets Artiste
   String sqlArt = "SELECT value(oa) FROM o_artiste oa";
   ResultSet resultsetArt = stmt.executeQuery(sqlArt);

   System.out.println("*********INFOS ARTISTES ****************");

 //Récupération et affichage d'un artiste
   while(resultsetArt.next()){
            Artiste art = (Artiste)resultsetArt.getObject(1, mapOraObjType);
            art.display();
  }
}
catch(ClassNotFoundException | SQLException | IOException e){
   System.out.println("Echec du mapping");
   e.printStackTrace();}}}
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢ Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées

- **Etape 4 : Lier les types Oracle aux classes Java correspondantes et charger les objets de la base vers les classes java**

-- Résultats

```
*******************INFOS TABLEAUX **********************
{
taid = 1
tanom = Faire le contraire
tadateAchat = 2012-10-10
Nom du peintre = VAUDIER
}
{
taid = 2
tanom = Les joueurs de cartes
tadateAchat = 1901-12-31
Nom du peintre = CEZANNE
}
```

-- Résultats

```
*******************INFOS TABLEAUX *****************
{
taid = 3
tanom = Le gar‡on au Gilet rouge
tadateAchat = 1900-10-10
Nom du peintre = CEZANNE
}
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢Mapping des objets complexes Oracle avec Java/JDBC et des classes spécialisées

➢Etape 4 : Lier les types Oracle aux classes Java correspondantes et charger les objets de la base vers les classes java

-- Résultats

```
**************INFOS ARTISTES **************************
{
arid = 1
arnom = VAUDIER
Prenoms[0]=Benjamin
Prenoms[1]=Ben
[ <CV/
  CV BEN
 /CV>]

<Tableaux:
  [taid=1 tanom=Faire le contraire]
>}
```

-- Résultats

```
**************INFOS ARTISTES **************************
{
arid = 2
arnom = CEZANNE
Prenoms[0]=Paul

[ <CV/
  CV CEZANNE
 /CV>]
<Tableaux:
  [taid=2 tanom=Les joueurs de cartes]
  [taid=3 tanom=Le gar‡on au Gilet rouge]
>}
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

➢Mapping des collections de type VARRAY

- Il s'agit de montrer comment accéder aux objets d'une collection de type VARRAY depuis Java.

```
//1.  Il faut import la classe jdbc Oracle appelée ARRAY
import oracle.sql.ARRAY;

// 2. Il faut  dans une cl asse : (1) définir un champ Varray, (2) Lire et
// (3)  écrire le varray dans le flot
public class Artiste implements SQLData{
// Définir champ privé de type ARRAY arprenoms
   private ARRAY arprenoms;
//Lire le VARRAY arprenoms dans le flot venant de la base
  public void readSQL(SQLInput stream, String typeName) throws
SQLException{
          arprenoms=(ARRAY) stream.readArray();
...}
```

```
//Ecrire le VARRAY arprenom vers le flot à envoyer dans
// la base
   public void writeSQL(SQLOutput stream) throws
SQLException{
          ...
          stream.writeArray(arprenoms);
          ...
}
```

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

➢Mapping des collections de type VARRAY

```
…
// Affichage des éléments du Varray arprenoms
 public void displayArprenoms() throws SQLException{

      // affichage des prénoms
       String [] lesArprenoms= (String[])this.getArprenoms().getArray();

       for (int i=0; i<lesArprenoms.length;i++){
          System.out.println("Prenoms["+i+"]="+lesArprenoms[i]);
       }

      }

}
```

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Mapping des références (REF)

- Il s'agit de montrer comment accéder depuis Java aux à l'instance d'un objet Oracle connaissant son REF

```
// 1. Il faut importer la classe jdbc Oracle appelée REF
import oracle.sql.REF;
…
// 2. Il faut  dans une cl asse : (1) définir un champ REF, (2) Lire et
// (3)  écrire le REF dans le flot

public class Tableau implements SQLData{
private Ref refArtiste;

//Lire le REF refArtiste dans le flot venant de la base
public void readSQL(SQLInput stream, String typeName) throws
SQLException{
        setRefArtiste((Ref) stream.readRef());
 }
```

```
//Ecrire le REF refArtiste vers le flot à envoyer dans la base

public void writeSQL(SQLOutput stream) throws
SQLException{
…
        stream.writeRef(getRefArtiste());
…}
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

➢Mapping des collections de type REF

```
// Affichage des Infos sur un artiste connaissant son REF
public void displayInfoArtisteFromRef() throws SQLException{

        Ref refArtiste1 = this.getRefArtiste();
        Artiste art1 = (Artiste) refArtiste1.getObject();
        System.out.println("Nom du peintre = "+art1.getArnom());
    }

}
```

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Mapping des collections de type Nested table

- Il s'agit de montrer comment accéder depuis Java aux éléments d'une nested table Oracle

```
// 1. Il faut importer la classe jdbc Oracle appelée ARRAY et REF
import oracle.sql.ARRAY;
import oracle.sql.REF;
…
// 2. Il faut  dans une cl asse : (1) définir un champ ARRAY,
// (2) Ecrire une méthode readSQL (3) Ecrire une méthode writeSQL
public class Artiste implements SQLData{
-- (1) Définir champ privé de type ARRAY arprenoms
   private ARRAY listRefTabs;
//(2) Lire la nested table listRefTabs dans le flot venant de la base
  public void readSQL(SQLInput stream, String typeName) throws
SQLException{…
                listRefTabs =(ARRAY) stream.readArray();
…}}
```

```
// (3) Ecrire la nested table listRefTabs vers le flot à
// envoyer à la base
   public void writeSQL(SQLOutput stream) throws
SQLException{
…
                stream.writeArray(listRefTabs);
…}
```

anr INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

➢Mapping des collections de type Nested table

```
…
// Affichage des éléments de la nested Table listRefTabs
  public void displayInfoTableauxArtiste() throws SQLException, IOException{
        // affichage des infos sur les tableaux
        Ref [] lesRefDesTableaux= (Ref[])this.getListRefTabs().getArray();
        System.out.println("<Tableaux:");
        for (Ref refTableau : lesRefDesTableaux) {
              Tableau tablo = (Tableau) refTableau.getObject();
              //tablo.display();
              System.out.println("   [taid="+tablo.getTaid()+" tanom="+tablo.getTanom()+"]"); }
              System.out.println(">");
        }
…
}
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

ESTIA INSTITUTE OF TECHNOLOGY
UNIVERSITÉ CÔTE D'AZUR

Mapping des Objets volumineux (LOB)

- Il s'agit de montrer comment accéder depuis Java aux à un objet volumineux de type CLOB

```
// 1. Il faut importer la classe jdbc Oracle Clob
import java.sql.Clob;
// 2. Il faut  dans une cl asse : (1) définir un champ Clob,
// (2) Ecrire une méthode readSQL (3) Ecrire une méthode writeSQL
public class Artiste implements SQLData{
        // (1) Définir champ privé de type Clob arcv
        private Clob arcv;

        //(2) Lire le Clob dans le flot venant de la base
        public void readSQL(SQLInput stream, String typeName) throws
        SQLException{…
                arcv=stream.readClob();
…}
```

```
//(3)Ecrire le Clob arcv  vers le flot à envoyer à la base
   public void writeSQL(SQLOutput stream) throws
SQLException{
        …
        stream.writeArray(arcv);
…}
```

anr
INVESTIR L'AVENIR
CC BY NC ND
DATUM ACADEMY

Module M3.6, section 3 , partie 2 : Mapping Objet relationnel dans Java/JDBC

➢Mapping des Objets volumineux (LOB)

```
…
// Affichage du CV d'un artiste
public void displayArcv() throws java.sql.SQLException, java.io.IOException{
        BufferedReader clobReader = new
         BufferedReader(this.getArcv().getCharacterStream());
        String ligne;
        System.out.println("");
        System.out.println("[ <CV/ ");
        while((ligne = clobReader.readLine()) != null){
           System.out.println("   "+ligne);
        }
        System.out.println(" /CV>] ");
        System.out.println("");
    }
```

Module M3.6, section 3 , partie 2 : QUIZ

- **Question 1 : Cochez ce qui est vrai par rapport à cette séquence de code**

```
Class.forName("oracle.jdbc.driver.OracleDriver" );
Connection conn = DriverManager.getConnection(
    "jdbc:oracle:thin:@144.21.67.201:1600/PDBBDS1.631174089.oraclecloud.internal", "LoginOracle", "XXX");
```

A: Class.forName charge depuis le cloud Oracle le driver JDBC

B: Class.forName charge le driver JDBC Oracle depuis un jar

C: Le driver utilisé est le driver JDBC de type 4

D: Le driver utilisé est le driver JDBC de type 2

- **Question 2 : : Cochez ce qui est vrai par rapport à cette séquence de code**

```
java.util.Map mapOraObjType = conn.getTypeMap();
mapOraObjType.put((Object)« loginOracle.TABLEAU_T",(Object)Class.forName("galerie.Tableau" ));
mapOraObjType.put((Object)" loginOracle.ARTISTE_T",(Object)Class.forName("galerie.Artiste" ));
```

A: mapOraObjType.put permet de créer le type ARTISTE_T

B: mapOraObjType.put permet de faire un mapping entre un type Oracle et la classe java spécialisée correspondante

C: loginOracle est le propriétaire du type dans la base Oracle

D: La classe spécialisée java se trouve dans le package « galerie »

Module M3.6, section 3 , partie 2 : QUIZ

➢ **Question 3 : Cochez ce qui est vrai par rapport à cette séquence de code**

```
Statement stmt = conn.createStatement();
String sqlTableau = "SELECT value(ot) FROM o_tableau ot";
ResultSet resultsetTableau = stmt.executeQuery(sqlTableau);
```

A: « stmt » contient les ressources nécessaires pour envoyer des requêtes au moteur Oracle

B: stmt.executeQuery compile la requête contenu dans la variable sqlTableau et l'envoie pour exécution à Oracle

C: La compilation et l'exécution se fait au niveau du moteur Oracle

D: Les lignes résultantes se trouvent dans la l'objet « resultsetTableau »

➢ **Question 4 : Cochez ce qui est vrai par rapport à cette séquence de code**

```
while(resultsetTableau.next()){
    Tableau tablo = (Tableau)resultsetTableau.getObject(1, mapOraObjType);
    tablo.display();
}
```

A: Le chiffre 1 correspond au nombre d'objet à extraire

B: Le chiffre 1 correspond au numéro de la colonne à extraire

C: *mapOraObjType permet d'aider à la conversion d'une instance du type en une instance de la classe spécialisé*

D: Le casting en tableau réussit toujours si C estvrai

Module M3.6, section 3 , partie 2 : QUIZ

➢ **Question 5 : Cochez ce qui est vrai par rapport à cette séquence de code**

```
public Ref getRefArtiste() {
    return refArtiste;
}
public void displayInfoEmpDeptFromRef() throws SQLException{
    Ref refArtiste1 = this.getRefArtiste();
    Artiste art1 = (Artiste) refArtiste1.getObject();
    System.out.println("Nom du peintre = "+art1.getArnom());
 }
```

A: Ref est la classe Java qui permet de définir un pointeur vers un objet venant de la base

B: La méthode getObject() renvoie une instance de l'objet pointée

C: Sans casting il n'est pas possible de récupérer une instance de Artiste

D: Ce qui est écrit en A est faut

E: getRefArtiste est un accesseur qui renvoie la référence

F: La référence affectée à refArtiste1 vient de la base de données

Module M3.6, section 3 , partie 2 : QUIZ

➢ **Question 6 : Cochez ce qui est vrai par rapport à cette séquence de code**

```
public void displayArprenoms() throws SQLException{
    String [] lesArprenoms= ( String [])this.getArprenoms().getArray();
    for (int i=0; i<lesArprenoms.length;i++){
      System.out.println("Prenoms["+i+"]="+lesArprenoms[i]);
    }
}
```

A: la fonction «displayArprenoms» affiche les informations sur une collection de type Varray

B : la fonction «displayArprenoms» affiche les informations sur une collection de type Nested Table

C: «lesArprenoms[i] » contient un prénom

D: « lesArprenoms » c'est un tableau de chaines de caractères

Module M3.6, section 3 , partie 2 : QUIZ

➢ **Question 7 : Cochez ce qui est vrai par rapport à cette séquence de code**

```
public void displayInfoTableauxArtiste() throws SQLException, IOException{
    Ref [] lesRefDesTableaux= (Ref[])this.getListRefTabs().getArray();
    System.out.println("<Tableaux:");
    for (Ref refTableau : lesRefDesTableaux) {
        Tableau tablo = (Tableau) refTableau.getObject();
        System.out.println("   [taid="+tablo.getTaid()+" tanom="+tablo.getTanom()+"]");
    }
    System.out.println(">");
}
```

A: La fonction «displayInfoTableauxArtiste» affiche les informations sur une collection de type Varray

B : La fonction «displayInfoTableauxArtiste» affiche les informations sur une collection de type Nested Table

C: La fonction «displayInfoTableauxArtiste» affiche les informations sur les tableaux d'un artiste

D: « lesRefDesTableaux » c'est un tableau de chaines de caractères

E: « lesRefDesTableaux » c'est un tableau de Ref

Module M3.6 : Bilan et Exercices

- **Bilan**

- **Exercices**

Bilan du MOOCEBFR3

- MOOCEBFR3 : **Ingénierie des Données du Big Data: SGBD Objet relationnel et SQL3 Oracle**
  - **Module M3.1** : Introduction, les types de base, les types abstraits
  - **Module M3.2** : Notion d'identité et les collections dans SQL3 Oracle
  - **Module M3.3** : Les tables objets, les liens d'associations
  - **Module M3.4** : Lien d'héritage et le Mapping objet des tables relationnelles SQL2 via les vues
  - **Module M3.5** : Gestion des objets volumineux
  - **Module M3.6** : PLSQL étendu aux objets complexes et Mapping Objet relationnel dans Java/JDBC
  - **Exercices**